
Méthodes et outils de développement logiciel

Développement avancé en C++ moderne

Isima

ISIMA

Université Clermont Auvergne

Bruno Bachelet / Loïc Yon – 2025-2026

- Rappels de l'année dernière (+ compléments)
 - ❑ Concepts de programmation objet
 - ❑ UML (diagrammes de classe)
 - ❑ Spécificités du langage C++

- Conversion et RTTI
 - ❑ Opérateurs de conversion
 - ❑ Mécanisme *Run-Time Type Information*

- Généricité: les *templates*
 - ❑ Spécificités du langage C++
 - ❑ Introduction des «concepts»
 - ❑ Spécialisation statique

- Gestion des exceptions
 - ❑ Garanties de reprise sur erreur
- Sémantique de mouvement
 - ❑ Principe et catégories de valeurs
 - ❑ Opérateurs de mouvement
- Pointeurs intelligents (*Smart Pointers*)
 - ❑ Réduction des risques d'erreurs
 - ❑ Pointeurs à propriété unique ou partagée
- Bibliothèque standard
 - ❑ Concepts associés (itérateur, foncteur)
 - ❑ Expressions lambda
 - ❑ STL: conteneurs et algorithmes

- Patrons de conception (*Design Patterns*)
 - Réutilisabilité dans la phase de conception
 - Recettes à des problématiques récurrentes

- Programmation parallèle (*Multithreading*)
 - Création de *threads*
 - Synchronisation
 - Abstraction (futurs et promesses)

- Métaprogrammation (*Template Metaprogramming*)
 - Faire travailler le compilateur
 - Exemples (évaluation partielle, métafonctions...)

PARTIE I

Rappels sur la programmation objet

Bruno Bachelet

Loïc Yon

- Définitions
 - Objet
 - Classe
- Formalisme UML
- Relations entre classes
 - Héritage
 - Agrégation
 - Association

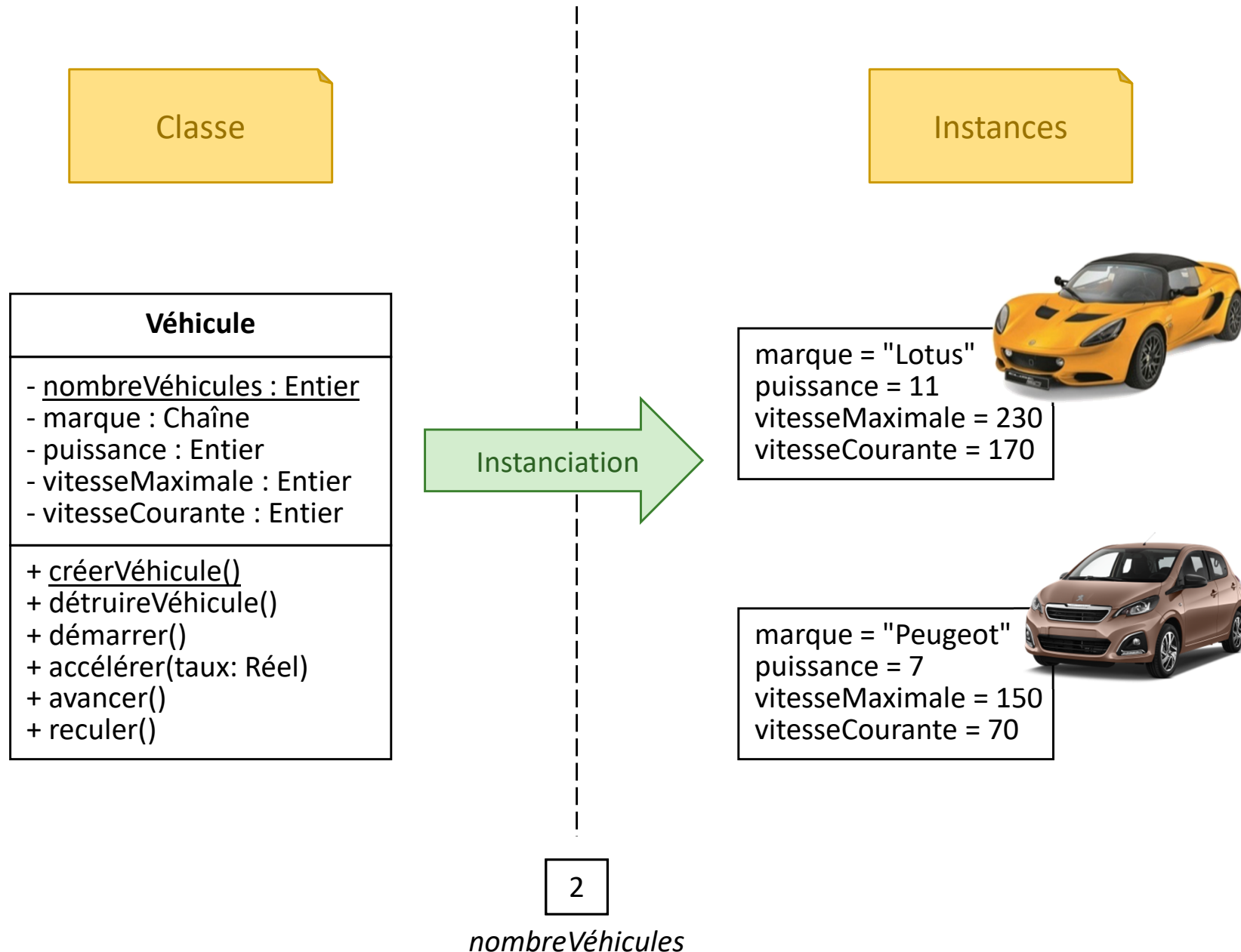
■ Objet

- ❑ Entité cohérente rassemblant des données et le code travaillant sur ces données
- ❑ Données = attributs
- ❑ Code = méthodes

■ Classe

- ❑ Fabrique à objets, i.e. une donnée qui décrit des objets
- ❑ Représente une catégorie d'objets

Exemple d'instanciation



- Attributs d'instance: une valeur par objet
 - Exemple: vitesse / couleur d'un véhicule
- Attributs de classe: une valeur par classe
 - Partagés par tous les objets de la classe
 - Exemple: nombre de véhicules présents à un instant donné
- Méthode d'instance: agit sur un objet particulier
 - Exemple: accélérer, freiner
- Méthode de classe: agit sur toute la classe
 - Exemple: ajouter un nouveau véhicule

- Encapsulation

- Protection des attributs
- Interface de communication

- Héritage

- Relation de généralisation / spécialisation
- Factorisation de données et comportement

- Polymorphisme

- Réponse spécifique à un message commun

- Séparation forte entre interface et implémentation
- Interface: partie visible d'un objet
 - ❑ Ensemble de messages paramétrables
 - ❑ Communiquer avec un objet = envoi de messages
 - ❑ Dans la pratique: appel direct de méthode
- Implémentation: partie cachée d'un objet
 - ❑ Attributs
 - ❑ Code des méthodes
- Intérêt: principe d'abstraction
 - ❑ Modification de l'implémentation d'un objet sans effet visible
 - ❑ Tant que l'interface n'est pas modifiée
⇒ aucune conséquence pour l'utilisateur

Relations fondamentales entre classes

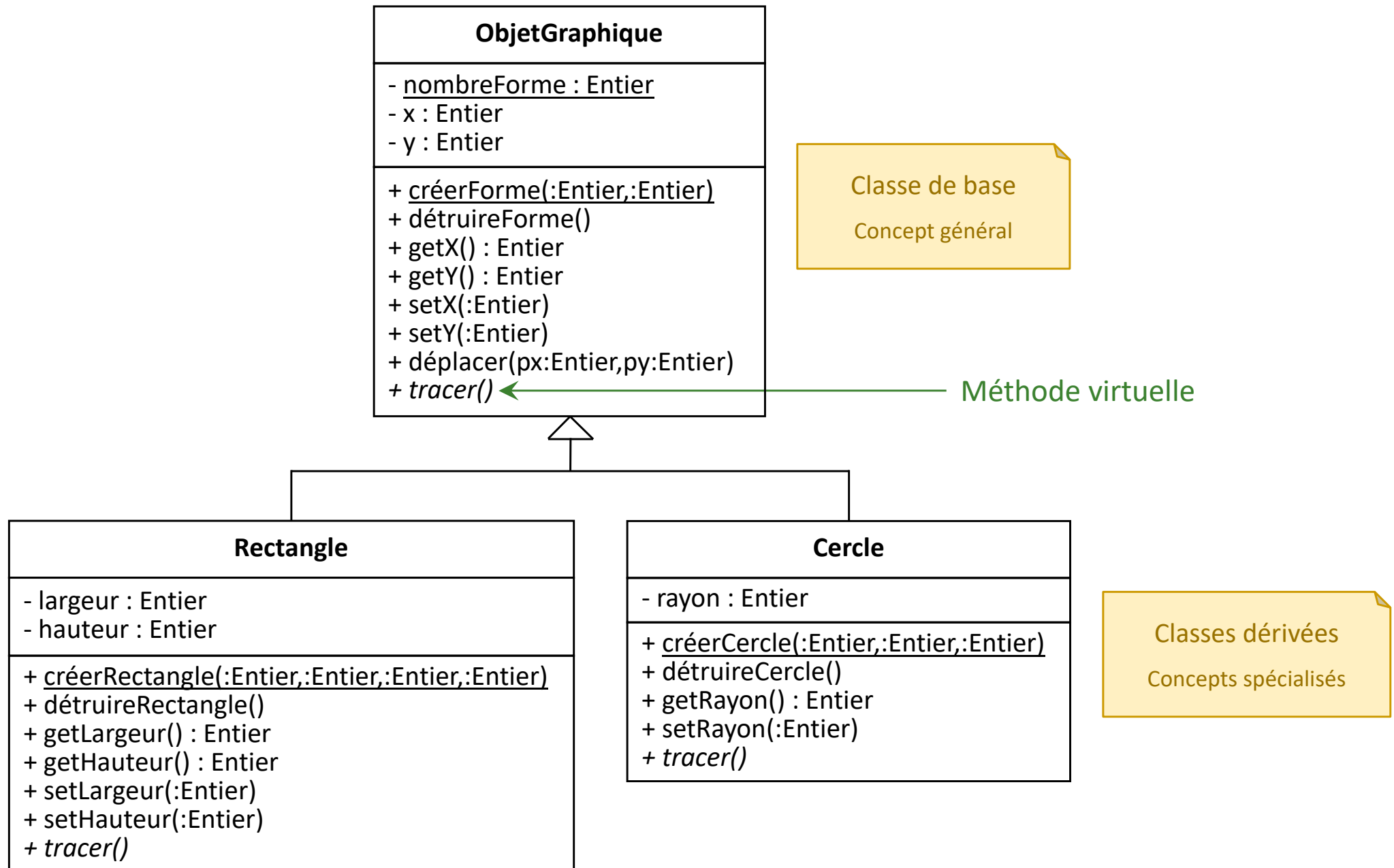
- 3 relations fondamentales
 - Héritage: généralisation / spécialisation
 - Symbolisé par «est une version spécialisée de» («*is a*»)
 - Agrégation / composition
 - Symbolisé par «contient», «regroupe» («*has a*»)
 - Association: communication
 - Symbolisé par «communique avec» («*uses a*»)
- Il existe d'autres relations mais celles-ci sont unanimement reconnues

- Concept naturel de généralisation / spécialisation
 - ❑ Classes organisées sous forme d'arborescence

- Vocabulaire
 - ❑ Classe spécialisée = sous-classe, classe fille / dérivée
 - ❑ Classe générale = super-classe, classe mère
 - ❑ La classe spécialisée dérive de sa classe mère

- Idée fondamentale
 - ❑ Classe *B* dérivant de classe *A*
 - ❑ *B* hérite de tous les attributs et méthodes de *A*
 - ❑ *B* ajoute ses propres attributs et méthodes

Exemple d'héritage



- Reprend les caractéristiques de la classe mère
 - ❑ Attributs et méthodes
 - ❑ Sauf généralement les constructeurs (cas particulier)
- Ajoute / modifie les siennes
 - ❑ Attributs et méthodes
- Peut répondre à de nouveaux messages
- Peut répondre différemment aux messages de la classe mère
 - ❑ Grâce au polymorphisme
- Attention à l'héritage des membres de classe
 - ❑ Ils ne sont pas «dupliqués» dans la classe fille
 - ❑ Ils sont uniques et propres à la classe mère
 - ❑ Mais sont accessibles depuis la classe fille

- Principe de substitution de Liskov
 - ❑ Partout où un objet de la super-classe est utilisé on peut le remplacer par un objet d'une sous-classe
- Construction d'un système *ex nihilo*
 - ❑ Identifier tous les composants
 - ❑ Factoriser les caractéristiques communes entre classes
 - ❑ Généraliser
- Extension d'un système existant
 - ❑ Identifier les différences avec les classes existantes
 - ❑ Ajouter les nouvelles classes dans l'arbre d'héritage
 - ❑ «Programmation différentielle»

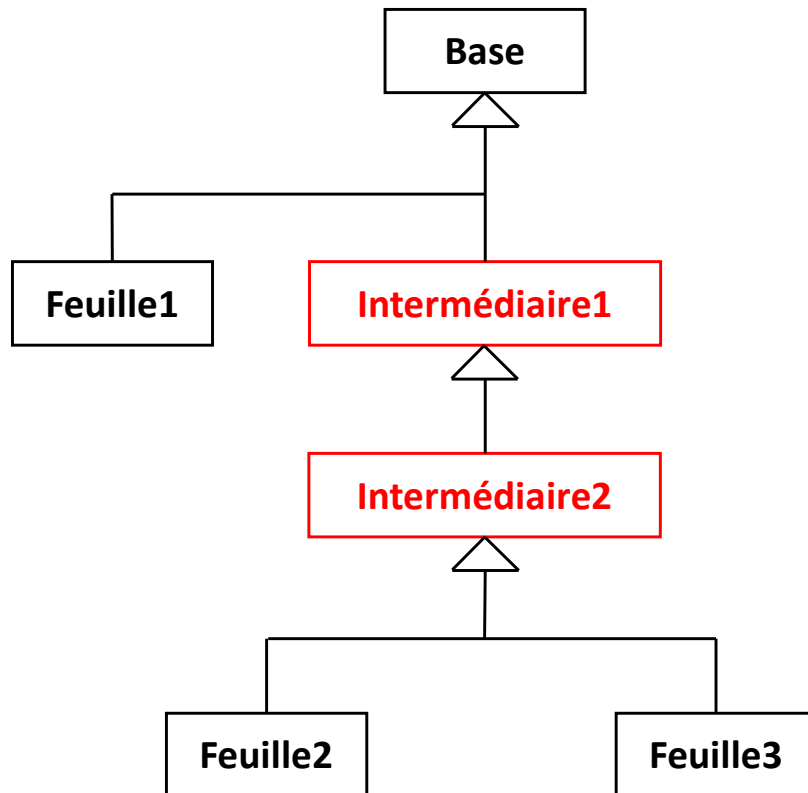
- Classe abstraite = classe qui ne peut pas être instanciée
 - ❑ Définit en général au moins une méthode abstraite
 - Sans implémentation
 - ❑ Peut définir des attributs

- Utilisée comme super-classe d'une hiérarchie
 - ❑ Exemple: *Véhicule*, *ObjetGraphique*
 - Aucun intérêt (ou sens) d'avoir des instances
 - Instances créées dans les classes dérivées
 - Support pour le polymorphisme

- Classe abstraite pure: modélise un «concept»
 - ❑ Toutes les méthodes sont abstraites
 - ❑ Similaire à une interface

- ❑ Partage de code
 - Réutilisabilité et fiabilité
 - Code des classes les plus hautes dans la hiérarchie utilisé plus souvent $\Rightarrow$ fiabilisation plus rapide
- ❑ Modélisation d'un concept naturel
- ❑ Quantité de code source réduite (factorisation)
- ❑ Maintenance facilitée
 - Héritage = code factorisé
 - Modification de l'implémentation d'une classe sans impact
 - ❑ Sur la hiérarchie d'héritage
 - Modification de l'interface d'une classe sans impact
 - ❑ Sur ses ancêtres

Dangers de l'héritage (1/2)



Exemple d'une classe
intermédiaire superflue

- Attention à la hiérarchie
 - Trop lourde, elle peut nuire à l'efficacité et à la compréhension du code
 - Ici, une classe intermédiaire peut être inutile
 - Classification trop fine
 - Couches de programmation différentielle
- Solution
 - Fusionner *Intermédiaire1* et *Intermédiaire2*

- Violation du principe d'encapsulation
 - Accès aux membres protégés de la classe mère
 - 2 niveaux d'interface
 - Violation (théorique): hériter pour accéder aux membres
 - Problèmes de maintenabilité

- Héritage de construction
 - Dériver sans respecter la généralisation / spécialisation
 - Attention à respecter le principe de Liskov
 - Exemple: *Rectangle* hérite *Ligne* pour utiliser son tracé ⇒ erreur !
 - Souvent, l'agrégation est plus adaptée
 - Dériver alors qu'un attribut suffirait
 - Dériver des classes d'animaux en raison de la couleur du pelage
 - Manque de discrimination fonctionnelle

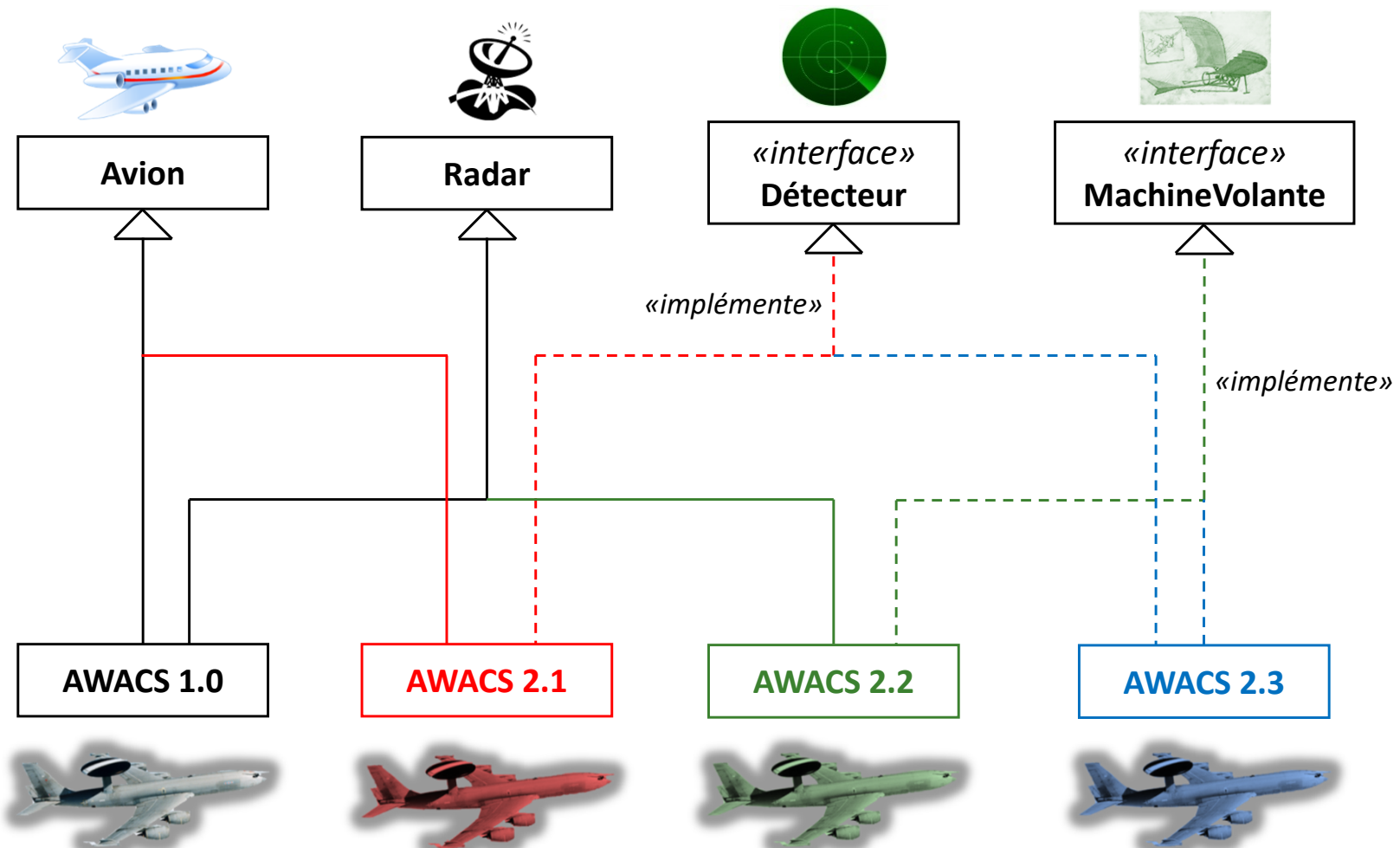
- Alternative à l'héritage multiple
 - ❑ Classe = interface + implémentation
 - ❑ Héritage multiple = problèmes dans l'héritage des implémentations

- Interface
 - ❑ Ensemble de méthodes abstraites
 - ❑ Similaire à une classe abstraite pure sans attribut
 - ❑ Définit une fonctionnalité (e.g. «*Comparable*» en Java)

- Vocabulaire
 - ❑ Une classe implémente une interface

- Héritage multiple vs. interfaces multiples
 - ❑ Hériter du concept primordial
 - ❑ Implémenter des interfaces

Exemple: AWACS



Pointillés = implémentation d'interface

- Une même méthode prend plusieurs formes
- Forme faible: la surcharge de nom («*overload*»)
 - ❑ Même nom pour plusieurs méthodes
 - ❑ Différence sur les paramètres: nombre et type
 - ❑ Exemple-type: surcharge des opérateurs
 - ❑ Pas vraiment un concept objet
- Forme forte: le polymorphisme dynamique («*override*»)
 - ❑ Méthode redéfinie dans une sous-classe
 - ❑ Comportement différent le long d'une hiérarchie
 - ❑ Paramètres strictement identiques
 - ❑ Différence sur le type véritable de l'objet
 - ❑ Exemple-type: affichage d'une liste hétérogène d'objets

- Souvent utilisé dans les agrégats
- Nécessite
 - Une hiérarchie de classes
 - Voir l'héritage
 - Une méthode virtuelle
 - Table des méthodes virtuelles
- Utilise la compatibilité ascendante des pointeurs (et références)
 - Classe *B* dérivant de classe *A*
 - *ptrA*: pointeur/référence sur une instance de *A*
 - *ptrB*: pointeur/référence sur une instance de *B*
 - *ptrA* ← *ptrB* est valide
- Repose sur la liaison différée (*dynamic dispatch*)
- Les interfaces apportent aussi le polymorphisme dynamique

- Modélisation du groupage
 - Appartenance
 - «est composé de»

- Agrégation vs. composition
 - Agrégation: agrégé indépendant de l'agrégéant
 - Composition: vie de l'agrégé dépend de l'agrégéant

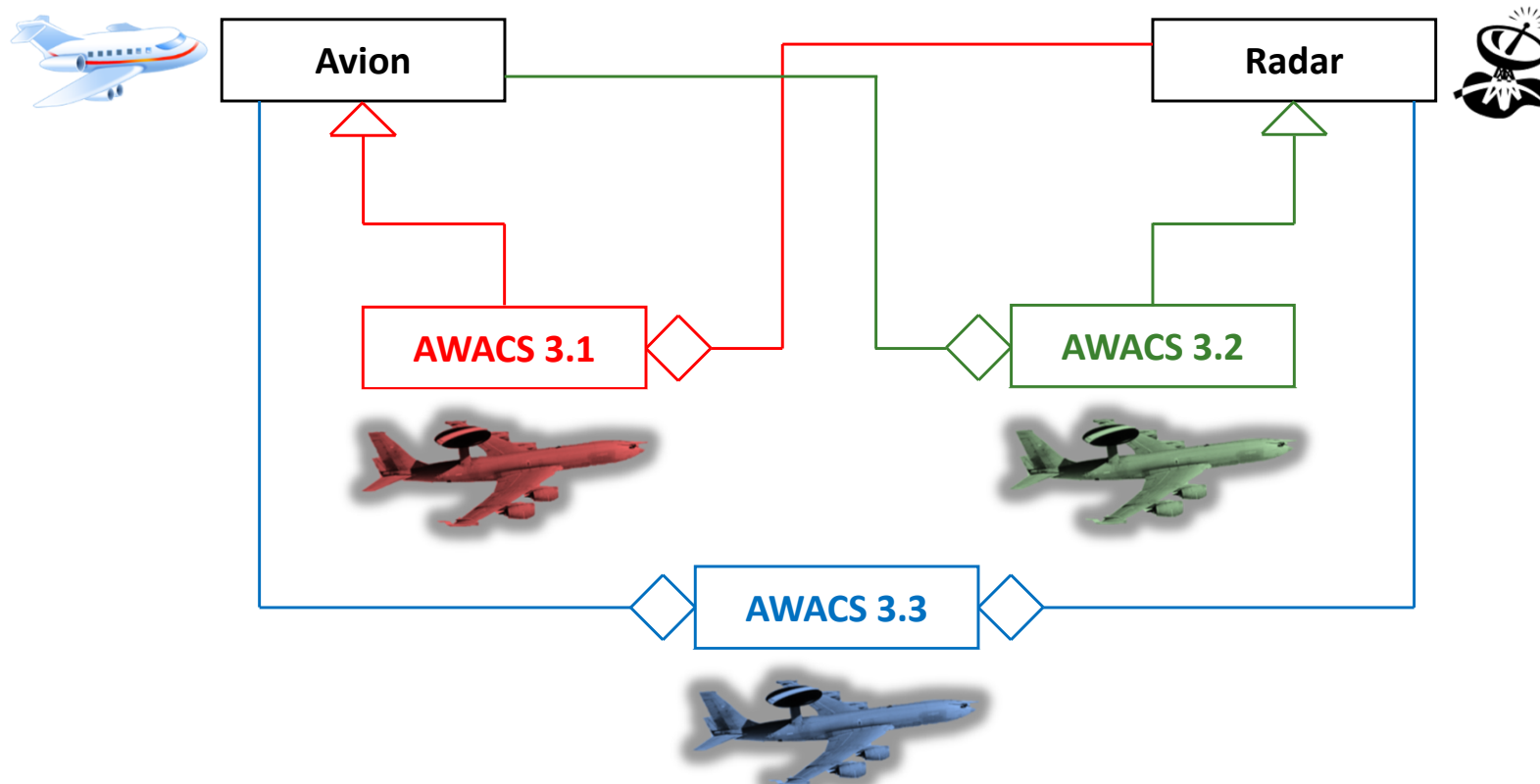
- Modèle naturel des conteneurs
 - Souvent, cardinalité 1-N
 - 1 agrégéant contient N agrégés

- Utiliser la composition pour remplacer l'héritage
 - Une classe encapsule une autre plutôt que d'en hériter
 - Attribution de caractéristiques sans lien de type
- Permet d'éviter un héritage conceptuellement bancal
 - Une erreur classique
- Evite un accès aux données membres
 - Respecte mieux la notion d'interface
 - Respect de l'encapsulation
- Parfois appelé «délégation» (cf. *GoF – Gang of Four*)
 - Les messages envoyés à l'objet qui encapsule sont délégués à l'objet encapsulé

Composition vs. héritage (2/2)

■ Exemple: AWACS

- ❑ *Avion* contenant un *Radar* (AWACS 3.1)
- ❑ *Radar* porté par un *Avion* (AWACS 3.2)
- ❑ Ensemble comprenant un *Avion* et un *Radar* (AWACS 3.3)



- Modélise des relations plus «floues»
 - ❑ «utilise», «est associé à», «communique avec»

- Caractéristiques
 - ❑ Habituellement nommée
 - ❑ Cardinalités M-N

- Association vs. agrégation
 - ❑ L'agrégation est une forme d'association
 - ❑ Pas toujours évident de les différencier
 - ❑ Agrégation: notion de parties / décomposition
 - ❑ Les moyens d'implémentation sont les mêmes
 - Attributs objets, références ou pointeurs

PARTIE II

Rappels de C++

Bruno Bachelet

Loïc Yon

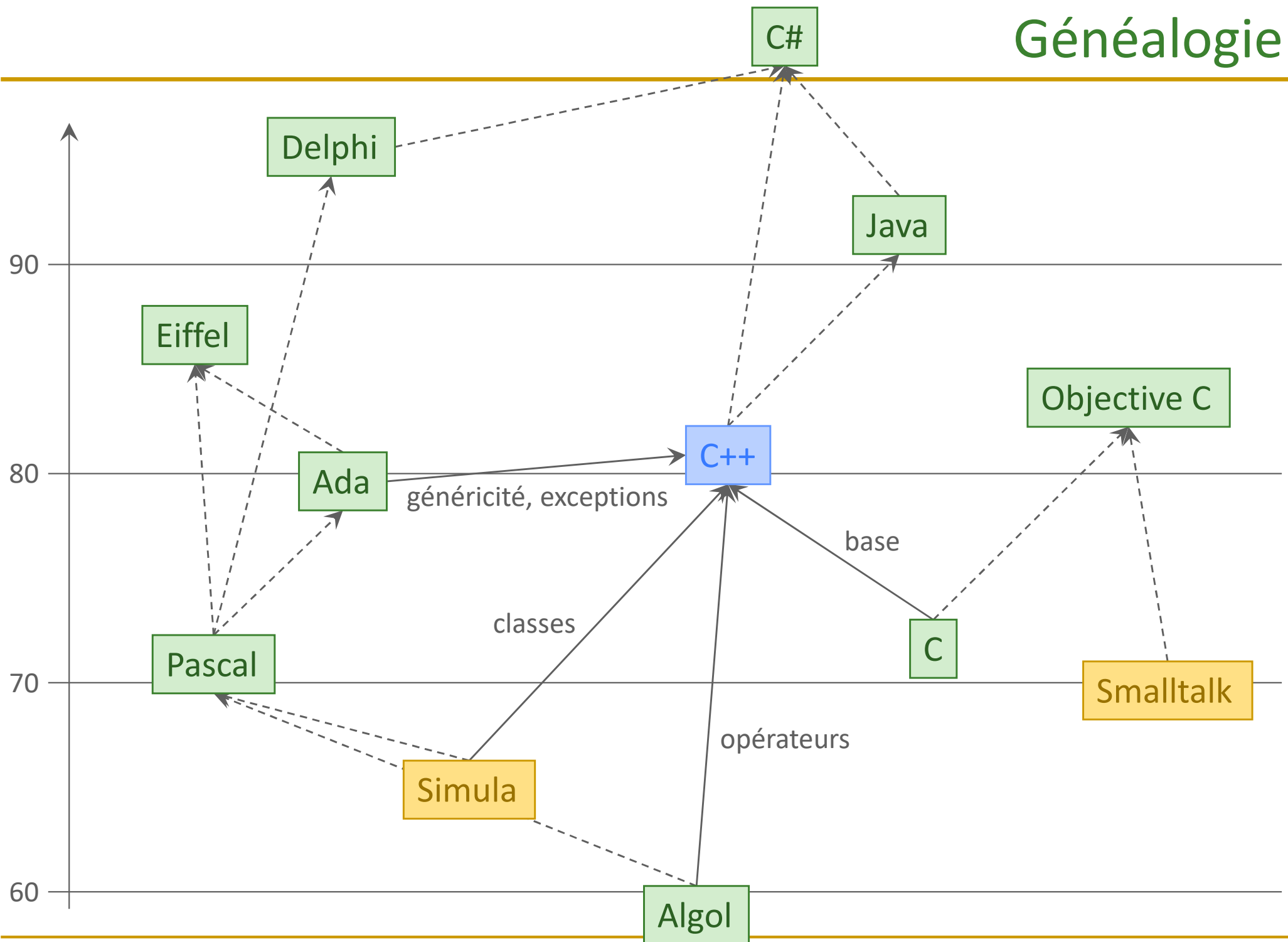
- Caractéristiques générales
 - ❑ Historique
 - ❑ Héritage des autres langages

- POO en C++
 - ❑ Définition d'une classe
 - ❑ Cycle de vie des objets
 - ❑ Relations entre classes

- Autres concepts
 - ❑ Surcharge d'opérateurs
 - ❑ Généricité
 - ❑ Exceptions

- Origines
 - ❑ Travaux de Bjarne Stroustrup (AT&T Bell)
 - ❑ «*C with classes*» (80) → C++ (83)
 - ❑ Normalisation en 98 (ISO/IEC 98-14882), norme C++98
 - ❑ A partir de 2003, norme C++03
 - ❑ A partir de 2011, cycle de 3 ans
 - ❑ C++11, C++14, C++17, C++20...
- Langage orienté objet (← SIMULA 67)
 - ❑ Typage fort
 - ❑ Maintien des types primitifs et des fonctions
- Support de la généricité et des exceptions (← ADA 79)
- Surcharge des opérateurs (← ALGOL 68)

Généalogie



- En C, paramètres uniquement passés par valeur
 - Passage en mode *in/out* $\Rightarrow$ «passage par adresse»
 - Passe (par valeur) l'adresse de la variable
 - Conséquences
 - Code peu lisible, passage de pointeurs, source d'erreurs

- En C++, utilisation de références (&)
 - Référence = nouvel alias d'une variable
 - Utilisation identique à une variable
 - Passage par référence simulé par pointeur masqué
 - Sauf pour les méthodes *inline*

■ A la mode C

```
void swap(int * a, int * b) {  
    int c = *b;  
    *b = *a;  
    *a = c;  
}
```

```
int main() {  
    int i = 5;  
    int j = 6;  
  
    swap(&i,&j);  
}
```

■ A la mode C++

```
void swap(int & a, int & b) {  
    int c = b;  
    b = a;  
    a = c;  
}
```

```
int main() {  
    int i = 5;  
    int j = 6;  
  
    swap(i,j);  
}
```

■ Avantages

- ❑ Code plus lisible
- ❑ Appel plus simple
- ❑ Moins d'erreurs
- ❑ Efficace

■ Inconvénients

- ❑ Syntaxe ambiguë à cause de «&»
- ❑ Peu évident à comprendre au départ

■ Déclaration d'une référence

- ❑ Se déclare «comme» un pointeur
- ❑ Se comporte comme un alias sur l'objet
- ❑ Nécessite un objet référencé à la déclaration
- ❑ Ne peut changer d'objet par la suite

```
int i = 5;  
int & j = i;  
j = 4; // maintenant i = 4 !
```

■ Référencer quoi ?

- ❑ Une référence non constante est toujours liée à une variable
- ❑ Une référence constante peut être liée à une constante
 - `const int & j = 4;`
- ❑ La référence nulle n'existe pas !

Règles d'usage des types: *const*, référence ?

Passage d'arguments

	Type primitif <i>T</i>	Classe <i>C</i>
Argument variable	<i>T</i> & arg	<i>C</i> & arg
Argument constant	<i>T</i> arg ou const <i>T</i> & arg	const <i>C</i> & arg

Retour de valeur

	Type primitif <i>T</i>	Classe <i>C</i>
Retour (mode lecture) d'un attribut	<i>T</i> m(...); ou const <i>T</i> & m(...);	const <i>C</i> & m(...);
Retour (mode lecture/écriture) d'un attribut	<i>T</i> & m(...);	<i>C</i> & m(...);
Retour d'un résultat produit par une méthode *	<i>T</i> m(...);	<i>C</i> m(...);

* Retour d'une variable locale toujours par copie

- En C++, utilisation de «**new**» et «**delete**»
 - Plus d'appel explicite à «**malloc**» et «**free**»
- Pour allouer / libérer une donnée

```
int * p = new int;  
...  
delete p;
```
- Pour allouer / libérer un tableau

```
int * p = new int[10];  
...  
delete[] p;
```
- Réalisent aussi la construction / destruction
 - **new** = allocation mémoire + appel constructeur
 - **delete** = appel destructeur + libération mémoire
- Mot-clé «**nullptr**» pour le pointeur nul (depuis C++11)

- En C: `printf` / `scanf` (et consorts)
- En C++: mécanisme de flux
 - Bibliothèque standard $\Rightarrow$ *namespace* «`std`»
 - Flux standards: `std::cin` / `std::cout` / `std::cerr`
 - Inclusion de `<iostream>` (ou `<fstream>` pour les fichiers)
- Pour lire depuis un flux (classe de base «`istream`»)

```
double x; int j;  
...  
flux >> x >> j;
```
- Pour écrire dans un flux (classe de base «`ostream`»)

```
double x; int j;  
...  
flux << x << " + " << j << " = " << (x + j) << std::endl;
```

Déclaration d'une classe (1/2)

- Mot-clé «`class`» ou «`struct`»
- Contient les attributs et les prototypes des méthodes
 - `class`: membres privés par défaut
 - `struct`: membres publics par défaut
- Modificateurs d'accès
 - `public`: membre accessible par tous
 - Réservé exclusivement aux méthodes de l'interface
 - `private`: membre accessible aux méthodes de la classe
 - Pour les attributs
 - Pour les méthodes non destinées à l'utilisateur
 - `protected`: membre accessible aux méthodes de la classe et de ses sous-classes
 - Assouplit l'accès privé à des fins de redéfinition dans les sous-classes
- Modificateur «`static`»
 - Définit un membre de classe

Déclaration d'une classe (2/2)

Point
- absc : Entier - ordo : Entier - <u>nb_points</u> : Entier
+ «constructeur» Point (:Entier,:Entier) + getX () : Entier + getY () : Entier + move (dx:Entier,dy:Entier) + moveTo (x:Entier,y:Entier) + <u>nbPoints</u> () : Entier

```
class Point {  
    private:  
        int absc;  
        int ordo;  
        static int nb_points;  
  
    public:  
        Point(int,int);  
        int getX() const;  
        int getY() const;  
        void move(int,int);  
        void moveTo(int,int);  
        static int nbPoints();  
};
```

Attention !

Définition d'une classe

```
Point::Point(int x, int y) { // Constructeur
    absc = x;
    ordo = y;
    nb_points++;
}
```

```
int Point::getX() const { return absc; } // Accesseur
```

```
void Point::move(int dx, int dy) { // Méthode d'instance
    absc += dx;
    ordo += dy;
}
```

```
int Point::nbPoints() // Méthode de classe
{ return nb_points; }
```

```
int Point::nb_points = 0; // Attribut de classe
```

- Appel méthode $\Rightarrow$ coût d'exécution
- Parfois, dommage d'utiliser un appel de méthode
 - Pour récupérer la valeur d'un attribut
 - Pour un traitement simple
- Méthode «*inline*»: développée «comme» une macro
 - S'applique aussi aux fonctions
- On ne force pas une méthode à être «*inline*», on demande et le compilateur décide
- Avantage
 - Rapidité d'exécution (coût appel + optimisation supplémentaire)

■ Inconvénients

- ❑ Augmentation taille exécutable
 - A utiliser donc sur des méthodes courtes
- ❑ Implémentation avec la déclaration de la classe
 - Dans un fichier d'entête

■ Implémentation toujours dans l'entête

- ❑ Définition avec la déclaration ⇒ méthode souhaitée «*inline*»

```
class Point {  
    ...  
    int getX() const { return absc; } // "inline" implicite  
    ...  
};
```

- ❑ Utilisation mot-clé «*inline*» (en dehors de la déclaration)

```
inline int Point::getX() const { return absc; }
```

Structure du code source

■ Fichier entête

- ❑ Déclaration de la classe
- ❑ Définitions méthodes «*inline*»

```
#ifndef CLASSE_HPP_ZZ2025
#define CLASSE_HPP_ZZ2025

// Includes
// Déclarations anticipées

class Classe {
    // Attributs
    // Prototypes méthodes
    // Méthodes inline
};
#endif
```

■ Fichier implémentation

- ❑ Définitions attributs de classe
- ❑ Définitions méthodes

```
#include "classe.hpp"

// Initialisation des
// attributs de classe

// Définitions des méthodes
// externalisées
```

S'assurer que l'identifiant du garde de l'entête est unique

- ❑ Rajouter un nom, une date...
- ❑ Utiliser un UUID

- Construction
 - Réserveation mémoire
 - Appel d'un constructeur

- Vie
 - Appel des méthodes

- Destruction
 - Appel du destructeur
 - Libération mémoire

- Rôle: initialiser les objets
- Syntaxe
 - ❑ Même nom que la classe
 - ❑ Pas de type de retour
 - ❑ Surcharge à volonté
 - ❑ Une particularité: la liste d'initialisation
- Exemples

```
Point::Point() { ... }  
Point::Point(int x, int y) { ... }  
Point::Point(const Point & p) { ... }
```

■ Syntaxe

- ❑ *nom_classe(...)* : *liste_initialisation* { ... }
- ❑ Liste = *nom_attribut(valeur)*, *nom_attribut(valeur)*...
- ❑ Les valeurs peuvent être des expressions
 - Calcul, appel de fonction...

■ Rôle: initialisation des attributs d'un objet

- ❑ Même sans liste, initialisation avant le bloc de code

■ Construction de chacun des attributs

- ❑ Dans l'ordre de déclaration
- ❑ Donc, il faut lister les attributs dans l'ordre de déclaration
- ❑ Si un attribut est omis dans la liste ⇒ construction par défaut
- ❑ Les attributs de type référence obligatoirement dans la liste

- Respecter l'ordre des attributs

```
class Rationnel {  
    private:  
        int num;  
        int den;  
  
    public:  
        Rationnel(int n=0, int d=1)  
        : den(d), num(n)  
        {}  
};
```

- Solution

```
Rationnel(int n=0, int d=1)  
: num(n), den(d)  
{}
```

- Initialisation plus complexe

- ❑ Ajout attribut «dist» dans «Point»
- ❑ Distance du point à l'origine

- Dans le corps du constructeur

```
Point::Point(int x, int y)  
: absc(x), ordo(y)  
{  
    dist = sqrt(x*x + y*y);  
}
```

- Ou dans la liste d'initialisation

```
Point::Point(int x, int y)  
: absc(x), ordo(y),  
  dist(sqrt(x*x + y*y))  
{}
```

■ Implicite

- ❑ `A a;` $\Rightarrow$ constructeur par défaut «`A()`»
- ❑ `A a = 4;` $\Rightarrow$ constructeur «`A(int)`»

■ Explicite

- ❑ `A a(4);` $\Rightarrow$ constructeur «`A(int)`»
- ❑ `f(A(4));` $\Rightarrow$ constructeur «`A(int)`» (objet créé à la volée)
- ❑ `A * a = new A(4);` $\Rightarrow$ constructeur «`A(int)`»

■ Attention à la syntaxe d'une construction explicite par défaut

- ❑ `A a();` $\Rightarrow$ déclaration fonction «`a()`»
- ❑ Pas de souci avec «`new`» et «`throw`»

■ Cas des types primitifs

- ❑ Paramètre *template* «`T`»: `T x = T();`
- ❑ Si `T = A` $\Rightarrow$ appel constructeur par défaut «`A()`»
- ❑ Si `T = int` $\Rightarrow$ initialise «`x`» à zéro
- ❑ Rappel: `int x;` $\Rightarrow$ aucune garantie que «`x`» vaut 0

Construction par liste de valeurs (1/2)

- ❑ C++03: possibilité d'initialiser des «agrégats» par liste
 - Agrégat = tableau ou classe avec restrictions
 - ❑ `int t[] = {7,8,9};`
 - Classe «agrégat» = attributs publics, pas de constructeur
 - ❑ `class Paire { public: int x; double y; };`
 - ❑ `Paire p = {3,7.0};`

- ❑ C++11: généralisation à n'importe quelle classe
 - ```
class Paire {
 private: int x; double y;
 public: Paire(int a, double b) : x(a), y(b) {}
};
```
  - `Paire p = {3,7.0};` ⇒ appel constructeur

# Construction par liste de valeurs (2/2)

---

- ❑ Autres syntaxes possibles
  - `Paire p{3,7.0};`
  - `return {3,7.0};` ⇒ déduction du type à l'aide de la signature
- ❑ Permet d'éviter certaines ambiguïtés de syntaxe
  - `Paire p();` ⇒ interprété comme une fonction
  - `Paire p{};` ⇒ interprété comme une variable
- ❑ Objectif: uniformiser l'initialisation des variables
- ❑ Mais problème: des subtilités subsistent !
  - `std::vector<int> v1(10,20);` ⇒ vecteur de 10 éléments
  - `std::vector<int> v2{10,20};` ⇒ vecteur de 2 éléments
  - La raison: l'initialisation par liste (*initializer list*)

## ■ Conseils

- ❑ Utiliser «`{}`» quand la volonté est d'initialiser par une liste de valeurs
  - `std::vector<int> v{10,20}` ⇒ vecteur contient les valeurs 10 et 20
- ❑ Utiliser «`()`» quand la volonté est d'initialiser par une «procédure»
  - `std::vector<int> v(10,20)` ⇒ vecteur contient 10 fois la valeur 20
- ❑ Utiliser «`{}`» pour la construction par défaut

```
class A {
 private: std::string s;
 public: A() : s{} {};
};
```
- ❑ Pour éviter des complications, privilégier l'usage de «`()`» avec...
  - Le mot-clé «`auto`» pour la déduction automatique de type
  - Un paramètre *template*

- Quatre types de durées de vie (*storage duration*)
  - ❑ Statique: variable globale, attribut de classe, variable locale statique
  - ❑ *Thread*: variable «globale» par *thread* (mot-clé «`thread_local`»)
  - ❑ Automatique: variable locale sur la pile
  - ❑ Dynamique: variable allouée sur le tas
    - `new` = allocation mémoire + appel constructeur
    - `delete` = appel destructeur + libération mémoire
  
- Gestion mémoire
  - ❑ Statique, *thread* et automatique: par le système
  - ❑ Dynamique: par le développeur

## ■ Moment de la construction

- ❑ Variables globales et attributs de classe: avant l'exécution du «**main**»
- ❑ Variables *thread-local*: à l'initialisation du *thread*
- ❑ Variables locales: à l'entrée dans le bloc
  - Variables locales statiques: à la 1<sup>ère</sup> entrée
- ❑ Variables dynamiques: à l'exécution de «**new**»

## ■ Moment de la destruction

- ❑ Variables statiques: après la sortie du «**main**»
  - Même chose pour les variables locales statiques
- ❑ Variables *thread-local*: à la fin du *thread*
- ❑ Variables locales sur la pile: à la sortie du bloc
- ❑ Variables dynamiques: à l'exécution de «**delete**»

- Utilisation du mot-clé «**const**» en fin de prototype
- Indique les méthodes ne modifiant pas l'objet
  - Qui ne modifient pas les attributs
- Limité aux méthodes d'instance
- Avantages
  - Seules méthodes utilisables sur un objet constant
    - Une méthode «non constante» ne peut pas être exécutée
  - La méthode ne peut pas modifier les attributs
  - Contrôlé à la compilation
- Signification plus subtile
  - «**const**» fait partie de la signature
  - Possibilité de définir deux versions



- Définition d'accesseurs (version 1 – recommandée)

```
class Exemple {
 private:
 string s;

 public:
 const string & getS() const { return s; }
 void setS(const string & x) { s = x; }
};
```

- Utilisation d'accesseurs

```
Exemple e1;
const Exemple e2;
```

```
e1.setS("nawouak"); ⇒ ok
```

```
e2.setS("nawouak"); ⇒ problème
```

```
std::cout << e1.getS() << std::endl; ⇒ ok
```

```
std::cout << e2.getS() << std::endl; ⇒ ok
```

- Définition d'accesseurs (version 2 – non recommandée)

```
class Exemple {
 private:
 string s;

 public:
 const string & getS() const { return s; }
 string & getS() { return s; }
};
```

- Utilisation d'accesseurs

```
Exemple e1;
const Exemple e2;
```

```
e1.getS() = "nawouak"; ⇒ ok
```

```
e2.getS() = "nawouak"; ⇒ problème
```

```
std::cout << e1.getS() << std::endl; ⇒ ok
```

```
std::cout << e2.getS() << std::endl; ⇒ ok
```

- Regrouper un ou plusieurs objets dans un autre = les attributs
- Trois manières d'agréger / trois types d'attributs
  - Attribut objet: construit en même temps que l'objet
  - Attribut référence: initialisation obligatoire dans le constructeur
    - Pas de changement par la suite
  - Attribut pointeur: peut être initialisé n'importe quand
    - Attention à la forme normale de Coplien
    - Si la mémoire de l'attribut est gérée par la classe
- Vie de l'objet agrégé
  - Objet construit par l'agrégeant
    - Attribut objet ou pointeur
  - Objet en provenance de l'extérieur
    - Recopie: attribut objet
    - Référence: attribut pointeur ou référence

- Une classe peut hériter d'une ou plusieurs autres
  - ❑ `class` *dérivée* : *modificateur mère1*, *modificateur mère2*...
  - ❑ Modificateur  $\Rightarrow$  limitation de visibilité
- Visibilité de l'héritage: qui voit l'héritage ?
  - ❑ `public`  $\Rightarrow$  tout le monde
  - ❑ `protected`  $\Rightarrow$  classes filles uniquement
  - ❑ `private`  $\Rightarrow$  classe uniquement
  - ❑ Perte du lien de parenté  $\Rightarrow$  plus de conversion ascendante
- Visibilité des membres de la classe mère

| Visibilité dans classe mère | Visibilité dans classe fille     |                                     |                                   |
|-----------------------------|----------------------------------|-------------------------------------|-----------------------------------|
|                             | Héritage « <code>public</code> » | Héritage « <code>protected</code> » | Héritage « <code>private</code> » |
| <code>public</code>         | <code>public</code>              | <code>protected</code>              | <code>private</code>              |
| <code>protected</code>      | <code>protected</code>           | <code>protected</code>              | <code>private</code>              |
| <code>private</code>        | <code>private</code>             | <code>private</code>                | <code>private</code>              |

- Rappel sur le modificateur «**protected**»
  - ❑ Permet de rendre un membre visible par les classes filles
  - ❑ Tout en restant caché vis-à-vis de l'extérieur
  
- Utilisation classique de l'héritage
  - ❑ Attributs «**private**» ⇒ encapsulation
  - ❑ Et héritage «**public**»
  
- Passer les attributs «**private**» en «**protected**» ?
  - ❑ Avantage: accessibles directement par les classes filles
  - ❑ Inconvénient: violation partielle de l'encapsulation
    - Problèmes de maintenabilité si héritage en cascade
    - Solution: méthodes protégées pour l'accès aux attributs privés

- Héritage privé  $\Rightarrow$  perte de l'interface
- Utilisation 1: s'approprier l'implémentation
  - ❑ Mais l'héritage n'a pas forcément de sens
  - ❑ L'agrégation peut être utilisée à la place
  - ❑ A éviter donc dans ce but
- Utilisation 2: proposer une nouvelle interface
  - ❑ Modéliser un «*wrapper*»
  - ❑ Solution possible: l'agrégation
  - ❑ Héritage privé  $\Rightarrow$  solution sans agrégation

- Rendre une méthode polymorphe (virtuelle): `virtual`
  - Virtuelle un jour, virtuelle toujours !
    - Mot-clé «`virtual`» pas nécessaire dans les sous-classes
  - Peut être redéfinie dans les sous-classes
  
- Classe abstraite en C++
  - Pas de mot-clé
  - Classe abstraite  $\Rightarrow$  au moins une méthode abstraite
  - Méthode abstraite = méthode virtuelle pure
    - Pas de code
    - `virtual type_retour nom_méthode(arguments) = 0;`
  - Redéfinition inéluctable dans les sous-classes
    - Car tant qu'une méthode est abstraite  $\Rightarrow$  pas d'instanciation

- Contrôle de la redéfinition d'une méthode (depuis C++11)
  - ❑ Lors de la redéfinition d'une méthode, une erreur est vite arrivée
  - ❑ Exemple
    - `class A { public: virtual void f(int); };`
    - `class B : public A { public: void f(double); };`
    - Compile, mais «B::f» ne redéfinit pas «A::f»
  - ❑ Mot-clé «`override`» ⇒ intention de redéfinition
    - `class B : public A`  
    `{ public: void f(double) override; };`
    - Contrôle à la compilation ⇒ erreur
  - ❑ Mot-clé «`final`» ⇒ pas de redéfinition
    - `void f(void) final` ⇒ pas de redéfinition possible de «f»
    - `class A final` ⇒ héritage de «A» impossible



# Redéfinition «par complément»

---

- Appel à une méthode de la classe mère (sans polymorphisme)
  - `classe_mère::nom_méthode(arguments)`

- Exemple: compléter l'implémentation de la classe mère

```
class Personne {
 ...
 virtual void afficher() const { cout << nom << " " << prenom; }
 ...
};
```

```
class Etudiant : public Personne {
 ...
 void afficher() const override {
 Personne::afficher();
 cout << " " << ecole;
 }
 ...
};
```

# Héritage et constructeur

- Les constructeurs ne peuvent pas être virtuels
  - ❑ Pas d'héritage (au sens classique) des constructeurs
  - ❑ Mais séquence de construction prédéfinie
- Exemple: *B* hérite de *A*
  - ❑ Construction *B*  $\Rightarrow$  construction partie *A*, puis construction attributs de *B*

```
❑ class A {
 protected: string s;

 public:
 A() { s="X"; }
 A(const string & ss) : s(ss) {}
};
```

Solution conseillée  
`A() : s("X") {}`

```
❑ class B : public A {
 private: string t;

 public:
 B() { s="Y"; t="Z"; }
 B(const string & ss, const string & tt) : A(ss), t(tt) {}
};
```

Solution conseillée  
`B() : A("Y"), t("Z") {}`

- Méthode virtuelle  $\Rightarrow$  destructeur virtuel
  - Destruction impérativement polymorphe
- Exemple

```
vector<ObjetGraphique *> v;
...
for (int i=0; i<v.size(); ++i) delete v[i];
```
- Si destructeur polymorphe
  - Appel destructeur sous-classe (e.g. `~Rectangle`)
  - Puis appel destructeur super-classe (`~ObjetGraphique`)
- Si destructeur non-polymorphe
  - Appel destructeur super-classe  $\Rightarrow$  incohérent !
- Important si ressource allouée dans le constructeur
  - Elle doit être libérée par le destructeur
  - Destructeur non-polymorphe  $\Rightarrow$  pas de libération

## ■ Duplication des attributs

- ❑ 

```
class A
{ A(...) {} };
```

```
class B : public A
{ B(...) : A(...) {} };
```

```
class C : public A
{ C(...) : A(...) {} };
```

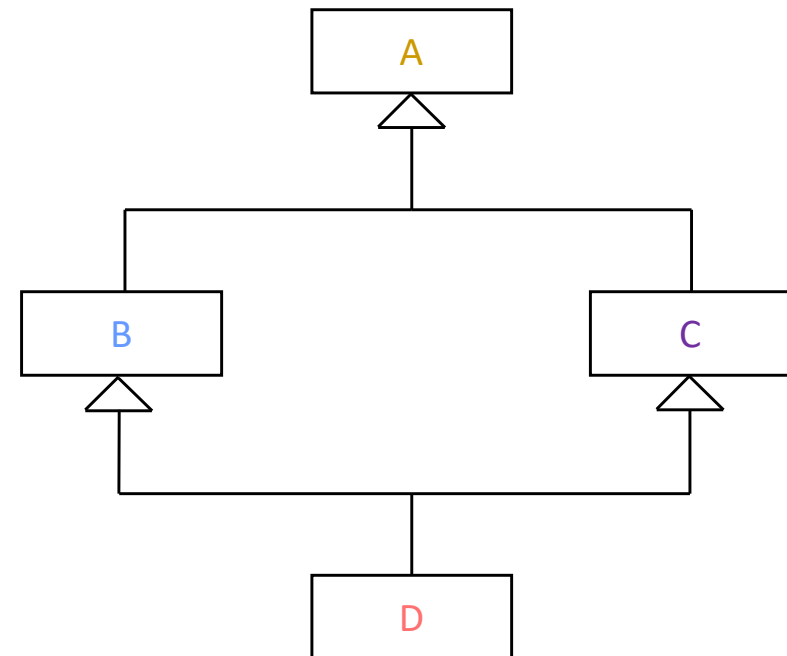
```
class D : public B, public C
{ D(...) : B(...), C(...) {} };
```

- ❑ 2 appels au constructeur de A dans D  
⇒ attributs de A dupliqués dans D

## ■ Collision des noms de membres hérités

- ❑ Si classe A définit la méthode `x()`
- ❑ `D::x()` signifie appel sur la partie A issue de l'héritage avec B ou C ?
- ❑ Distinction possible via `B::x()` ou `C::x()` ou conversion vers `B&` ou `C&`

## Héritage en diamant



## ■ Solution: héritage «virtuel»

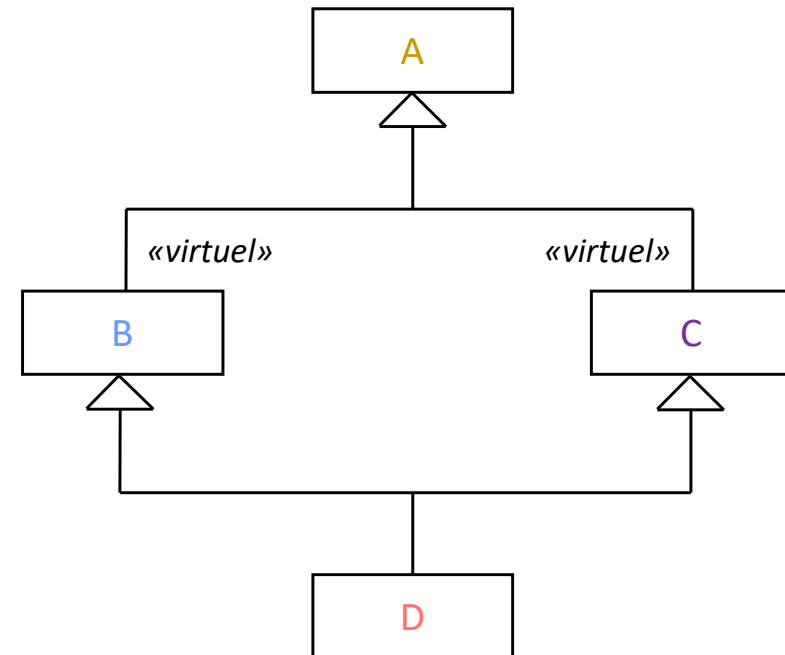
```
class A
{ A(...) {} };

class B : virtual public A
{ B(...) : A(...) {} };

class C : virtual public A
{ C(...) : A(...) {} };

class D : public B, public C
{ D(...) : A(...), B(...), C(...) {} };
```

- Une seule copie de A
- Appel explicite au constructeur de A dans D
- Dans les constructeurs de B et C, les arguments destinés à A sont ignorés



## ■ Autres solutions: héritage d'interfaces (classes abstraites en C++) ou délégation

# Réutilisation de constructeurs (1/2)

---

- Définition d'un constructeur à partir d'un autre (depuis C++11)  
⇒ évite la duplication de code

```
class Personne {
 private:
 string nom;
 string prenom;

 public:
 Personne(const string & n,
 const string & p) : nom(n), prenom(p) {}

 Personne() : Personne("Doe", "John") {}
};
```

# Réutilisation de constructeurs (2/2)

- «Héritage» des constructeurs de la classe mère (depuis C++11)

```
class Etudiant : public Personne {
 using Personne::Personne;

 private:
 string ecole;
 ...
};
```

- Tous les constructeurs sont «hérités»

- Chaque constructeur est «recréé» dans la classe fille

- Avec la même signature que la classe mère
- Appelle le constructeur de la classe mère
- Et construit par défaut les attributs de la classe fille

- Exemple

- `Etudiant e("Doe","Jane");`

- Appelle constructeur équivalent à

- `Etudiant(const string & n, const string & p)  
 : Personne(n,p), ecole() {}`

# Choix explicite des opérateurs par défaut

---

- ❑ Par défaut, une classe possède
  - Un constructeur par défaut
    - ❑ Si aucun autre constructeur n'est manuellement défini
  - Un constructeur de copie
  - Un opérateur d'affectation
- ❑ C++03: pour empêcher l'utilisation  $\Rightarrow$  déclarer privé
- ❑ C++11: choix explicite  $\Rightarrow$  «`default`» ou «`delete`»
- ❑ Exemple

```
class NonCopiable {
 NonCopiable() = default;
 NonCopiable(const NonCopiable &) = delete;
 NonCopiable & operator = (const NonCopiable &) = delete;
};
```



## ■ Constructeurs

### □ Constructeur par défaut

- `A();`

### □ Constructeur de copie

- `A(const A &);`

## ■ Affectation $\Rightarrow$ méthode

### □ `a = b` $\Leftrightarrow$ `a.operator=(b)`

### □ `A & operator=(const A & x) {` ... // Recopie de x dans "this" return `*this`; }

### □ Retour de l'objet pour le chaînage

`a = b = c`  $\Leftrightarrow$  `a.operator=(b.operator=(c))`

## ■ Opérations arithmétiques binaires $\Rightarrow$ fonctions

□  $c = a + b \Leftrightarrow c = \text{operator}+(a,b)$

```
□ A operator+(const A & x, const A & y) {
 A resultat;
 ... // Calcul x + y
 return resultat;
}
```

□ Retour par copie car objet local

## ■ Opérations logiques binaires $\Rightarrow$ fonctions

□  $\text{if } (a < b) \Leftrightarrow \text{if } (\text{operator}<(a,b))$

```
□ bool operator<(const A & x, const A & y) {
 bool resultat;
 ... // Comparaison x et y
 return resultat;
}
```

- Opérations arithmétiques unaires  $\Rightarrow$  méthodes
- Incrément préfixé  $\Rightarrow$  retour d'une référence
  - $b = ++a \Leftrightarrow b = a.operator++()$
  - ```
A & operator++() {  
    ... // Incrémentation de "this"  
    return *this;  
}
```
- Incrément postfixé $\Rightarrow$ retour d'une copie avant incrément
 - $b = a++ \Leftrightarrow b = a.operator++(0)$
 - ```
A operator++(int) {
 A copie = *this;
 ... // Incrémentation de "this"
 return copie;
}
```

- Opérateurs (binaires) de flux  $\Rightarrow$  fonctions

- `f << a`  $\Leftrightarrow$  `operator<<(f,a)`

- ```
std::ostream & operator<<(std::ostream & flux,  
                           const A & x) {  
    ... // Ecriture de x dans le flux  
    return flux;  
}
```

- Retour du flux pour le chaînage

- `f << a << b` $\Leftrightarrow$ `operator<<(operator<<(f,a),b)`

- Ne jamais passer un flux par copie

- Autres symboles surchargeables

- `()`, `[]`, `*`, `,` ...

PARTIE III

Conversion et RTTI

Bruno Bachelet

Loïc Yon

Implémentation d'une conversion (1/2)

- Possibilité 1: constructeur avec un seul argument

```
class Chaîne {  
    ...  
    Chaîne(const char * s);  
    ...  
};
```

- Fournit une conversion implicite `const char * → Chaîne`

```
void display(const Chaîne &);  
...  
display("aZeRtY"); ⇒ construction objet temporaire «Chaîne»
```

- Conversion implicite parfois non désirée

- `Vecteur::Vecteur(int n);` ⇒ conversion `int → Vecteur`

- Conversion explicite: mot-clé «`explicit`»

- `explicit Chaîne(const char * s);`

- `display("aZeRtY");` ⇒ erreur de compilation

- `display(Chaîne("aZeRtY"));` ⇒ obligation d'expliciter la conversion

Implémentation d'une conversion (2/2)

- Possibilité 2: opérateur de conversion

```
class B {  
    ...  
    public: operator A() {  
        A a;  
        ... // Conversion de "this" dans "a"  
        return a;  
    }  
};
```

- Conversion implicite par défaut

- «`explicit`» possible depuis C++11

- Comment choisir entre les deux possibilités ?

- Constructeur: nécessite l'accès au code de la classe cible
 - Pas toujours possible de prévoir les conversions *a priori*
- Opérateur: nécessite l'accès au code de la classe source
 - N'est donc pas possible pour les types et les classes tierces

- Il existe plusieurs opérateurs de conversion
- *(type)*
 - ❑ Conversion de valeurs à partir d'opérateurs
- `static_cast`
 - ❑ Conversion de pointeurs/références avec vérification à la compilation
- `dynamic_cast`
 - ❑ Conversion de pointeurs/références avec vérification à l'exécution
- `const_cast`
 - ❑ Conversion portant uniquement sur l'aspect constant
- `reinterpret_cast`
 - ❑ Conversion de pointeurs sans vérification de type

■ Opérateur hérité du C

- ❑ Mais deux syntaxes possibles
- ❑ `c = (Chaine)s;`
- ❑ `c = Chaine(s);`

■ Conversion d'objets

- ❑ Effectuée à partir des opérateurs définis par le programmeur
- ❑ Aucun opérateur $\Rightarrow$ conversion interdite

■ Conversion de types primitifs

- ❑ Opérateurs de conversion fournis par défaut

■ Conversion de pointeurs et références

- ❑ Toujours autorisée

■ Exemple

```
class Animal { ... virtual void manger(); ... };  
class Poisson : public Animal { ... void manger() override; ... };  
class Plante { ... };
```

```
Animal * girafe = new Animal();  
Animal * requin = new Poisson();  
Plante * sapin = new Plante();
```

```
Animal * animal; Poisson * poisson;
```

■ Conversions toujours autorisées

- ❑ `animal = (Animal *)sapin;` ⇒ (1) conversion fausse
- ❑ `poisson = (Poisson *)girafe;` ⇒ (2) conversion fausse
- ❑ `poisson = (Poisson *)requin;` ⇒ (3) conversion ok

■ Eviter l'utilisation de l'opérateur (*type*) pour des pointeurs/références

- ❑ Cas (1): détection possible à la compilation ⇒ opérateur «`static_cast`»
- ❑ Cas (2) & (3): détection à l'exécution ⇒ opérateur «`dynamic_cast`»

- Vérifie la conversion de pointeurs/références à la compilation
- Utilisé lors d'une conversion ascendante (*upcast*)
 - Conversion d'une classe fille vers une classe mère
⇒ toujours possible
 - `animal = static_cast<Animal *>(poisson);`
⇒ autorisée et valide
- Attention: conversion autorisée dès qu'il y a un lien d'héritage
 - `poisson = static_cast<Poisson *>(girafe);`
⇒ autorisée mais incorrecte
 - Conversion descendante (mère → fille)
⇒ vérification à l'exécution nécessaire
⇒ utiliser l'opérateur «`dynamic_cast`»

- Conversion refusée s'il n'y a pas de lien d'héritage
 - ❑ `animal = static_cast<Animal *>(sapin);` ⇒ refusée
 - ❑ `int * pi = ...;`
`float * pf = static_cast<float *>(pi);` ⇒ refusée
- Fonctionne de la même manière sur les références
- Conversion vers «`void *`» autorisée
 - ❑ `void * nawouak = static_cast<void *>(girafe);`
- Conversion depuis «`void *`» devrait être refusée
 - ❑ `animal = static_cast<Animal *>(nawouak);`
 - ❑ Peut être autorisée suivant le compilateur
 - ❑ Conseil: utiliser «`reinterpret_cast`» dans cette situation

- Vérifie la conversion de pointeurs/références à l'exécution
 - ❑ Même vérification que «`static_cast`» effectuée à la compilation
 - ❑ Ne peut pas être employé pour convertir à partir de «`void *`»
- Utilisé lors d'une conversion descendante (*downcast*)
 - ❑ Conversion d'une classe mère vers une classe fille
⇒ pas toujours possible
 - ❑ Une vérification à l'exécution est nécessaire
 - ❑ `poisson = dynamic_cast<Poisson *>(requin);` ⇒ autorisée
- A l'exécution, la conversion peut échouer
 - ❑ Conversion de pointeurs ⇒ pointeur nul retourné
 - ❑ Conversion de références ⇒ exception levée
 - ❑ `poisson = dynamic_cast<Poisson *>(girafe);` ⇒ refusée

- Conversion plus coûteuse que «*static_cast*»
⇒ à éviter donc quand «*static_cast*» suffit

- Conversion par référence évite les recopies

```
Animal girafe;  
Poisson requin;
```

```
Animal & animal_1 = girafe;  
Animal & animal_2 = requin;
```

```
Poisson & poisson_1 = dynamic_cast<Poisson &>(animal_1);  
⇒ exception levée à l'exécution
```

```
Poisson & poisson_2 = dynamic_cast<Poisson &>(animal_2);  
⇒ conversion autorisée
```

- Permet de retirer l'aspect constant d'une valeur
- N'a pas d'intérêt lors d'une copie
 - ❑ `void f(const Chaine & c1) { ... Chaine c2 = c1; ... }`
 - ❑ L'opération ne pose aucun problème
 - ❑ Car une copie est effectuée et elle ne possède pas l'aspect constant
- Vraiment utile pour les références (ou les pointeurs)
 - ❑ `void f(const Chaine & c1)`
 `{ ... Chaine & c2 = const_cast<Chaine &>(c1); ... }`
 - ❑ «`const_cast`» indispensable ici pour autoriser la conversion
- L'usage de cet opérateur est à éviter
 - ❑ Il permet de briser des règles fondamentales
 - ❑ Souvent, obligation d'utiliser «`const_cast`» ⇒ erreur de conception
 - Soit en imposant à tort la constance sur la variable
 - Soit en omettant des méthodes qui permettraient un accès constant
 - ❑ La solution «peut» être le modificateur «`mutable`»

- Conversion de pointeurs sans aucune vérification
 - Aucune instruction générée, simple changement de type du pointeur

- Exemple

```
struct ip_t { // Champs de bits
    unsigned int n1 : 8;
    unsigned int n2 : 8;
    unsigned int n3 : 8;
    unsigned int n4 : 8;
};

int main() {
    char * data = read_from_network();
    ip_t * ip = reinterpret_cast<ip_t *>(data);

    std::cout << ip->n1 << "." << ip->n2 << "."
               << ip->n3 << "." << ip->n4 << std::endl;
}
```


Conversions: conclusion

	Chaine vers char *	Poisson * vers Animal *	Animal * vers Poisson *	Objet * vers void *	void * vers Objet *
(type)	<u>Oui</u>	Oui	Oui	Oui	Oui
static_cast	Oui	<u>Oui</u>	Oui	<u>Oui</u>	Ne devrait pas
dynamic_cast	Non applicable	Oui	<u>Oui</u> (après vérification)	Oui	Non applicable
reinterpret_cast	Non applicable	Oui	Oui	Oui	<u>Oui</u>

- *Run-Time Type Information*
- Très utile pour déterminer la classe réelle d'un objet à l'exécution
 - Celui-ci doit être pointé ou référencé
 - Pour que les liens d'héritage s'appliquent
- Même type de contrôle que «`dynamic_cast`»
- Mot-clé «`typeid`» retourne un objet de type «`type_info`»
 - `#include <typeinfo>`
- Exemple

```
Poisson p("Maurice",10,20,3);
Mammifere m("Rantanplan",5,9,17);
Animal * animal_1 = &p;
Animal * animal_2 = &m;
...
std::cout << typeid(*animal_1).name();
```

- L'objet «`type_info`» contient des informations sur le type
 - Nom du type: méthode «`name`»
 - Plus intéressant, opérateurs «`==`» et «`!=`»

- Permet de vérifier que deux objets sont du même type

```
if (typeid(*animal_1) == typeid(*animal_2))
    std::cout << "Ils sont de même type." << std::endl;
else
    std::cout << "Ils ne sont pas de même type." << std::endl;
```

- «`typeid`» peut s'appliquer sur un type

```
if (typeid(*animal_1) == typeid(Poisson))
    std::cout << "C'est un poisson.";
else std::cout << "Ce n'est pas un poisson.";
```

- Attention au piège: pensez à déréférencer les pointeurs
 - Car pas de lien d'héritage entre les pointeurs
 - Aucun lien entre «`Animal *`» et «`Poisson *`»

■ Exemple

```
Animal * ptr = new Poisson("Maurice",10,20,3);  
Animal & ref = *ptr;
```

■ Résultats de comparaisons de types

	<code>typeid(Animal)</code>	<code>typeid(Poisson)</code>	<code>typeid(Animal *)</code>	<code>typeid(Poisson *)</code>
<code>typeid(ptr)</code>	<code>!=</code>	<code>!=</code>	<code>==</code>	<code>!=</code>
<code>typeid(ref)</code>	<code>!=</code>	<code>==</code>	<code>!=</code>	<code>!=</code>
<code>typeid(*ptr)</code>	<code>!=</code>	<code>==</code>	<code>!=</code>	<code>!=</code>
<code>typeid(&ref)</code>	<code>!=</code>	<code>!=</code>	<code>==</code>	<code>!=</code>

PARTIE IV

Généricité

Bruno Bachelet

Loïc Yon

- Concept apparu dès les années 70
- Ce n'est pas un concept objet
 - ❑ Les principes objets ne sont pas nécessaires
- ... mais proposé par les langages objets
 - ❑ ADA
 - ❑ C++
 - ❑ Java, C#

- Définir des entités abstraites du type de données
 - Structures de données: vecteur, pile, file, ensemble...
 - Algorithmes: chercher, trier, insérer, extraire...

⇒ Abstraction de données

- Autre manière de factoriser le code
 - Dans une fonction, les paramètres sont des valeurs
 - Dans sa définition, des valeurs sont inconnues
 - Au moment de l'appel (à l'exécution), ces valeurs sont fixées
 - Dans un générique, les paramètres sont des types
 - Dans sa définition, des types sont inconnus
 - Au moment d'utiliser le générique (à la compilation), ces types sont fixés

- Un générique est un modèle
 - ❑ Instanciation = création d'un élément à partir d'un modèle
 - ❑ Instancier un générique $\Rightarrow$ fixer le type de ses paramètres

- Spécificités en C++
 - ❑ Génériques appelés «*templates*»
 - ❑ Des constantes peuvent aussi être des paramètres
 - ❑ Peuvent être génériques: fonctions, classes ou méthodes
 - Depuis C++14: variables globales ou attributs de classe
 - ❑ Possibilité de «spécialisation statique»
 - Une autre forme de polymorphisme
 - Permet la spécialisation pour certains types de données

■ Algorithme de tri

- ❑ Fonctionne de la même manière sur tout type de données
 - Entiers, flottants, chaînes, instances d'une classe A...
- ❑ Suppose des fonctionnalités sur le type manipulé
 - Une relation d'ordre
 - ❑ Opérateur «<»
 - ❑ Une fonction ou un objet tiers (e.g. foncteur)
 - Un mécanisme de copie
 - ❑ Opérateur «=»

■ Type pile

- ❑ Fonctionne de la même manière sur tout type de données
- ❑ Suppose un mécanisme de copie

Héritage et généricité complémentaires

- Tous les deux fournissent une forme de polymorphisme
 - ❑ La généricité agit à la compilation | L'héritage agit à l'exécution
- Contribuent tous les deux à développer du code générique
 - ❑ Tous les deux font abstraction du type
 - ❑ L'un par un mécanisme de paramètre | L'autre par un processus de généralisation
- Avec l'héritage
 - ❑ Plus de flexibilité, mais moins de sûreté (e.g. `vector<ObjetGraphique *>`)
 - ❑ Contrôles de type effectués à l'exécution
 - ❑ Peut entraîner des ralentissements significatifs
- Avec la généricité
 - ❑ Moins de flexibilité, mais plus de sûreté (e.g. `vector<T>`)
 - ❑ Contrôles de type effectués à la compilation
 - ❑ Moins de ralentissement (voire aucun) à l'exécution

- Mot-clé «`template`»
- Précède un composant générique
 - ❑ Fonction, classe ou méthode
 - ❑ Ou variable (depuis C++14)
- `template` *<liste_paramètres> composant*
- Définit une liste de paramètres de différentes natures
 - ❑ Point commun: leur valeur sera fixée à la compilation ⇒ instantiation
 - ❑ Les plus courants
 - Un type: `typename T` (ou `class T`)
 - Une constante: `int N`
 - ❑ Les autres
 - Un *template*: `template <typename> class C`
 - Un «pack» de paramètres: `typename... P` (depuis C++11, cf. *variadic templates*)

- Définition d'une fonction générique

```
template <typename T>  
const T & min(const T & a, const T & b)  
{ return (a < b ? a : b ); }
```

- Suppose l'opérateur de comparaison sur le type paramétré «T»

- Appel à une fonction générique (instanciation + exécution)

```
int i,j;  
...  
int k = min<int>(i,j);
```

- Instanciation $\Rightarrow$ fixer les types paramétrés

- Pas obligatoire de préciser les types paramétrés à l'instanciation
- Si le compilateur a suffisamment d'informations, il déduit les types
 - Comme avec la surcharge de nom
 - Forme de polymorphisme statique
 - `int i,j; ... min(i,j);` ⇒ instanciation de `min<int>`
 - `double a,b; ... min(a,b);` ⇒ instanciation de `min<double>`
- Le compilateur peut effectuer des conversions implicites si les types ne correspondent pas tout à fait

- Définition d'une classe générique

```
template <typename T, int N>
class Pile {
private:
    T elements[N];
    int sommet;
public:
    Pile();
    void ajouter(const T &);
    T retirer();
};
```

- Instanciation d'une classe générique

- `Pile<int,256> p;`
- `using pile_double_t = Pile<double,100>;`

- Possibilité d'une valeur par défaut pour un paramètre
- Constante par défaut
 - ❑ `template <typename T, int N = 256>`
 `class Pile;`
 - ❑ `Pile<int>` ⇒ instantiation de `Pile<int, 256>`
- Type par défaut
 - ❑ `template <typename T, typename C = int>`
 `class TableHachage;`
 - ❑ `TableHachage<string>`
 ⇒ instantiation de `TableHachage<string, int>`

- Exemple plus subtil

- Le type de structure utilisée pour modéliser une pile devient un paramètre

- Définition de la classe

```
template <typename T, typename C>
class Pile {
    private:
        C elements;
    ...
};
```

- Instanciation: `Pile<int, std::vector<int>>`

- Possibilité d'une structure par défaut

- `template <typename T, typename C = std::vector<T>>`
 `class Pile;`
- `Pile<int>` ⇒ instanciation de `Pile<int, std::vector<int>>`

Paramètre «*template template*»

- Possibilité d'avoir une classe générique comme paramètre
 - Mot-clé «`template`» utilisé dans les paramètres du générique
 - Exemple: pile paramétrée par la classe générique de la structure sous-jacente
- Définition de la classe

```
template <typename T, template <typename> class C>
class Pile {
    private:
        C<T> elements;
    ...
};
```
- Instanciation: `Pile<int, std::vector>`
- Attention: «`C`» n'est pas un type mais bien un modèle !
 - `C` = classe générique, `C<T>` = type de la structure sous-jacente
 - `Pile<int, std::vector<int>>` est donc incorrect

- Exemple: copie de piles de types différents

```
Pile<double> p1;
```

```
Pile<int> p2;
```

```
...
```

```
p1 = p2; // Incorrect, "p1" et "p2" de types différents
```

- Solution avec une méthode générique

```
template <typename T> class Pile {
```

```
...
```

```
    template <typename U> void copier(const Pile<U> &);
```

```
};
```

- Appel à la méthode générique

- ❑ Instanciation implicite: `p1.copier(p2);`

- ❑ Instanciation explicite: `p1.copier<int>(p2);`

- Opérateur d'affectation générique

- ❑ `template <typename U> Pile<T> & operator=(const Pile<U> &);`

Implémentation d'un *template* (1/2)

- Normalement, séparation interface et implémentation
 - ❑ Fichier entête
 - Déclaration méthodes + attributs
 - ❑ Fichier implémentation
 - Définition méthodes + attributs statiques
- Pour la suite, méthode «*template*»
= méthode générique ou méthode d'une classe générique
- Implémentation des méthodes *template* dans un entête
 - ❑ Utilisation des méthodes *template* «similaire» aux méthodes *inline*
 - ❑ Ce sont des modèles de méthodes
 - ❑ Leur implémentation doit être visible au moment de l'appel
- Conseil: placer les implémentations en dehors de la classe

Implémentation d'un *template* (2/2)

```
template <typename T, int N>
class Pile {
private:
    T elements[N];
    int sommet;
public:
    Pile();
    void ajouter(const T &);
    T retirer();
};
```

```
template <typename T, int N> Pile<T,N>::Pile() : sommet(0) {}
```

```
template <typename T, int N>
void Pile<T,N>::ajouter(const T & e) { elements[sommet++] = e; }
```

```
template <typename T, int N>
T Pile<T,N>::retirer() { return elements[--sommet]; }
```

Compilation d'un générique (1/2)

- Un code générique n'est pas compilé
 - Analyse «succincte» au niveau syntaxique
- Un code instancié est compilé
 - Analyse «complète» au niveau sémantique
- Instanciation = réécriture
 - Code générique dupliqué
 - Types paramètres remplacés par types concrets
- Equivalent d'un copier-coller-remplacer
 - Permet une efficacité optimale du code

Compilation d'un générique (2/2)

- Attention: une instance par jeu de paramètres
 - ❑ Travail du compilateur important $\Rightarrow$ temps de compilation
 - ❑ Duplication de code $\Rightarrow$ taille de l'exécutable

- Attention: aucun lien entre deux instances (en C++)
 - ❑ Pas de parenté entre les instances d'une classe générique
 - ❑ Pas de passerelle de conversion
 - `Pile<int> p1;`
 - `Pile<double> p2;`
 - `p2 = p1;` $\Rightarrow$ interdit (bien que la conversion `int` $\rightarrow$ `double` existe)
 - ❑ Même dans le cas de constantes
 - `Pile<int,10> p1;`
 - `Pile<int,20> p2;`
 - `p2 = p1;` $\Rightarrow$ interdit

- Amitié = rompre l'encapsulation avec un composant bien identifié
 - ❑ Mot-clé «friend»
- A éviter, mais parfois nécessaire
 - ❑ Entre composants d'un même module
 - ❑ Evite des méthodes publiques inutiles hors module
 - ❑ En C++, pas d'amitié implicite inter-module comme en Java
- Autoriser la classe «B» à voir les membres cachés de la classe «A»
 - ❑

```
class A {  
    friend class B;  
    ...  
};
```
 - ❑ Membre caché = «protected» ou «private»
- L'amitié n'est pas réciproque (ni transitive)
 - ❑ Pour avoir la réciprocité ⇒

```
class B { friend class A; ... };
```

- Une fonction peut être amie
 - ❑ `class A { friend void f(int); ... };`
 - ❑ La fonction «`f(int)`» voit les membres cachés de «`A`»
 - ❑ Ce n'est pas le cas des autres surcharges de «`f`»

- Une méthode peut être amie
 - ❑ `class A { friend void B::g(); ... }`
 - ❑ La méthode «`g`» de la classe «`B`» voit les membres cachés de «`A`»

- Déclaration préalable pas nécessaire pour établir une amitié
 - ❑ Sauf cas particuliers avec généricité (voir plus loin)

- L'amitié ne remplace pas une déclaration
 - ❑ «`A`» ne peut pas utiliser «`f`» ou «`B`» sans déclaration préalable

- Pour utiliser une classe ou une fonction, celle-ci doit être connue
 - ❑ Elle doit être déclarée
 - ❑ Pour une fonction $\Rightarrow$ prototype
 - ❑ Pour une classe $\Rightarrow$ déclaration complète ou «anticipée»
- Dépendance réciproque entre classes
 $\Rightarrow$ déclaration anticipée («*forward declaration*»)
- Avant chaque classe, déclaration anticipée de l'autre classe
 - ❑ Entête classe A

```
class B; // Déclaration anticipée
...
class A { ... void m1(B & b); ... };
```
 - ❑ Entête classe B

```
class A; // Déclaration anticipée
...
class B { ... void m2(A & a); ... };
```

- Déclaration anticipée = déclaration partielle d'un type
 - ❑ Seul le nom est indiqué
 - ❑ Rien n'est précisé sur la structure du type

⇒ Restrictions tant qu'il n'est pas complètement déclaré

- Aucune méthode ou attribut ne peut être appelé
 - ❑ `A a;` ⇒ interdit
 - ❑ `A::x;` ⇒ interdit
 - ❑ `A::m();` ⇒ interdit
- Le type peut être utilisé sans restriction dans les déclarations
 - ❑ `void m(A *);` ⇒ ok
 - ❑ `void m(A &);` ⇒ ok
 - ❑ `void m(A);` ⇒ ok

- Seuls les pointeurs et références peuvent être utilisés dans les déclarations
 - Variables
 - `A * a; ⇒ ok`
 - `A & a; ⇒ ok`
 - `A a; ⇒ interdit`
 - `a->m(); ⇒ interdit`
 - Arguments
 - `void m(A *) { ... } ⇒ ok`
 - `void m(A &) { ... } ⇒ ok`
 - `void m(A) { ... } ⇒ interdit`
- Possibilité de faire des alias d'une déclaration anticipée
 - `using mon_ami = A;`

■ Exemples

- ❑ `template <typename T> class B;`
- ❑ `template <typename T> void f();`

■ Amitié avec toutes les instances

- ❑ `class A { template <typename T> friend class B; ... };`
- ❑ `class A { template <typename T> friend void f(); ... };`

■ Amitié avec une instance particulière

- ❑ Attention: une déclaration préalable (de «B» et «f») est nécessaire
- ❑ `class A { friend class B<int>; ... };`
- ❑ `class A { friend void f<int>(); ... };`

- Cas d'une classe générique: exemple d'amitié avec une instance

```
template <typename T> class Vecteur; // Déclaration anticipée
```

```
template <typename T> // Prototype nécessite déclaration  
ostream & operator << (ostream &, const Vecteur<T> &);
```

```
template <typename T> class Vecteur {  
    // Amitié nécessite prototype  
    friend ostream & operator<< <T> (ostream &, const Vecteur<T> &);  
  
    private:  
        T * elements;  
        int nb;  
        ...  
};
```

```
template <typename T> // Définition nécessite amitié  
ostream & operator<<(ostream & f, const Vecteur<T> & v) {  
    for (int i = 0; i < v.nb; ++i) f << v.elements[i] << " ";  
    return f;  
}
```

■ Héritage «simple»

- Héritage d'une instance d'une classe générique
- Exemple: «NuagePoint» hérite de «Vecteur<Point>»

■ Illustration

- `template <typename T>`
 `class A { ... };`
- `class B : public A<int> { ... };`

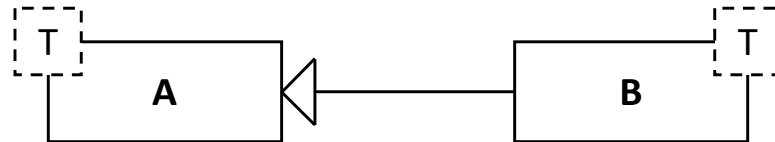


- Héritage «classique»

- Héritage entre deux classes génériques
- Exemple: «FileAttente<T>» hérite de «Vecteur<T>»

- Illustration

- `template <typename T>`
 `class A { ... };`
- `template <typename T>`
 `class B : public A<T> { ... };`

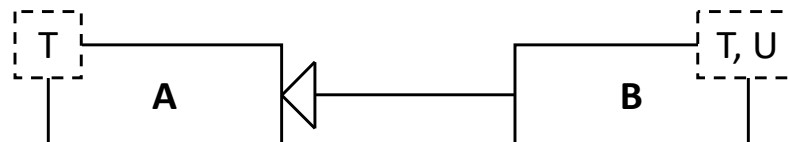


■ Héritage avec «extension»

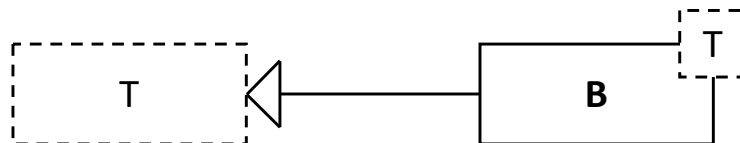
- Héritage entre classes génériques avec ajout d'un paramètre
- Exemple: «FilePriorite<T,C>» hérite de «Vecteur<T>»
 - C = objet comparateur qui indique la relation d'ordre

■ Illustration

- `template <typename T>`
`class A { ... };`
- `template <typename T, typename U>`
`class B : public A<T> { ... };`



- Héritage «générique»
 - Héritage d'une classe qui est un paramètre
 - Extension potentielle de toutes les classes
 - Exemple: «Comparable<T>» hérite de «T»
 - Toute classe peut devenir un «comparable»
- Voir *The Curiously Recurring Template Pattern* (CRTP)
- Illustration
 - `template <typename T> class B : public T { ... };`



PARTIE V

Gestion des exceptions

Bruno Bachelet

Loïc Yon

- Pour gérer les erreurs: les «exceptions»
- Mécanisme qui permet de séparer
 - ❑ La détection d'une erreur
 - ❑ La prise en charge de l'erreur
- Exemple: code de calcul + interface graphique
 - ❑ Le code de calcul détecte des erreurs
 - ❑ L'interface graphique est informée
et affiche un message dans une fenêtre
- Permet de conserver une modularité
- Exception = objet qui est créé lorsqu'une erreur survient

- Mot-clé «**throw**» dans une méthode
 - ❑ Au lieu de gérer l'erreur localement, l'erreur est transmise à la méthode appelante
 - ❑ On dit qu'une exception est «levée» / «lancée»
 - ❑ `if (erreur) throw std::string("oops !");`
 - ❑ Interruption de la suite normale du code
- L'objet transmis contient des renseignements sur l'erreur
- Pour traiter une exception, il faut...
 - ❑ Surveiller ⇒ `try`
 - ❑ Capter ⇒ `catch`

- Bloc «**try**» définit une zone de surveillance
 - ❑ **try** {
 // Code susceptible de lancer une exception
}
 - ❑ **throw** ⇒ suspension de l'exécution normale
- Bloc «**catch**» décrit le traitement d'une exception
 - ❑ **catch**(**const exception & e**) { /* Gestion exception */ }
 - ❑ Reprise de l'exécution suspendue par «**throw**»
- Plusieurs «**catch**» peuvent se succéder
 - ❑ Le premier qui correspond au type de l'erreur sera exécuté
 - ❑ Donc placement des «**catch**» du plus spécifique au plus général
 - ❑ **catch**(**const MonException & e**) { ... }
 catch(**const std::exception & e**) { ... }
 catch(...) { ... }

- Obligation de rattraper toutes les exceptions potentielles

- ❑ Gestion immédiate: «`catch`» dans la méthode
- ❑ Possibilité de «renvoyer» à la méthode appelante avec «`throw`»

- Exemple: «`ExceptionFichier`» hérite de «`std::exception`»

```
void lectureFichier(const std::string & nom) {  
    // Code susceptible de lever une exception «ExceptionFichier»  
}
```

```
void traitement() {  
    try {  
        lectureFichier("mon_fichier.dat");  
        // Code susceptible de lever une exception «std::exception»  
    }
```

```
    catch(const ExceptionFichier & e) { cout << "Erreur fichier !" << endl; }
```

```
    catch(const std::exception & e) { cout << "Erreur traitement !" << endl; }
```

```
    // Exécuté si aucune exception levée ou exception traitée  
    cout << "Fin du traitement" << endl;  
}
```

- Si possible, utiliser une classe standard
 - `invalid_argument`, `out_of_range`, `overflow_error`...
- Sinon, créer ses propres classes d'exceptions
 - Spécialiser la classe de base «`std::exception`» ou une sous-classe
 - Encapsuler des informations sur l'erreur
 - Redéfinir la méthode «`what`» pour retourner un message décrivant l'erreur

- Exemple

```
class MonException : public std::exception {
private:
    // Attributs (infos sur l'erreur)

public:
    MonException(...);
    // Accesseurs
    const char * what() const noexcept override;
}
```

- Toujours avoir un catch «universel»
 - ❑ `catch (...) { traitement }`
 - ❑ Permet de gérer les imprévus
 - ❑ Placé en général au niveau le plus haut
 - ❑ Dans la fonction «`main`» par exemple

- Fournir des garanties en cas d'exception
 - ❑ Que se passe-t-il en cas d'exception en plein milieu d'une série d'opérations ?
 - ❑ Garantir une certaine cohérence
⇒ «*Exception safety*»

- Aucune garantie
 - ❑ Les données peuvent se retrouver dans un état incohérent
 - ❑ Fuite mémoire, crash possible
- Garantie «*no leak*»
 - ❑ Pas de fuite mémoire ou d'erreur de pointeur
- Garantie «*invariants preserved*»
 - ❑ Les données restent dans un état cohérent
 - ❑ Effet de bord possible
- Garantie «*no change*»
 - ❑ Les données conservent leurs valeurs originales
 - ❑ Pas d'effet de bord
- Garantie «*no throw*»
 - ❑ Toutes les opérations s'exécutent avec succès
 - ❑ Aucune exception ne sort de la méthode

- Exemple: opérateur d'affectation d'un vecteur d'entiers

- Aucune garantie

```
Vecteur & Vecteur::operator=(const Vecteur & v) {  
    if (this != &v) {  
        delete[] tab;  
        tab = new int[v.size];  
        size = v.size;  
        for (unsigned i = 0; i < size; ++i) tab[i] = v.tab[i];  
    }  
  
    return *this;  
}
```

- Rappel: «new» peut lever une exception
⇒ incohérence en cas d'erreur: «tab» a été libéré et «size» n'est pas nulle

- Garantie «no change»

```
Vecteur & Vecteur::operator=(const Vecteur & v) {  
    if (this != &v) {  
        int * t = new int[v.size];  
        delete[] tab;  
        tab = t;  
        size = v.size;  
        for (unsigned i = 0; i < size; ++i) tab[i] = v.tab[i];  
    }  
  
    return *this;  
}
```

- «this» n'est pas modifié avant l'exception éventuelle

- Inconvénients

- ❑ Code plus compliqué
- ❑ Plus difficile de mutualiser du code avec le constructeur

- Autre solution: l'idiome «*copy-and-swap*»
 - `Vecteur & Vecteur::operator=(const Vecteur & v) {`
 `Vecteur v2(v);`
 `this->swap(v2);`
 `return *this;`
}
 - `void Vecteur::swap(Vecteur & v) {`
 `std::swap(size,v.size);`
 `std::swap(tab,v.tab);`
}
- Exception levée lors de la copie
⇒ «`v`» et «`this`» restent dans leur état initial
- `swap` ⇒ échange des contenus

- «*copy-and-swap*» $\Rightarrow$ garantie «*no change*»
 - A condition que «*swap*» soit garanti «*no throw*»
- Avantage: réutilisation du code du constructeur de copie
- Attention aux performances de «*swap*» !
 - Tous les attributs ne seront pas des entiers ou des pointeurs
 - Utiliser «*std::swap*» pour les classes de la bibliothèque standard
 - Proposer un «*swap*» efficace pour vos classes
 - A partir de C++11, utilisation des opérateurs «de mouvement»
- Possibilité d'une écriture encore plus compacte
 - Passage de l'argument par copie
 - ```
Vecteur & Vecteur::operator=(Vecteur v) {
 this->swap(v);
 return *this;
}
```

## PARTIE VI

# Généricité avancée

Bruno Bachelet

Loïc Yon

# Nécessité des «concepts» (1/3)

- Exemple: algorithme de tri générique
  - Classe générique «**AlgoTri**» avec paramètre «**T**»
  - **T** = type des éléments à trier
  - Éléments comparés à l'aide de la méthode «**estAvant**»
    - Permet un tri décroissant par exemple

- Code possible

```
template <typename T>
void AlgoTri<T>::trier(T t[], int n) {
 for (int i = 0; i < n-1; i++)
 for (int j = i+1; j < n; j++)
 if (t[j].estAvant(t[i]))
 { T x = t[i]; t[i] = t[j]; t[j] = x; }
}
```

- Hypothèse: le type «**T**» possède la méthode «**estAvant**»
  - Vérification faite à la compilation, au moment de l'instanciation

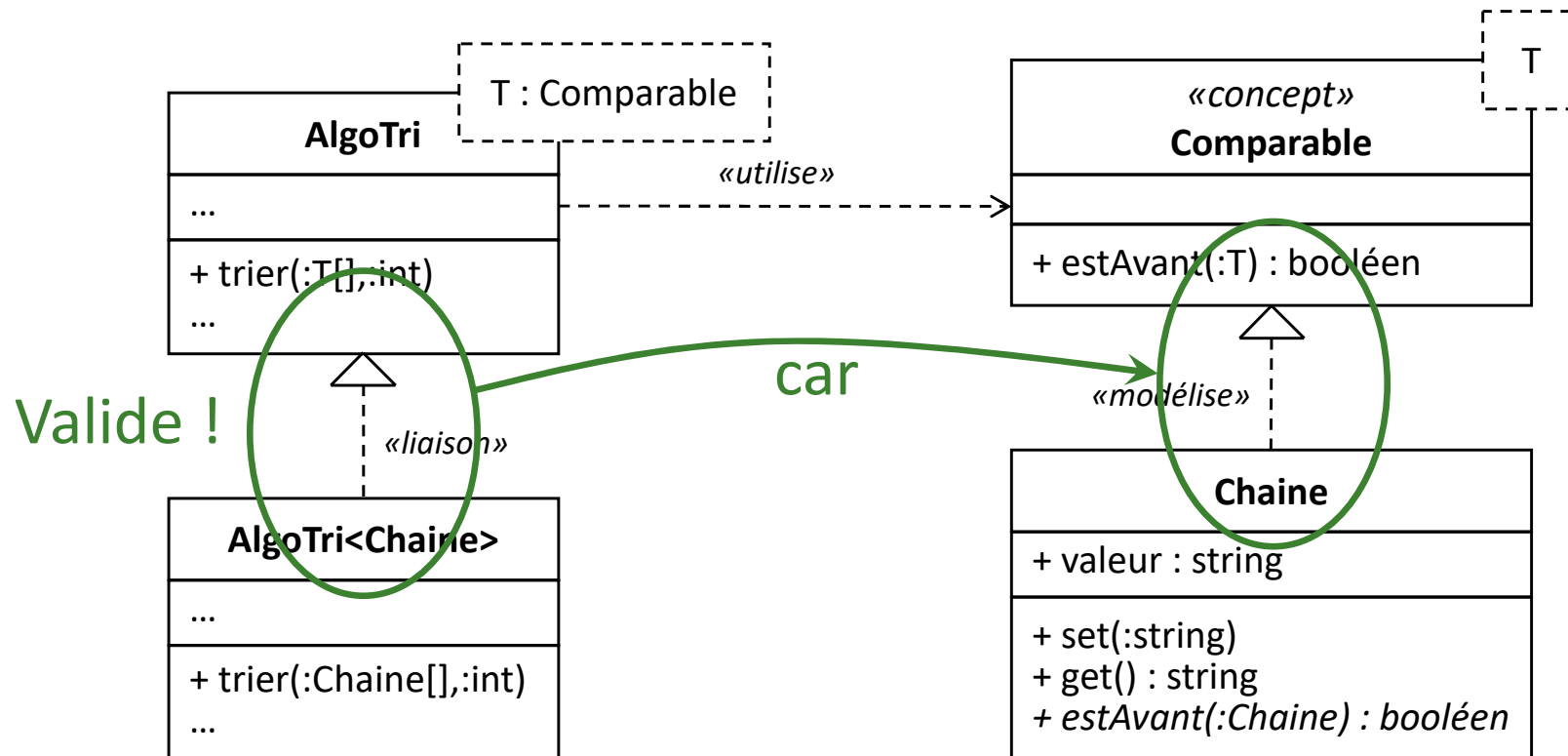
# Nécessité des «concepts» (2/3)

---

- L'interface supposée de «T» fait partie d'un «concept»
- Concept = ensemble «nommé» de spécifications/contraintes
  - ❑ Concerne l'interface: existence d'une méthode
  - ❑ Concerne l'implémentation: sémantique d'une méthode
  - ❑ Mais aussi toute contrainte pertinente liée à l'utilisation du type
- Proposition de définition du concept «Comparable»
  - ❑ La méthode d'instance «estAvant» doit exister
    - Signature exacte (UML): **booléen estAvant(:T)**
  - ❑ Elle doit définir une relation d'ordre (partielle) entre deux objets
    - Pseudo-code: **si a.estAvant(b) = vrai alors b.estAvant(a) = faux**
- Le type «T» de l'algorithme peut être contraint par ce concept
  - ❑ «T» respecte le concept «Comparable»
  - ❑ On dit aussi: «T» modélise le concept «Comparable»



# Nécessité des «concepts» (3/3)



- En Java: concepts limités aux interfaces
  - ❑ Un paramètre générique peut être contraint par des interfaces
- En C++: concepts partiellement intégrés au langage depuis C++20
  - ❑ Avant: seule une documentation permettait de les identifier (cf. doc STL)
  - ❑ Maintenant: un paramètre générique peut être contraint par des concepts

## ■ Concept «Comparable»

- ❑ `template <typename T>`  
`concept Comparable = requires(const T a, const T b) {`  
 `{ a.estAvant(b) } -> std::convertible_to<bool>;`  
`};`
- ❑ «`a.estAvant(b)`» doit être valide pour les objets constants «`a`» et «`b`» de type «`T`»  
⇒ la méthode «`estAvant`» doit exister et être constante
- ❑ Et elle doit retourner une valeur convertible en booléen  
(«`convertible_to`» est un concept standard)

## ■ Pour formuler des exigences ⇒ expression «`requires`»

- ❑ Syntaxe: `requires (paramètres optionnels) { exigences }`
- ❑ Lister les exigences («*requirements*») à appliquer...
- ❑ ...aux paramètres du concept (le paramètre «`T`» dans l'exemple)
- ❑ ...aux paramètres locaux de l'expression (les variables «`a`» et «`b`» dans l'exemple)
- ❑ Analysées à la compilation mais non évaluées à l'exécution

- Contrainte sur la classe «AlgoTri» ⇒ mot-clé «requires»

```
template <typename T> requires Comparable<T>
class AlgoTri {
 public:
 void trier(T[],int);
};
```

- Syntaxe alternative

```
template <Comparable T> class AlgoTri { ... }
```

- Possibilité aussi d'écrire la liste d'exigences directement

```
template <typename T> requires (
 requires(const T a, const T b) {
 { a.estAvant(b) } -> std::convertible_to<bool>;
 }
)
class AlgoTri { ... }
```

## ■ Contrainte atomique

### □ Modéliser un concept

- `Comparable<T>`  $\Rightarrow$  «T» modélise «Comparable»

### □ Valider une expression booléenne

- `std::is_convertible<T,int>::value`  $\Rightarrow$  «T» est convertible en type «int»
- «is\_convertible» est un «type trait» standard
- *Type trait* = métafonction qui vérifie une propriété d'un type (cf. métaprogrammation)

### □ Liste d'exigences (expression «requires»)

- `requires(T a, const T b) { a = b; }`

## ■ Combinaison de contraintes

### □ Conjonction

- `Comparable<T> && Copiable<T>`

### □ Disjonction

- `std::integral<T> || std::floating_point<T>`

## ■ Un concept est une contrainte nommée

- 4 types de «*requirements*» pour formuler une liste d'exigences
- *Simple requirement*: impose qu'une expression est valide
  - `a + b;`  $\Rightarrow$  expression (non évaluée) valide (elle compile)  $\Rightarrow$  opérateur «+» existe
- *Type requirement*: impose qu'une expression est un type valide
  - `typename T::value_type;`  $\Rightarrow$  «T» possède le type interne «value\_type»
  - `typename A<T>`  $\Rightarrow$  instantiation valide  $\Rightarrow$  «T» satisfait les contraintes de «A»
- *Compound requirement*: impose qu'une expression est valide + des contraintes sur son type de retour
  - `{ a * 2 } -> std::convertible_to<T>`  
 $\Rightarrow$  expression (non évaluée) valide et résultat convertible en «T»
- *Nested requirement*: impose des contraintes supplémentaires
  - `requires std::integral<T> || std::floating_point<T>`  
 $\Rightarrow$  «T» modélise l'un des deux concepts  $\Rightarrow$  «T» représente un entier ou un flottant

- Concept à plusieurs paramètres  $\Rightarrow$  mécanisme de substitution
- Exemple: `std::convertible_to`
  - Concept standard à deux paramètres
  - `template <typename FROM, typename TO> concept convertible_to = ...;`
- Dans une contrainte: type contraint ajouté implicitement comme premier paramètre du concept
  - `template <std::convertible_to<double> T>`  
 $\Leftrightarrow$  `requires std::convertible_to<T,double>`
  - `{ a * b } -> std::convertible_to<double>`  
 $\Leftrightarrow$  `requires std::convertible_to<decltype(a * b),double>`
  - Remarque: «`decltype`» détermine le type d'une expression sans l'évaluer (voir détails plus tard)

- Exemple précédent (tri) incomplet
- «T» doit aussi être un type «copiable»
  - ❑ Opérateur d'affectation nécessaire
  - ❑ Constructeur de copie nécessaire

- Concept «Copiable»

```
template <typename T>
concept Copiable = requires(T a, const T b) {
 a = b;
 T(b);
};
```

- «T» est donc contraint par deux concepts

```
template <typename T>
requires (Comparable<T> && Copiable<T>) class AlgoTri { ... };
```

- Composant générique = modèle indépendant des types
- Mais cela peut être pénalisant
  - Exemple: recherche d'un élément dans une structure
  - Approches différentes suivant que la structure soit triée ou non

## ⇒ Mécanisme de spécialisation «statique»

- Spécialisation du modèle générique pour un jeu de paramètres
- Jeu de paramètres partiel ou complet
  - On parle aussi d'«instanciation» partielle ou complète
- Associé au polymorphisme statique de l'instanciation
  - «Meilleure» instanciation choisie en fonction du jeu de paramètres



# Spécialisation d'une fonction générique

- Modèle générique d'une fonction de calcul de moyenne

```
template <int N> double moyenne(int * tab) {
 double somme = 0.0;
 for (int i = 0; i < N; ++i) somme += tab[i];
 return (somme/N);
}
```

- Spécialisation du modèle pour  $N = 2$  et  $N = 1$

```
template <> double moyenne<2>(int * tab)
{ return (double(tab[0] + tab[1])/2); }
```

```
template <> double moyenne<1>(int * tab)
{ return double(tab[0]); }
```

- Attention

- ❑ Déclarer d'abord la version générique, puis les versions spécifiques
- ❑ Spécialisation «partielle» d'une fonction (ou méthode) interdite

# Spécialisation d'une classe générique

- Modèle générique d'un vecteur d'éléments

```
template <typename T> class Vecteur {
 private:
 T * elements;
 int taille;
 ...
 public: T operator[](int i) { return elements[i]; }
};
```

- Spécialisation du modèle pour  $T = \text{bool}$

```
template <> class Vecteur<bool> {
 private:
 char * elements;
 int taille;
 ...
 public: bool operator[](int i)
 { return ((elements[i/8] >> (i%8)) & 1); }
};
```

- Mécanisme statique lors de l'instanciation d'un modèle

- Sélection de la version la plus spécialisée
- En fonction du jeu de paramètres

⇒ Génération du code le plus dédié possible

- Exemples d'instanciation avec polymorphisme

- Nombre entier (statique) inconnu: `moyenne<N>(tab);`
  - `N = 10` ⇒ version générique
  - `N = 2` ⇒ version spécialisée
- Type inconnu: `Vecteur<T> v;`
  - `T = int` ⇒ version générique
  - `T = bool` ⇒ version spécialisée

- Concepts et contraintes peuvent servir à définir une spécialisation

- `template <typename T> void f(T &);` ⇒ définition version générique
- `template <std::integral T> void f(T &);` ⇒ définition version spécialisée

- Spécialisation partielle  
= spécialisation avec un jeu de paramètres incomplet

- Retour sur l'exemple de calcul de moyenne

```
template <typename T, int N> class Moyenne {
 public: static T calculer(T * tab) {
 T somme = T();
 for (int i = 0; i < N; ++i) somme += tab[i];
 return (somme/T(N));
 }
};
```

- Spécialisation pour  $N = 2$  ( $T$  reste inconnu)

```
template <typename T> class Moyenne<T,2> {
 public: static T calculer(T * tab)
 { return ((tab[0] + tab[1])/T(2)); }
};
```

- Instanciation partielle  $\Rightarrow$  passage par une classe
  - Rappel: instanciation partielle interdite pour une fonction
- Peut impliquer une lourdeur d'écriture
  - `double m = Moyenne<double,10>::calculer(tab);`
- Solution possible: proposer une fonction «aidante»

```
template <int N, typename T>
inline T moyenne(T * tab) {
 return Moyenne<T,N>::calculer(tab);
}
```

- Facilite l'instanciation grâce à la déduction automatique de type
  - `double m = moyenne<10>(tab);`
  - Remarque: l'ordre de déclaration des paramètres a son importance
    - «N» est fourni explicitement
    - «T» est déduit des arguments de la fonction

# Retour sur l'héritage avec généricité

---

## ■ Exemple

- ❑ `template <typename T> class A`  
    `{ ... public: void m(); ... };`
- ❑ `template <typename T> class B : public A<T>`  
    `{ ... public: void n() { ... m(); ... } ... };`

## ■ Instanciation partielle ⇒ doute

- ❑ La version générique de «A» peut être remise en question par une spécialisation
- ❑ La méthode «m» peut ne pas exister dans cette spécialisation  
⇒ appel à la fonction «m» si elle existe au lieu de la méthode

## ■ Conseil: toujours utiliser «this->» sur un membre hérité

- ❑ Cela provoquera une erreur si une spécialisation ne possède pas ce membre
- ❑ Toujours mieux qu'un comportement implicite (très probablement non voulu)
- ❑ Solution: `void n() { ... this->m(); ... }`

- Deux syntaxes possibles
  - ❑ C++03: `typedef pair<int,double> paire_t;`
  - ❑ C++11: `using paire_t = pair<int,double>;`
  - ❑ Strictement équivalentes ⇒ privilégier la seconde
  
- Alias de type *template* possible avec «using»
  - ❑ `template <typename T> using paire_t = pair<int,T>;`
  - ❑ Forme d'instanciation «partielle»
  
- Type interne à une classe
  - ❑ Possibilité de déclarer une classe dans une autre
    - `class vector { ... class iterator { ... }; ... };`
  - ❑ Et de déclarer des alias de type
    - `class vector { ... using iterator = ...; ... };`

- Indique que ce qui suit est un type
- Utilisé dans la déclaration d'un paramètre
  - ❑ `template <typename T>`
  - ❑ Equivalent à: `template <class T>`
- Utilisé pour lever une ambiguïté
  - ❑ A cause de l'instanciation partielle
  - ❑ Tant que l'instanciation n'est pas effective ⇒ doute
  - ❑ Exemple
    - `template <typename T> class vector`  
`{ public: using value_type = T; /* Type interne */ };`
    - `template <typename T> class B`  
`{ public: using type_elt = typename vector<T>::value_type; };`
  - ❑ Doute sur la nature de «`vector<T>::value_type`»
    - Type ou attribut ? ⇒ `typename`



- Déclarer le type d'une variable n'est pas toujours évident
  - Compliqué à écrire (e.g. instantiation d'un *template*)
  - Polymorphisme statique  $\Rightarrow$  difficile de connaître le type d'un retour
- Alors que le compilateur peut le déduire
  - Contrôle des types à la compilation
  - Capable de détecter une erreur de type  $\Rightarrow$  capable de corriger

- Exemple

```
std::vector<int> v = { ... };
? it = std::find(v.begin(),v.end(),5);
if (it != v.end()) *it = 0;
```

- Première possibilité: mot-clé «**auto**»
  - ❑ Attention au changement de signification
  - ❑ C++03: **auto** **int** **x**; ⇒ incorrect en C++11
  - ❑ C++11: **auto** **x** = ...; ⇒ déduction du type
- **auto** = joker
  - ❑ Le programmeur laisse le compilateur déduire
  - ❑ **auto** **it** = **std::find**(**v.begin**(),**v.end**(),5);
- Peut remplacer le retour d'une fonction
  - ❑ **auto** **add**(**double** **a**, **double** **b**) { **return** **a + b**; }
  - ❑ **template** <**typename** **T**, **typename** **U**>  
    **auto** **add**(**const** **T** & **a**, **const** **U** & **b**) { **return** **a + b**; }
- Peut remplacer tout ou partie d'un type
  - ❑ **auto** \* **x**, **auto** & **x**, **const** **auto** & **x**...
  - ❑ **auto** **x** = **new** **auto**(5);

- Seconde possibilité: `decltype(expression)`
- Identifie le type d'une expression
  - ❑ Demande au compilateur de déduire le type
  - ❑ Mais l'expression n'est pas compilée, ni évaluée à l'exécution
- Exemple

```
decltype(v.begin()) it;
it = std::find(v.begin(), v.end(), 5);
```
- Quelle que soit l'approche, le type est connu à la compilation
  - ❑ Les contrôles de types sont donc préservés
- Mais le type n'est pas explicite dans le code

- *Variadic template* = générique à paramètres variables
  - ❑ Depuis C++11
  - ❑ Liste des paramètres *templates* non fixée
  - ❑ A l'instar des arguments variables d'une fonction
- Permet de modéliser des collections hétérogènes
  - ❑ `template <typename... TYPES> class Tuple;`
- Syntaxe simple pour instancier le générique
  - ❑ `Tuple<int,double,std::string> t;`
- Mais syntaxe pas toujours intuitive pour écrire le générique
  - ❑ Mécanisme d'«expansion»
  - ❑ Ou approche récursive (métaprogrammation)

- Paramètres variables = «pack» de paramètres
  - Pack représenté par le symbole «...»
  - Pack = 0 à  $n$  paramètres
- Pack de types: `template <typename... TYPES>`
- Pack de nombres: `template <int... VALEURS>`
- Pack d'arguments: déclaration à partir d'un pack de types
  - `TYPES... args`  $\Rightarrow$   $n$  arguments, chacun d'un type du pack
- Permet de renforcer le contrôle de types des arguments variables
  - `fprintf(const char * format, ...);`
    - Impossible d'identifier les types des arguments variables
  - `template <typename... TYPES>`  
`void fprintf(const char * format, TYPES... args);`
    - Possibilité d'identifier le type de chaque argument

# Accès aux éléments d'un pack (1/2)

---

- `template <typename... TYPES>`
  - ⇒ `template <typename  $T_1$ , ..., typename  $T_n$ >`
    - Il s'agit d'une illustration:  $T_1...T_n$  n'existent pas explicitement
- Accès à un paramètre d'un pack impossible directement
  - Un paramètre n'a pas d'identifiant
    - Aucun moyen d'obtenir le nom d'un paramètre
  - Un paramètre n'a pas de numéro
    - Aucun moyen direct d'obtenir le  $n^{\text{ième}}$  paramètre
- Nombre d'éléments d'un pack: opérateur `sizeof...(PACK)`
- Parcours et identification possibles par récursivité
  - Ecriture de *templates* récursifs ⇒ voir métaprogrammation

# Accès aux éléments d'un pack (2/2)

- Exemple: implémentation possible de «`sizeof...`»
  - Approche récursive par spécialisation de *template*
  - Proposition d'utilisation du *template*: `taille<PACK>::val`
- Version primaire = déclaration
  - `template <typename... TYPES> struct taille;`
- Spécialisation n°1 = récursion
  - $taille(\{ T_1, \dots, T_n \}) = taille(\{ T_2, \dots, T_n \}) + 1$
  - `template <typename PREMIER, typename... RESTE>`  
`struct taille<PREMIER,RESTE...>`  
`{ static const int val = taille<RESTE...>::val + 1; };`
- Spécialisation n°2 = arrêt
  - $taille(\{ \}) = 0$
  - `template <> struct taille<>`  
`{ static const int val = 0; };`

- Autre possibilité d'utilisation d'un pack  $\Rightarrow$  mécanisme d'expansion
- On décrit un schéma d'expansion
  - Expression contenant l'identifiant d'un pack
  - Et terminée par « ... »
  - Exemples
    - `TYPES...`
    - `const TYPES &...`
    - `vector<TYPES>...`
- Expansion  $\Rightarrow$  réplication du schéma
  - Pour chaque paramètre du pack
  - Séparation par une virgule
  - Exemples
    - `TYPES...  $\Rightarrow$   $T_1, \dots, T_n$`
    - `vector<TYPES>...  $\Rightarrow$  vector< $T_1$ >, ..., vector< $T_n$ >`
    - `const TYPES &... x  $\Rightarrow$  const  $T_1$  &  $x_1$ , ..., const  $T_n$  &  $x_n$`



- Le mécanisme d'expansion s'applique aussi au pack d'arguments

- ```
template <typename... TYPES>
void f(TYPES... args) { g(args...); }
```

- La localisation de «...» est importante

- Exercice: trouver les expansions suivantes (réf. Andrei Alexandrescu)

- ```
g(A<TYPES>::m(args)...);
```
- ```
g(A<TYPES...>::m(args...));
```
- ```
g(A<TYPES...>::m(args)...);
```

- Depuis C++17: introduction des «*fold expressions*»

- Expansion possible avec les opérateurs binaires

- Exemples

- $(args + ...) \Rightarrow (arg_1 + (... + (arg_{n-1} + arg_n)))$
- $(... + args) \Rightarrow (((arg_1 + arg_2) + ...) + arg_n)$

## PARTIE VII

# Opérations de mouvement

Bruno Bachelet

Loïc Yon

- En C++03, deux catégories de valeurs
  - ❑ A noter que ces notions évoluent avec les normes du C++
- *lvalue*
  - ❑ Historiquement: valeur à gauche (*left-handed*) d'une affectation
  - ❑ Valeur «localisable»: accessible via variable, référence ou pointeur
  - ❑ Zone mémoire identifiable ⇒ peut être modifiée
- *rvalue*
  - ❑ Historiquement: valeur à droite (*right-handed*) d'une affectation
  - ❑ Valeur ne pouvant pas être modifiée
  - ❑ Typiquement, une valeur à usage ponctuel
    - Littéral: `factorielle(5);`
    - Temporaire construit à la volée: `display(string("Hello !"));`
    - Retour (par copie) d'une fonction: `s = add(a,b);`

- En C++03, référence non constante sur une *rvalue* interdite
- Exemple
  - ❑ `display(string("Hello !"));`
  - ❑ Création à la volée d'un objet  $\Rightarrow$  *rvalue*
- Quel prototype de fonction pour récupérer la *rvalue* ?
  - ❑ Copie: `void display(string s);`  $\Rightarrow$  OK
  - ❑ Référence constante: `void display(const string & s);`  $\Rightarrow$  OK
  - ❑ Référence non constante: `void display(string & s);`  $\Rightarrow$  non !

- Depuis C++11, changement de définition
  - ❑ Et nouvelles catégories de valeurs: *glvalue*, *xvalue*, *prvalue*...
  - ❑ [http://en.cppreference.com/w/cpp/language/value\\_category](http://en.cppreference.com/w/cpp/language/value_category)
- De manière informelle
  - ❑ *lvalue*  $\Rightarrow$  comme avant
  - ❑ *rvalue*  $\Rightarrow$  valeur qui peut être modifiée sans effet de bord
    - Cas d'un temporaire
    - Usage unique, donc sa modification est sans conséquence
- Ce qui change concrètement
  - ❑ On peut faire une référence non constante sur une *rvalue*
  - ❑ On peut volontairement transformer une *lvalue* en *rvalue*

## ■ Nouvelle syntaxe: `&&`

- ❑ Référence sur une *rvalue*
- ❑ Il s'agit d'une référence  $\Rightarrow$  mêmes règles que «`&`»
  - Caractère constant / non constant
  - Une méthode ne peut pas retourner de référence sur une variable locale

## ■ Retour à l'exemple précédent

- ❑ Rappel: `display(string("Hello !"));`
- ❑ Référence sur *rvalue* non constante: `void display(string && s);`  $\Rightarrow$  OK
- ❑ Mais cette version de la fonction n'est applicable que pour une *rvalue*

## ■ Pourquoi avoir une référence non constante sur une *rvalue* ?

- ❑ Pour pouvoir la «dépouiller»
- ❑ Autrement dit, récupérer son contenu directement au lieu de le copier
- ❑ La *rvalue* ne sera plus utilisable par la suite

## ■ Exemple

```
class Vecteur {
 private:
 int * tab_;
 unsigned taille_;

 public:
 explicit Vecteur(unsigned);
 Vecteur(const Vecteur &);
 ~Vecteur();

 Vecteur & operator=(const Vecteur &);
 ...
};
```

## ■ Dépouiller un vecteur

```
void Vecteur::depouiller(Vecteur && victime) {
 if (tab_) delete[] tab_;
 tab_ = victime.tab_;
 taille_ = victime.taille_;
 victime.tab_ = nullptr;
 victime.taille_ = 0;
}
```

## ■ Cas d'utilisation

- ❑ Vecteur add(const Vecteur & a, const Vecteur & b)  
 { Vecteur v = ...; return v; }
- ❑ Vecteur v1 = ...;
- ❑ Vecteur v2 = ...;
- ❑ Vecteur v3 = ...;
- ❑ **v1**.depouiller(**v2**) ⇒ interdit («**v2**» n'est pas une *rvalue*)
- ❑ **v1**.depouiller(add(v2,v3)) ⇒ OK



- Dépouillement  $\Rightarrow$  l'objet n'est plus utilisable...
- ...sauf qu'il doit être détruit !

$\Rightarrow$  Conserver une certaine cohérence des objets dépouillés

- Pour que l'appel au destructeur libère bien les ressources restantes

- Une possibilité: échanger les contenus

```
void Vecteur::depouiller(Vecteur && victime) {
 std::swap(tab_,victime.tab_);
 std::swap(taille_,victime.taille_);
}
```

- Le dépouillement peut s'avérer utile pour optimiser la copie d'objets

# Opérateurs de mouvement (1/3)

- Exemple: `v3 = v1 + v2;`
  - Opérateur «+»
    - ⇒ construction variable locale
    - ⇒ retour variable locale par copie
  - Affectation
    - ⇒ copie du retour
- Pire des cas (sans optimisation) ⇒ 2 copies inutiles du tableau
  - Construction par copie (retour) + affectation
  - Remarque: l'optimisation évite normalement la construction par copie du retour (cf. *copy elision*, optimisation garantie en C++17 dans certaines conditions)
- Depuis C++11: 2 nouveaux opérateurs pour optimiser la copie
  - Constructeur de mouvement / *move constructor*
    - `Vecteur(Vecteur && v)`
  - Affectation par mouvement / *move assignment*
    - `Vecteur & operator=(Vecteur && v)`

- Constructeur de mouvement

```
Vecteur(Vecteur && v)
: tab_(v.tab_), taille_(v.taille_) {
 v.tab_ = nullptr;
 v.taille_ = 0;
}
```

- Affectation par mouvement

```
Vecteur & operator=(Vecteur && v) {
 std::swap(tab_, v.tab_);
 std::swap(taille_, v.taille_);
 return *this;
}
```

- Remarque: pas d'intérêt à capter spécifiquement une *rvalue* constante

- Sélection automatique de l'opérateur le plus adapté
  - ❑ Pas d'opérateur de mouvement  $\Rightarrow$  opérateur de copie
  - ❑ Opérateurs de mouvement + copie disponibles
    - Argument = *lvalue* ou *rvalue* constante  $\Rightarrow$  opérateur de copie
    - Argument = *rvalue* non constante  $\Rightarrow$  opérateur de mouvement
- Quand définir ces opérateurs de mouvement ?
  - ❑ Lorsque la copie d'un objet est coûteuse
  - ❑ La bibliothèque standard utilisera ces opérateurs autant que possible
- Sous certaines conditions, opérateurs disponibles par défaut
  - ❑ [http://en.cppreference.com/w/cpp/language/move\\_constructor](http://en.cppreference.com/w/cpp/language/move_constructor)
  - ❑ [http://en.cppreference.com/w/cpp/language/move\\_operator](http://en.cppreference.com/w/cpp/language/move_operator)

- Comment «forcer» l'utilisation de ces opérateurs ?
  - ❑ Possibilité de convertir une *lvalue* en *rvalue*  $\Rightarrow$  `std::move`
  - ❑ Cela permet de favoriser un mouvement plutôt qu'une copie
  - ❑ Mais ensuite l'objet concerné ne doit plus être utilisé

- Exemple

```
template <typename T> inline void swap(T & a, T & b) {
 T tmp = a;
 a = b;
 b = tmp;
}
```

- Trop de copies !
  - ❑ Après chaque affectation, la valeur du membre de droite sans intérêt
  - ❑ On pourrait donc le dépouiller plutôt que le copier
  - ❑ En utilisant les opérateurs de mouvement

- Solution potentiellement plus efficace

```
template <typename T> inline void swap(T & a, T & b) {
 T tmp = std::move(a);
 a = std::move(b);
 b = std::move(tmp);
}
```

- Plus aucune copie, mais des mouvements...

- ❑ ...à condition que «T» implémente les opérateurs de mouvement

- Comment fonctionne «`std::move`» ?

- ❑ Conversion via «`static_cast`»:  $T \& \rightarrow T \&\&$
- ❑ Mais il y a quelques subtilités (cf. *collapsing rules*)

- Ne forcer la conversion que si la valeur devient inutile !

## PARTIE VIII

# Pointeurs «intelligents»

Bruno Bachelet

Loïc Yon

- Nécessaires pour une gestion dynamique de la mémoire
  - Allocation sur le tas
- Mais des risques liés à une gestion manuelle
  - Oubli de libération  $\Rightarrow$  fuite mémoire
  - Pointeur sur une zone libérée  $\Rightarrow$  comportement indéfini
- Même en faisant attention, risque d'erreur
- Exemple (testez avec Valgrind)

```
void f() {
 A * p = new A();

 if (...) throw std::exception(); // Fuite mémoire !

 delete p;
}
```



- *Smart pointers*

- ❑ Reposent sur le *design pattern* «proxy»
- ❑ Exploitent la technique RAI du C++

- Proxy = objet qui encapsule un objet

- ❑ Il se fait passer pour l'objet ⇒ interface similaire
- ❑ Il contrôle l'appel à ses méthodes

- RAI = *Resource Acquisition Is Initialization*

- ❑ Acquisition d'une ressource liée à la durée de vie de l'objet
- ❑ Construction objet ⇒ acquisition ressource
- ❑ Tant que l'objet est disponible ⇒ utilisation ressource
- ❑ Destruction de l'objet ⇒ libération ressource
- ❑ Destruction garantie même en cas d'erreur (cf. mécanisme exceptions)

- *Smart pointer* = objet qui encapsule un pointeur
- Contrôle les opérations élémentaires
  - ❑ Construction, affectation, destruction
- Réduit éventuellement l'interface
  - ❑ Pour empêcher l'accès direct à la mémoire pointée (e.g. «*weak\_ptr*»)
  - ❑ Pour empêcher la copie du pointeur (e.g. «*unique\_ptr*»)
- Complète l'interface
  - ❑ Opérateurs de mouvement pour le transfert de propriété (e.g. «*unique\_ptr*»)
  - ❑ Méthodes pour le partage de propriété (e.g. «*shared\_ptr*»)
- Réduction des risques d'erreur
  - ❑ Destruction du *smart pointer* ⇒ libération de la mémoire liée au pointeur
  - ❑ Libération après avoir appelé le destructeur des données pointées

- Nouvelle proposition depuis C++11
- `unique_ptr` (propriété unique)
  - ❑ Garantit un pointeur unique sur une zone mémoire
  - ❑ Garantit la libération de la mémoire
- `shared_ptr` (propriété partagée)
  - ❑ Permet plusieurs pointeurs sur une même zone mémoire
  - ❑ Comptage des pointeurs
  - ❑ Garantit la libération de la mémoire quand plus aucun pointeur  
≈ *garbage collection*
- `weak_ptr` (sans propriété)
  - ❑ Permet de s'assurer que le pointeur est toujours valide avant d'accéder à la mémoire associée

- `unique_ptr`  $\Rightarrow$  un seul pointeur sur une zone mémoire
  - ❑ `std::unique_ptr<A> p(new A);`
  - ❑ `std::unique_ptr<A[]> p(new A[10]);`
- Destruction du *smart pointer*  $\Rightarrow$  libération de la mémoire
  - ❑ Evite toute fuite mémoire
- Déréférencement possible
  - ❑ Manipulation classique des opérateurs «`*`», «`->`» et «`[]`»
  - ❑ Répercussion sur le pointeur encapsulé

- Propriété unique  $\Rightarrow$  impossible de le copier
  - ❑ Constructeur de copie impossible
  - ❑ Affectation par copie impossible
- Possibilité de mouvement  $\Rightarrow$  transfert de propriété
  - ❑ Constructeur et affectation par mouvement possibles
- Exemple de transfert de propriété
  - ❑ `p1 = std::move(p2);`
  - ❑ «p1» pointe où «p2» pointait
  - ❑ «p2» pointe sur «`nullptr`»
- Abandon de propriété
  - ❑ Méthode «`release`»

## ■ Exemple

```
void f() {
 std::unique_ptr<A> p1; // Pointeur vide

 {
 std::unique_ptr<A> p2(new A);
 std::unique_ptr<A[]> p3(new A[3]);

 p1 = std::move(p2); // Transfert de propriété
 // Destruction p3 ⇒ libération mémoire (3 objets)
 // Destruction p2 ⇒ rien ne se passe
 }

 // Destruction p1 ⇒ libération mémoire (1 objet)
}
```

# Cas d'utilisations d'un *unique\_ptr* (1/2)

---

## ■ Garantir la destruction d'une zone mémoire renvoyée

- ❑ 

```
std::unique_ptr<A> getA() {
 std::unique_ptr<A> p(new A);
 ...
 return p; // Pas de copie (car optimisation)
 ⇒ retour en rvalue
}
```
- ❑ 

```
std::unique_ptr<A> x = getA();
// Opération de mouvement ⇒ transfert de propriété
// (Destruction rvalue ⇒ rien ne se passe)
...
// Destruction x ⇒ libération mémoire
```

# Cas d'utilisations d'un *unique\_ptr* (2/2)

---

## ■ Transmettre un pointeur unique en argument

```
❑ void display(std::unique_ptr<A> x) {
 std::cout << *x << std::endl;
 // Destruction de x ⇒ libération de la mémoire
}

❑ {
 std::unique_ptr<A> p(new A);

 display(std::move(p));
 // Mouvement ⇒ transfert de propriété
 ...
 // Destruction de p ⇒ rien ne se passe
}
```



- *shared\_ptr*  $\Rightarrow$  plusieurs pointeurs sur une même zone
  - `std::shared_ptr<A> p1(new A);`
  - `std::shared_ptr<A> p2 = p1;`
- Propriété multiple  $\Rightarrow$  copie autorisée
- Les *smart pointers* sont comptés et partagent le compteur
  - Accès au compteur via la méthode «`use_count`»
- Destruction *smart pointer*  $\Rightarrow$  décrémentation compteur
- Changement de pointeur  $\Rightarrow$  décrémentation compteur
  - Via opérateur «`=`» ou méthode «`reset`»
- Compteur = 0  $\Rightarrow$  libération de la mémoire

## ■ Exemple

```
void f() {
 std::shared_ptr<A> p1(new A);
 std::shared_ptr<A> p2; // Pointeur vide

 {
 std::shared_ptr<A> p3(new A);

 p2 = p3; // p2 et p3 pointent sur la même zone
 p1.reset(new A); // Destruction de l'objet pointé
 // et pointage sur le nouvel objet

 std::cout << p3.use_count() << std::endl; // ⇒ 2

 // Destruction de p3 ⇒ compteur = 1 ⇒ rien ne se passe
 }

 // Destruction de p2 ⇒ compteur = 0 ⇒ libération mémoire
 // Destruction de p1 ⇒ compteur = 0 ⇒ libération mémoire
}
```

- *weak\_ptr* = pointeur sur mémoire gérée par «*shared\_ptr*»
- N'acquiert pas la propriété
  - ❑ Pas d'impact sur le compteur
  - ❑ Pas d'impact sur la destruction
- Déréférencement impossible directement
  - ❑ Il faut obtenir un «*shared\_ptr*»
  - ❑ Via la méthode «*lock*»
- Test de validité du pointeur
  - ❑ Appel méthode «*expired*»
- Permet un accès fiable à la donnée pointée

- Exemple d'accès (invalide) sans *smart pointer*

```
A * p1 = new A;
A * p2 = p1;
...
delete p1;
...
std::cout << *p2 << std::endl;
```

- Exemple d'accès (sécurisé) avec *smart pointer*

```
std::shared_ptr<A> p1(new A);
std::weak_ptr<A> p2 = p1;
...
p1.reset(); // Libération mémoire, p1 pointe sur «nullptr»
...
auto p3 = p2.lock();
if (p3) std::cout << *p3 << std::endl;
```

- Allocation mémoire  $\Rightarrow$  pointeur immédiatement dans un *smart pointer*
  - ❑ Juste à la sortie du «new»
  - ❑ Ne pas utiliser le pointeur brut
  - ❑ Ne pas le détruire

- Exemple de problème

```
A * p = new A;
std::shared_ptr<A> p1(p); // Compteur = 1
std::shared_ptr<A> p2(p); // Compteur = 1
// Destruction $\Rightarrow$ erreur à la seconde exécution
```

- Solution

```
std::shared_ptr<A> p1(new A); // compteur = 1
std::shared_ptr<A> p2 = p1; // compteur = 2
```

- Recommandation: utiliser «make\_shared» et «make\_unique»

- `make_shared<T>(a1,...,an)` / `make_unique<T>(a1,...,an)`
  - ❑ Effectuent l'allocation et retournent un *smart pointer*
  - ❑ Se chargent de l'appel au constructeur: `T(a1,...,an)`
- Exemple: `auto p = make_shared<A>();`
- Avantages
  - ❑ Aucun accès au pointeur  $\Rightarrow$  évite tout risque de fuite
  - ❑ Simplifie l'écriture: évite la répétition du type du *smart pointer* avec «`auto`»
  - ❑ Pour «`make_shared`»: plus efficace
    - Allocation groupée: compteurs (pointeurs *shared* et *weak*) + objet(s) pointé(s)
- Inconvénients
  - ❑ Fonctionne seulement avec les constructeurs publics de la donnée
  - ❑ Problème aussi avec les structures «agrégats» (i.e. construction avec «`{}`»)
  - ❑ Pour «`make_shared`»: libération de la mémoire possiblement différée
    - Allocation groupée  $\Rightarrow$  libération impossible si pointeur *weak* actif (besoin compteurs)

## PARTIE IX

# Bibliothèque standard

Bruno Bachelet

Loïc Yon

- Généralités
  - Historique
  - Espaces de nommage
- Grands principes
  - Séparation conteneurs-algorithmes
  - Itérateurs
  - Foncteurs / Lambdas
- Conteneurs
  - Conteneurs de séquences
  - Conteneurs adaptateurs
  - Conteneurs associatifs
- Algorithmes



- Concept de généricité introduit dès les années 70
  - ⇒ Développement de structures de données et d'algorithmes génériques
- *Standard Template Library*
  - ❑ Travaux d'Alexander Stepanov
  - ❑ Premiers développements en 1979
  - ❑ Portage en ADA en 1987
  - ❑ Portage en C++ en 1992
- Normalisée en 1994, puis intégrée à la norme C++98
- STL fait partie de la bibliothèque standard du C++
  - ❑ Concerne la partie conteneurs (structures de données) et algorithmes

## ■ *Namespaces*

- ❑ Permettent d'organiser les composants en modules
- ❑ Mais leur fonction est très limitée
- ❑ Déterminent simplement une zone avec un nom
- ❑ Aucune règle d'accessibilité (privé, public...)

## ■ Evitent les collisions de nom

- ❑ `std::vector`  $\neq$  `boost::mpl::vector`  $\neq$  `boost::fusion::vector`

## ■ Permettent de regrouper des fonctions et des classes

- ❑ Interface d'une classe = méthodes mais aussi fonctions
  - Les opérateurs binaires (externes) notamment
- ❑ Résolution de la surcharge d'une fonction
  - Les surcharges dans les *namespaces* des arguments sont considérées

- Mot-clé «`namespace`» ⇒ délimite un bloc

```
namespace monespace {
 class A { ... };
 void f();
 using t = ...;
}
```

- Tous les composants à l'intérieur du bloc sont préfixés

- `monespace::A`, `monespace::f`, `monespace::t`...

- Peut être ouvert autant de fois que nécessaire

```
namespace monespace { class A; }
...
namespace monespace { void f(); }
```

- S'utilise aussi bien dans «`.hpp`» que dans «`.cpp`»

- Une déclaration et sa définition doivent être dans le même *namespace*

- Imbrication de *namespaces* possible

```
namespace monespace {
 void f();
 ...
 namespace monsousespace {
 void g();
 ...
 }
}
```

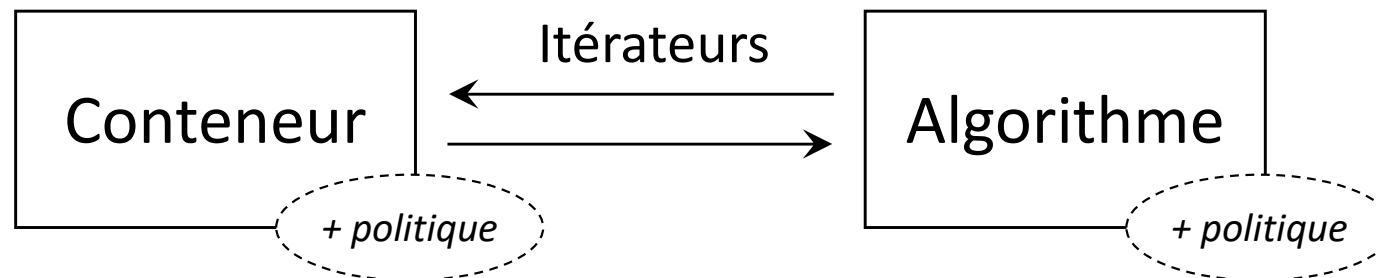
- Utiliser un composant provenant d'un *namespace*
  - ❑ `monespace::f();`
  - ❑ `monespace::monsousespace::g();`
- Préfixe « `::` » seul  $\Rightarrow$  référence au *namespace* global
- Possibilité de créer des alias
  - ❑ `namespace fus = boost::fusion;`

- Possibilité d'«importer» des symboles
  - Pour éviter d'écrire le préfixe
- Importer un symbole: **déclaration** «using»
  - `using std::vector;`
  - `vector<int> v; // Utilisation implicite de «std::vector»`
  - `std::string s;`
- Importer tous les symboles: **directive** «using»
  - `using namespace std;`
  - `vector<int> v;`
  - `string s;`
- Conseils pratiques
  - Ne jamais mettre d'importation dans un fichier entête (.hpp)
  - Préférer les déclarations aux directives dans un fichier d'implémentation (.cpp)

- «Petit mais costaud»
  - ❑ Fournir des classes compactes
  - ❑ Spécialisées / centrées autour d'une fonctionnalité
  - ❑ Avec uniquement les méthodes essentielles
- Séparation des conteneurs et des algorithmes
  - ❑ Impossible de prévoir tous les algorithmes d'un conteneur
  - ⇒ Algorithmes définis à part des conteneurs
- Stratégies d'accès / parcours des conteneurs
  - ❑ Pourquoi lier un algorithme à une stratégie de parcours ?
  - ❑ Pourquoi lier un algorithme à un conteneur spécifique ?
  - ⇒ Abstraction du conteneur et de la stratégie de parcours: les «itérateurs»
- Algorithmes «génériques»
  - ❑ Pouvoir utiliser un algorithme dans un maximum de situations (e.g. tri)
  - ⇒ Algorithmes «à trous» via les «politiques» (e.g. foncteurs, lambdas)

# Interaction conteneur-algorithmes

- En général, trois entités nécessaires pour manipuler un conteneur
  - ❑ Un conteneur pour le stockage des objets
  - ❑ Des itérateurs pour les accès aux objets
  - ❑ Des algorithmes pour la manipulation des objets
  - ❑ Optionnel: politiques pour paramétrer les algorithmes et/ou les conteneurs
- Fonctionnement conjoint conteneur-algorithmes
  - ❑ Les algorithmes opèrent sur le conteneur via des itérateurs



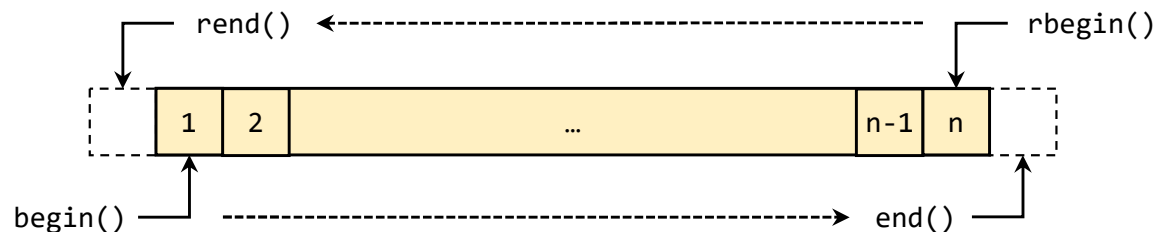
- Parcourir un conteneur  $\Rightarrow$  un intermédiaire
  - ❑ Permet des parcours simultanés
  - ❑ Permet de parcourir une sous-partie du conteneur
  - ❑ Permet de faire abstraction du conteneur
  - ❑ Permet différentes stratégies d'accès (lecture/écriture) et de parcours (sens)
- Un itérateur est un objet
  - ❑ Qui pointe sur un élément d'un conteneur
  - ❑ Qui permet de passer d'un élément à un autre dans le conteneur
- API indépendante de la véritable structure de données
- Il s'agit d'un *design pattern* commun



- Rend un algorithme indépendant du conteneur sous-jacent
  - Manipulation homogène de l'itérateur quel que soit le conteneur
  - Il peut même ne pas y avoir de conteneur derrière un itérateur
    - Séquences générées à la volée, lecture/écriture dans un flux...
- Par rapport à un parcours avec indice
  - Beaucoup plus efficace pour certaines structures de données
    - Exemples: liste, arbre
  - Différents types de parcours possibles sur une même séquence
    - Exemples: parcours préfixe, infixé et postfixé
  - Modification de la séquence en cours d'itération possible
- Implémentation d'un itérateur
  - Il doit souvent connaître l'implémentation de son conteneur
  - Deux possibilités: classe amie ou classe imbriquée
  - Le conteneur doit fournir des méthodes pour produire des itérateurs

- Interface d'un itérateur en C++
  - Forme canonique de Coplien
    - Constructeur par défaut
    - Constructeur par copie
    - Opérateur d'affectation
    - Destructeur
  - API et sémantique du pointeur (arithmétique partielle)
    - Opérateurs de comparaison « != » et « == »
      - Attention: ne pas utiliser l'opérateur « < »
    - Opérateur de déréférencement « \* »
    - Opérateurs d'incrément « ++ » (préfixé et postfixé)
- Manipulation similaire à celle des pointeurs  
⇒ tableaux et conteneurs STL manipulables indifféremment

- 4 types d'itérateurs associés à chaque conteneur
  - Types imbriqués
    - `type_conteneur::iterator`
    - `type_conteneur::const_iterator`
    - `type_conteneur::reverse_iterator`
    - `type_conteneur::const_reverse_iterator`
  - `const` = accès en lecteur seule
  - `reverse` = parcours inversé (dernier → premier)
- «Balises» (itérateurs repères) fournies par le conteneur



- Du premier au dernier: `conteneur.begin()` → `conteneur.end()`
- Du dernier au premier: `conteneur.rbegin()` → `conteneur.rend()`

- Parcours d'un conteneur à l'aide d'itérateurs

- ❑ `conteneur_t c;`  
...  
`conteneur_t::iterator it; // Accès avec écriture`  
  
`for (it = c.begin(); it != c.end(); ++it) do_something(*it);`

- L'algorithme «`find`» retourne un itérateur sur l'élément trouvé

- ❑ Sinon retourne la balise de fin du conteneur
  - ❑ Permet une opération immédiate sur l'objet
  - ❑ Complexité de l'accès au suivant:  $O(1)$
  - ❑ `conteneur_t c;`  
...  
`conteneur_t::iterator it;`  
  
`it = std::find(c.begin(), c.end(), elt);`  
`if (it != c.end()) do_something(*it);`

# Boucle «for» simplifiée (1/2)

- Depuis C++11, syntaxe simplifiée pour le parcours de collections
  - ❑ Pour les tableaux de taille fixe (i.e. taille connue à la compilation)
  - ❑ Pour les conteneurs standards (ou tout conteneur respectant l'API)
- `for (élément : conteneur)`
  - ❑ *élément* = variable qui représente l'élément parcouru à chaque itération
- Exemple pour les tableaux
  - ❑ `float t[10];`
  - ❑ `for (float & v : t) v *= 2; // Accès avec écriture`
  - ❑ `for (float v : t) std::cout << v << " "; // Accès avec lecture seule`
- Code équivalent à une boucle avec indices
  - ❑ `for (unsigned i = 0; i < 10; ++i) t[i] *= 2;`
  - ❑ `for (unsigned i = 0; i < 10; ++i) std::cout << t[i] << " ";`

# Boucle «for» simplifiée (2/2)

## ■ Exemple pour les conteneurs standards

- ❑ `std::list<Point> points;`
- ❑ `for (Point & p : points) p.x += dx;`
- ❑ `for (const Point & p : points) std::cout << p.x << " ";`

## ■ Code équivalent à une boucle avec itérateurs

- ❑ 

```
std::list<Point>::iterator it = points.begin();
while (it != points.end()) {
 (*it).x += dx; // Accès avec écriture
 ++it;
}
```
- ❑ 

```
std::list<Point>::const_iterator it = points.begin();
while (it != points.end()) {
 std::cout << (*it).x << " "; // Accès lecture seule
 ++it;
}
```

## ■ S'applique à toute classe disposant de l'API des itérateurs

- Tous les itérateurs ne fournissent pas les mêmes fonctionnalités
  - de parcours
    - Exemple: impossible de reculer un itérateur sur une liste simplement chaînée
  - de manipulation de l'élément
    - Exemple: impossible de modifier un élément
- «Concepts» pour spécifier différents types d'itérateurs
- Importants pour écrire des algorithmes génériques
  - Documenter les fonctionnalités requises par les itérateurs
  - Proposer des implémentations spécialisées pour certains itérateurs

- **InputIterator**

- Accès à l'élément en lecture + avancée dans la séquence

- **OutputIterator**

- Accès à l'élément en écriture + avancée dans la séquence

- **BidirectionalIterator**

- **InputIterator** + recul dans la séquence

- **RandomAccessIterator**

- **BidirectionalIterator** + «saut» dans la séquence

- Concepts formalisés en C++20 (cf. **Legacy\*Iterator**)

- Liste complète: <http://en.cppreference.com/w/cpp/iterator>



- Avant C++20, concepts implicites dans les noms des paramètres

```
template <typename InputIterator,
 typename OutputIterator>
OutputIterator copy(InputIterator first,
 InputIterator last,
 OutputIterator result) {
 while (first != last) *result++ = *first++;
 return result;
}
```

- Spécialisation en fonction du concept: `std::advance(it,n)`

- ❑ Contrainte: «`it`» doit modéliser «`InputIterator`»
- ❑ Si «`it`» modélise «`BidirectionalIterator`»  $\Rightarrow$  autoriser «`n`» négatif
- ❑ Si «`it`» modélise «`RandomAccessIterator`»
  - Implémentation en temps constant  $O(1)$ : `it += n;`
- ❑ Sinon
  - Implémentation en temps linéaire  $O(n)$ : `if (n > 0) while (n-- > 0) ++it;`

- Politique = fonction représentée sous la forme d'un objet
  - ❑ Permet l'écriture d'algorithmes «à trous»
  - ❑ A l'exécution, on passe une politique à l'algorithme
  - ❑ La politique comble les trous de l'algorithme
- Il s'agit du *design pattern* «stratégie»
- Intérêts
  - ❑ Paramétrisation des algorithmes
  - ❑ Possibilité d'avoir un état interne (via les attributs)
- La bibliothèque standard privilégie les «foncteurs»
  - ❑ Foncteur = politique qui a l'apparence d'une fonction  
⇒ fonctions et foncteurs manipulables indifféremment
- Depuis C++11: expressions lambdas  
⇒ génération implicite de foncteurs

- Exemple: algorithme de tri

```
template <typename T>
void trier(vector<T> & v) {
 for (int i = 0; i < v.size()-1; ++i)
 for (int j = i+1; j < v.size(); ++j)
 if (v[j] < v[i]) std::swap(v[i],v[j]);
}
```

- Pas très flexible

- ❑ «T» doit implémenter l'opérateur «<»
- ❑ Comment faire un tri décroissant ?

- Solution: passer la relation d'ordre en paramètre sous forme d'objet

```
template <typename T, typename R>
void trier(vector<T> & v, const R & rel) {
 for (int i = 0; i < v.size()-1; ++i)
 for (int j = i+1; j < v.size(); ++j)
 if (rel.estAvant(v[j],v[i])) std::swap(v[i],v[j]);
}
```

- Implémentation d'une politique (relation d'ordre)

- ❑ `template <typename T> class OrdreCroissant {`  
    `public: bool estAvant(const T & a, const T & b) const { return a < b; }`  
    `};`
- ❑ `template <typename T> class OrdreDecroissant {`  
    `public: bool estAvant(const T & a, const T & b) const { return a > b; }`  
    `};`
- ❑ Exemple d'appel: `trier(v,OrdreCroissant<int>());`

- Foncteur = politique qui a l'apparence d'une fonction
  - Surcharge de l'opérateur «`()`»
  - Arité en fonction du besoin (e.g. relation d'ordre  $\Rightarrow$  2 arguments)
  - Syntaxe: `type_retour operator()(arguments)`

- Exemple: relation d'ordre

```
template <typename T> class OrdreCroissant {
 public: bool operator()(const T & a, const T & b) const { return a < b; }
};
```

- Exemple: algorithme de tri

```
template <typename T, typename R>
void trier(vector<T> & v, const R & rel) {
 for (int i = 0; i < v.size()-1; ++i)
 for (int j = i+1; j < v.size(); ++j)
 if (rel(v[j],v[i])) std::swap(v[i],v[j]);
}
```

- Objet  $\Rightarrow$  possibilité d'un état interne conservé par les attributs
- Exemple: générateur de nombres pairs
  - ❑ Opérateur «`()`» (non constant) et sans paramètres
  - ❑ Retourne le prochain nombre pair

- Implémentation

```
class GenerateurPair {
 private:
 int val;
 public:
 GenerateurPair() : val(0) {}
 int operator()() { val += 2; return val; }
};
```

- Exemple d'utilisation

- ❑ `GenerateurPair gen;`
- ❑ `std::cout << gen() << ' ' << gen() << std::endl;`

- Arithmétique: addition, soustraction, multiplication, division...
  - `plus<T>`, `minus<T>`, `multiplies<T>`, `divides<T>`...
- Comparaison: inférieur, supérieur, égal...
  - `less<T>`, `less_equal<T>`, `equal_to<T>`...
- Logique: et, ou, non
  - `logical_and<T>`, `logical_or<T>`...
- Utilisent simplement les opérateurs correspondants
  - `plus<T>`  $\Rightarrow$  `T operator()(const T & a, const T & b) const`  
`{ return a + b; }`
  - `less<T>`  $\Rightarrow$  `bool operator()(const T & a, const T & b) const`  
`{ return a < b; }`
  - `logical_and<T>`  $\Rightarrow$  `bool operator()(const T & a, const T & b) const`  
`{ return a && b; }`

- Algorithme générique  $\Rightarrow$  algorithme à trous
  - ❑ `std::sort(v.begin(), v.end(), std::greater<int>());`
  - ❑ Dernier paramètre = foncteur ou pointeur de fonction
  
- Implémentation à l'aide de la généricité
  - ❑ `template <typename IT, typename COMP>`  
`void sort(const IT & debut, const IT & fin,`  
`const COMP & comparer);`
  - ❑ `comparer()`  $\Rightarrow$  appel opérateur «`()`» si foncteur
  - ❑ `comparer()`  $\Rightarrow$  appel fonction si pointeur de fonction
  
- Souvent, créer un foncteur est fastidieux
  - ❑ Trouver un nom
  - ❑ Ecrire la classe
  - ❑ Pour un usage souvent ponctuel



## ■ Expression lambda (depuis C++11)

- ❑ Permet l'écriture d'une fonction à la volée
- ❑ Pour un usage ponctuel
- ❑ Fonction «anonyme»
- ❑ Fonction «contextualisée» (cf. mécanisme de capture)

## ■ Exemple

- ❑ 

```
std::sort(v.begin(),v.end(),
 [] (int x, int y) { return x > y; });
```
- ❑ Tri par ordre décroissant

## ■ Syntaxe: `[capture] (arguments) -> retour { code }`

- ❑ Arguments, retour, code = éléments d'une fonction normale
  - Remarque: utilisation de la syntaxe alternative (depuis C++11) de retour de fonction
- ❑ Capture = liste des variables du contexte «capturées» par la lambda

- `[] (int x, int y) { return x > y; }`
  - Type de retour déduit automatiquement
    - A condition que tous les retours soient du même type
  - Equivalent à: `[] (int x, int y) -> bool { return x > y; }`
  
- Equivalent au foncteur suivant
  - ```
struct Anonyme {  
    bool operator()(int x, int y) const  
    { return x > y; }  
};
```
 - Remarque: opérateur «`()`» constant

- Ou à la fonction suivante
 - `inline bool anonyme(int x, int y) { return x > y; }`

Implémentation des lambdas

- Implémentation libre des lambdas

- Souvent sous la forme d'un foncteur
- Mais pour les lambdas sans capture, une fonction suffit

- Dépend donc du compilateur

⇒ impossible de connaître *a priori* le type d'une lambda

- Mais possibilité de stocker une lambda dans une variable

- `auto f = [] (int x, int y) { return x > y; };`
- `if (f(v[i],v[j])) ...`
- `auto` ⇒ type de la lambda identifié par le compilateur pour déclarer «f»

- Et aussi de «capter» le type d'une lambda

- `using lambda_t = decltype(f);`
- `typeid(lambda_t).name()` ⇒ `main::{Lambda(int, int)#2}` (g++ 4.8.3)
- `decltype` ⇒ type de l'expression (ici variable «f») demandé au compilateur

- Une lambda peut utiliser des variables de son contexte
⇒ mécanisme de «capture»
- Exemple: filtrer les valeurs d'un échantillon
 - ❑ `std::replace_if(v.begin(),v.end(),filtre,-1);`
 - ❑ `filtre` = prédicat (politique de «test»)
 - ❑ Prédicat testé sur chaque élément «x» ⇒ `if (filtre(x)) ...`
 - ❑ Prédicat vérifié ⇒ valeur de «x» remplacée par «-1»
- Capture des données de l'intervalle du filtre

```
int min = ...;
int max = ...;
...
auto filtre = [min,max] (int x) { return (x<min || x>max); };
```
- Variables capturées listées dans «[...]»
 - ❑ Variable utilisée sans être capturée ⇒ erreur

■ Deux types de captures

- ❑ `[variable]` $\Rightarrow$ capture par copie
- ❑ `[&variable]` $\Rightarrow$ capture par référence

■ Capture par copie = copie de la variable capturée

- ❑ Modification de la variable dans le contexte $\Rightarrow$ aucun impact dans la lambda

■ Exemple

```
int min = 5;
int max = 7;

...
auto filtre = [min,max] (int x) { return (x<min || x>max); };
...
min = 3;
max = 10;
...
replace_if(v.begin(),v.end(),filtre,-1); // filtre = [5;7]
```

- Capture par référence = référence sur la variable capturée
 - Evite la copie (important pour les gros objets)
 - Modification de la variable dans le contexte $\Rightarrow$ impact dans la lambda
 - Attention à la durée de vie des variables capturées par référence

- Exemple

```
int min = 5;
int max = 7;
...
auto filtre = [&min,&max] (int x) { return (x<min || x>max); };
...
min = 3;
max = 10;
...
replace_if(v.begin(),v.end(),filtre,-1); // filtre = [3;10]
```

■ Capture automatique possible

- ❑ Variable utilisée $\Rightarrow$ variable capturée
- ❑ Seules les variables utilisées dans la lambda sont capturées
- ❑ `[]` $\Rightarrow$ aucune capture
- ❑ `[=]` $\Rightarrow$ capture automatique par copie
- ❑ `[&]` $\Rightarrow$ capture automatique par référence

■ Exemple: capture automatique par copie

```
int min = 5;
```

```
int max = 7;
```

```
...
```

```
auto filtre = [=] (int x) { return (x < min || x > max); };
```

- Capture de «`this`»

- `[this]` ⇒ capture du pointeur de l'objet du contexte

- Exemple

```
class Statistique {  
    private:  
        int min_;  
        int max_;  
  
    public:  
        ...  
        void filtrer(vector<int> & v) const {  
            auto filtre = [this] (int x) {  
                return (x < this->min_ || x > this->max_);  
            };  
  
            replace_if(v.begin(),v.end(),filtre,-1);  
        }  
};
```


- Lambda avec capture $\Rightarrow$ implémentation par foncteur

- Capture par copie

- `[min,max] (int x) { return (x<min || x>max); };`

- Foncteur équivalent

```
struct Anonyme {
```

```
    int min;
```

```
    int max;
```

```
    Anonyme(int a, int b) : min(a), max(b) {}
```

```
    bool operator()(int x) const {
```

```
        return (x<min || x>max);
```

```
    }
```

```
};
```

- Capture par référence

- `[&min,&max] (int x) { return (x<min || x>max); };`

- Foncteur équivalent

```
struct Anonyme {  
    int & min;  
    int & max;
```

```
    Anonyme(int & a, int & b) : min(a), max(b) {}
```

```
    bool operator()(int x) const {  
        return (x<min || x>max);  
    }  
};
```

- Opérateur « `()` » constant dans les exemples précédents
- Rappel: dans une méthode constante...
 - ❑ Les attributs deviennent constants
 - ❑ Mais attention au cas des pointeurs/références
 - ❑ Les pointeurs/références sont constants mais pas les objets référencés !
- Par défaut, une lambda est «constante»
⇒ implémentation d'un foncteur avec opérateur « `()` » constant
- Lambda constante
⇒ les variables capturées par copie sont constantes
 - ❑ Car les variables deviennent des attributs du foncteur
 - ❑ Capture par copie ⇒ attribut valeur ⇒ variable capturée constante
 - ❑ Capture par référence ⇒ attribut référence ⇒ variable capturée modifiable

- Lambda non constante $\Rightarrow$ mot-clé «`mutable`»
 - $\Rightarrow$ Modification possible des variables capturées par copie
 - $\Rightarrow$ Foncteur avec opérateur «`()`» non constant

- Exemple: produire des nombres pairs

```
int cpt = 32;
...
auto gen = [cpt] () mutable {
    cpt += 2;
    return cpt;
};
...
std::generate(v.begin(),v.end(),gen);
```

- Attention: une lambda peut donc être un objet non constant
 - ❑ `template <typename LAMBDA> void algo(const LAMBDA &)  $\Rightarrow$  erreur possible`

- Trois manières de modéliser une fonction
 - Pointeur de fonction
 - Une méthode est considérée comme une fonction dont le 1^{er} argument est le pointeur de l'objet
 - Foncteur
 - Objet avec opérateur « `()` »
 - Lambda
 - Type inconnu
 - Implémentation comme fonction ou foncteur
- Types différents, mais même manière d'être appelés
- Comment faire abstraction de ces trois types ?
- Objectif: algorithme recevant indifféremment en paramètre un pointeur de fonction, un foncteur ou une lambda

Abstraction de fonction par généricité (1/2)

- Une approche: abstraction par un paramètre générique
 - ❑ Avantage: très efficace
 - Instanciation adaptée au type de modélisation
 - ❑ Inconvénient: difficile de contrôler le paramètre
 - Comment être sûr qu'il représente bien une fonction ?
 - Solution alternative: abstraction par un «adapteur», mais surcoût (cf. `std::function`)

- Passage par référence constante ?
 - ❑ `template <typename F> void algo(const F & f);`
 - ❑ Problème pour les lambdas/foncteurs non constants

- Passage par référence non constante ?
 - ❑ `template <typename F> void algo(F & f);`
 - ❑ Problème pour les *rvalues* ou les pointeurs de fonction
 - ❑ Et souvent une lambda est une *rvalue*: `algo([...] (...) {...});`

Abstraction de fonction par généricité (2/2)

- Passage par copie ?

⇒ inefficacité

- Solution: passage par «référence universelle»

- ❑ Appelée aussi *forwarding reference* (cf. *collapsing rules*)
- ❑ `template <typename F> void algo(F && f);`
- ❑ Accepte des valeurs constantes ou non
- ❑ Accepte des *lvalues* ou des *rvalues*

- Exemple

```
template <typename IT, typename GEN>
void generate(const IT & debut, const IT & fin, GEN && generer) {
    for (IT it = debut; it != fin; ++it) *it = generer();
}
```

- Quatre grandes classes de conteneurs
 - ❑ Séquences élémentaires
 - Vecteur, liste, file à double entrée
 - Tableau statique (depuis C++11)
 - ❑ Adaptations des séquences élémentaires
 - Pile, file, file à priorité
 - ❑ Conteneurs associatifs triés
 - Ensemble avec/sans unicité
 - Association avec clé unique/multiple
 - ❑ Conteneurs associatifs non triés (depuis C++11)

- Utilisation intensive de la généricité
 - ❑ Type de données
 - ❑ Allocateur de mémoire
 - ❑ Comparateur
 - ❑ ...

- Choix du conteneur ?
- Selon les fonctionnalités disponibles
 - ❑ Un morceau d'API commun
 - ❑ Un morceau d'API spécifique à chaque conteneur
- Selon la complexité des opérations
 - ❑ Opérations en $O(1)$, $O(\log n)$, $O(n)$
 - ❑ Parfois amortie
- Critères de choix
 - ❑ Chercher le conteneur le plus «naturel» pour l'algo voulu
 - ❑ Analyser la complexité du traitement
 - ❑ Chercher le conteneur offrant la meilleure complexité globale

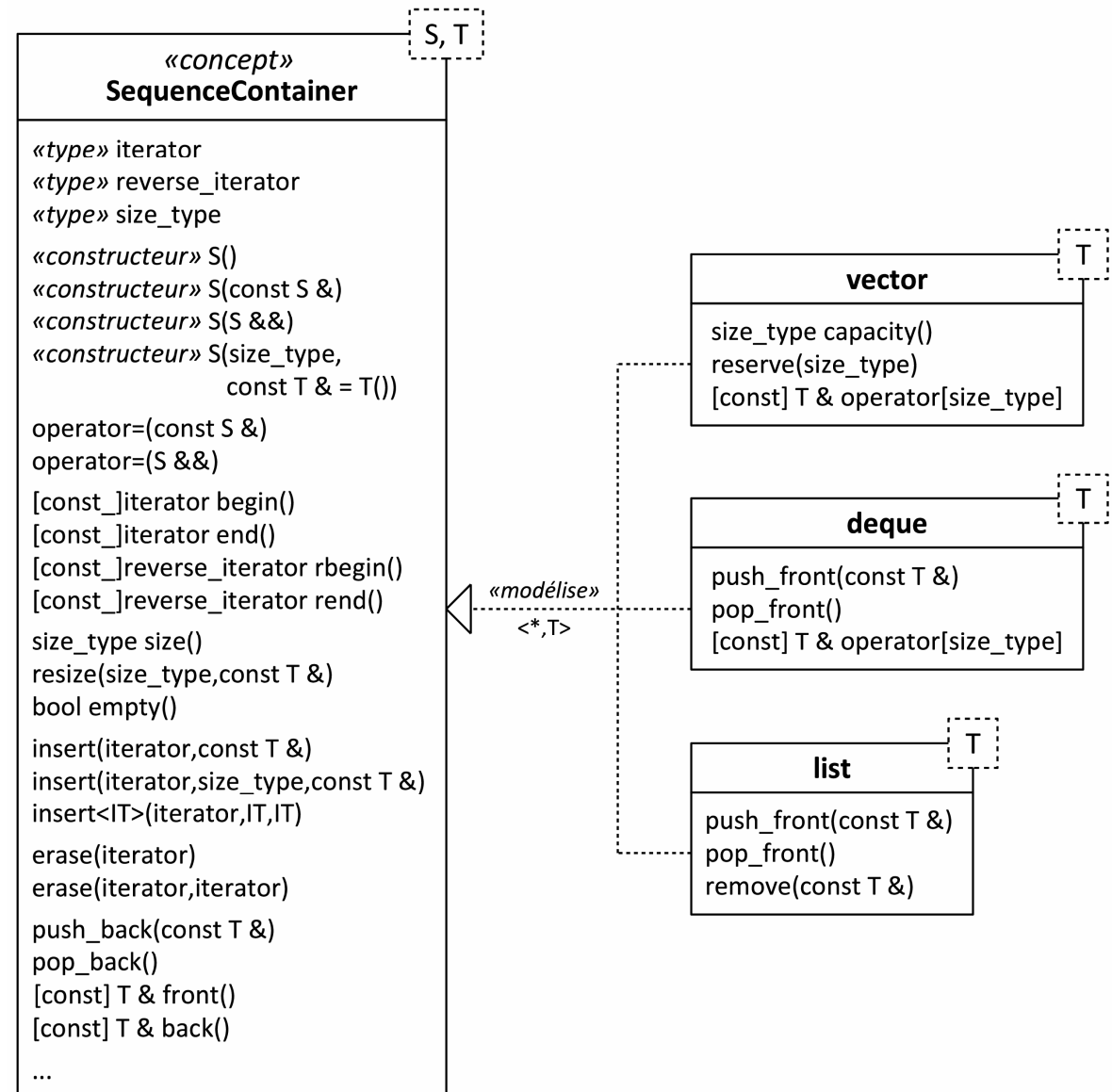
- Des fonctionnalités communes
- Forme canonique de Coplien (+ opérateurs de mouvement)
- Dimensionnement automatique de la capacité
 - Excepté pour le tableau statique (`array`)
 - Exemple du vecteur
 - Lorsque l'insertion d'un élément survient en limite de capacité
 - Augmentation de la capacité
- Balises des itérateurs
- Quelques méthodes
 - `size_type C::size() const` // Nombre d'éléments
 - `size_type C::max_size() const` // Nombre max d'éléments
 - `bool C::empty() const` // Est vide ?
 - `void C::swap(C & conteneur)` // Echange de contenu
 - `void C::clear()` // Vide le conteneur

Conteneurs en séquence (1/2)

- Vecteur
(`vector<T>`)
- Liste doublement chaînée
(`list<T>`)
- File à double entrée
(`deque<T>`)

Depuis C++11

- Tableau
(`array<T,N>`)
- Liste simplement chaînée
(`forward_list<T>`)



- Quelques méthodes communes

- Insertion (avant la position indiquée)

- ❑ `void S::insert(S::iterator pos, T & elt)`
- ❑ `template <typename InputIterator>`
`void S::insert(S::iterator pos,`
`InputIterator debut, InputIterator fin)`

- Suppression

- ❑ `S::iterator S::erase(S::iterator pos)`
- ❑ `S::iterator S::erase(S::iterator debut, S::iterator fin)`

- Accès / ajout en tête et fin

- ❑ `void S::push_back(const T & elt)`
- ❑ `void S::pop_back()`
- ❑ `T & S::front() | const T & S::front() const`
- ❑ `T & S::back() | const T & S::back() const`

■ Tableau qui se redimensionne automatiquement

- Éléments contigus en mémoire (compatibilité avec les tableaux C)

■ Efficacité

- + Accès direct aux éléments (opérateur «`[]`») en $O(1)$
- + Ajout / suppression en fin en $O(1)$ (amorti)
- Ajout / suppression ailleurs en $O(n)$

■ Utilisation

- Entête: `<vector>`
- Déclaration: `std::vector<T> v;`

■ Méthodes spécifiques

- Contrôle capacité

- `size_type V::capacity() const` // Capacité actuelle
- `void V::reserve(size_type nb)` // Ajustement capacité

- Accès par indice aux éléments

- `T & V::operator[](size_type id)` // Lecture/écriture
- `const T & V::operator[](size_type id) const` // Lecture seule

- Liste doublement chaînée
- Efficacité
 - + Ajout / suppression n'importe où en $O(1)$
 - Pas d'accès direct aux éléments
- Utilisation
 - Entête: `<list>`
 - Déclaration: `std::list<T> l;`
- Méthodes spécifiques
 - Ajout / suppression en tête
 - `void L::push_front(const T & elt)`
 - `void L::pop_front()`
 - Suppression d'un élément
 - `void L::remove(const T & elt)`

- Similaire au vecteur + possibilité d'opérations en tête
 - Contiguïté des éléments non garantie
- Efficacité
 - + Accès direct aux éléments (opérateur «`[]`») en $O(1)$
 - + Ajout/suppression en tête et fin en $O(1)$ (amorti)
 - Ajout/suppression ailleurs en $O(n)$
- Utilisation
 - Entête: `<deque>`
 - Déclaration: `std::deque<T> d;`
- Méthodes spécifiques
 - Ajout / suppression en tête
 - `void D::push_front(const T & elt)`
 - `void D::pop_front()`
 - Accès par indice aux éléments
 - `T & D::operator[](size_type id)`
 - `const T & D::operator[](size_type id) const`

File à double entrée (2/2)

```
std::deque<std::string> file;

file.push_back("...");
file.push_front("B");
file.push_front("A");
file.push_back("Y");
file.push_back("Z");

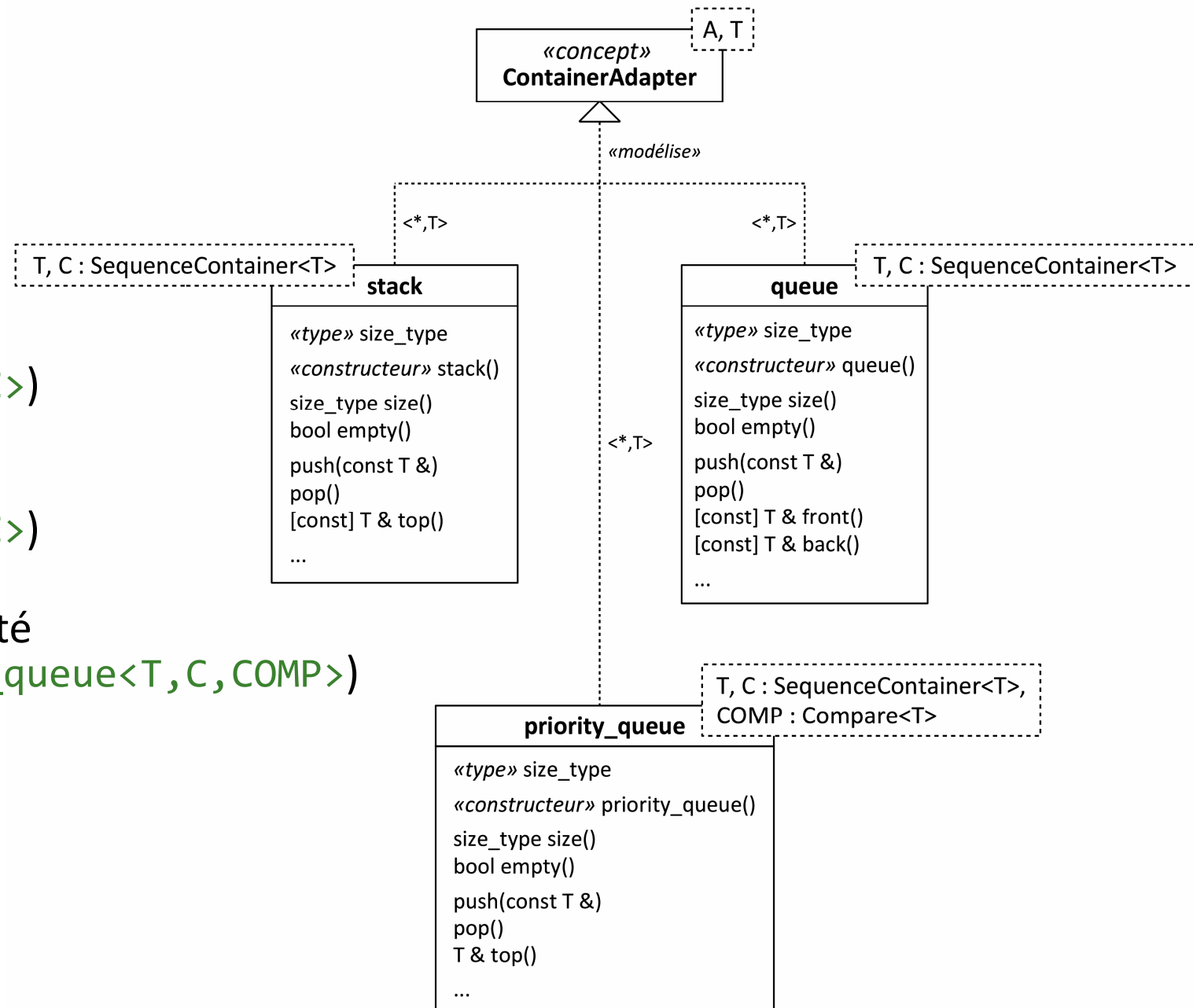
for (const std::string & s : file)
    std::cout << s << " "; // ⇒ A B ... Y Z

file.pop_front();
file.pop_back();

for (const std::string & s : file)
    std::cout << s << " "; // ⇒ B ... Y
```


Conteneurs adaptateurs (1/3)

- Pile
(`stack<T,C>`)
- File
(`queue<T,C>`)
- File à priorité
(`priority_queue<T,C,COMP>`)

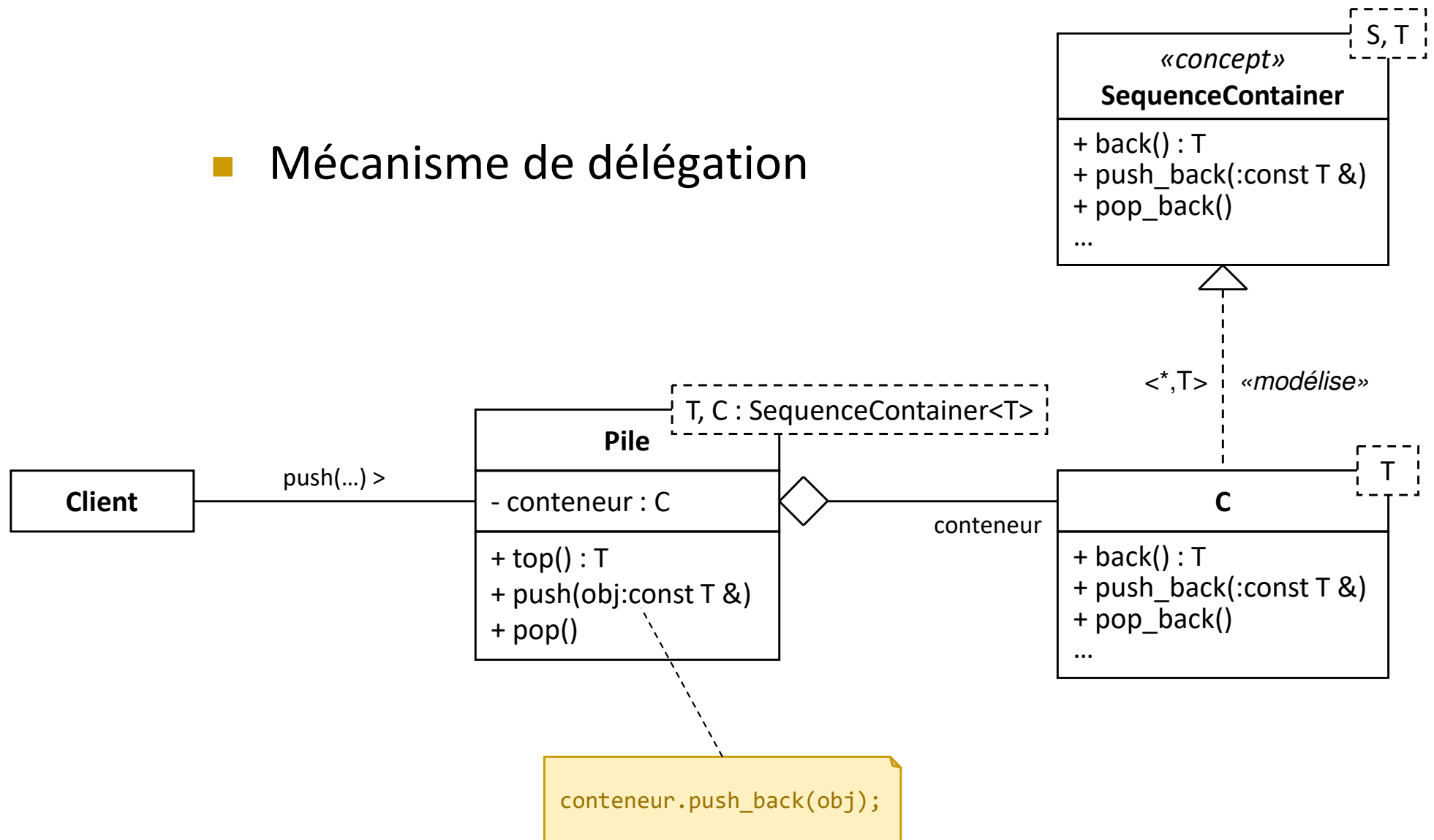


- Définis à partir d'un conteneur en séquence
 - ❑ Celui-ci est paramétrable
 - ❑ Utilise la structure de données du conteneur

- Propose une API spécifique
 - ❑ Celle-ci est réduite
 - ❑ Pas d'itérateurs

- Mécanisme de délégation
 - ❑ Agrégation du conteneur
 - ❑ Délégation des opérations au conteneur

■ Mécanisme de délégation



- Accès seulement au sommet de la pile
 - Pas de possibilité de voir les éléments empilés
- Utilisation
 - Entête: `<stack>`
 - Déclaration
 - `std::stack<T> s; // C = deque<T>`
 - `std::stack<T, std::vector<T>> s;`
- Méthodes spécifiques
 - Empilement / dépilement
 - `void S::push(const T & elt)`
 - `void S::pop()`
 - Accès au sommet
 - `T & S::top()`
 - `const T & S::top() const`

- Structure FIFO (*First In First Out*)
 - Ajout en fin, retrait en tête
 - Pas de possibilité de voir les éléments dans la file
- Utilisation
 - Entête: `<queue>`
 - Déclaration
 - `std::queue<T> q; // C = deque<T>`
 - `std::queue<T, std::list<T>> q;`
 - Ne peut pas utiliser «vector» (manque «pop_front»)
- Méthodes spécifiques
 - Ajout / retrait
 - `void Q::push(const T & elt)`
 - `void Q::pop()`
 - Accès aux extrémités
 - `T & Q::front()` | `const T & Q::front() const`
 - `T & Q::back()` | `const T & Q::back() const`

■ File d'attente à priorité

- ❑ Ajout en fin, retrait de l'élément le plus «grand» ⇒ politique «comparateur»
- ❑ Pas de possibilité de voir les éléments dans la file

■ Utilisation

- ❑ Entête: `<queue>`
- ❑ Déclaration
 - `std::priority_queue<T> p; // C = vector<T>, COMP = less<T>`
 - `std::priority_queue<T, std::deque<T>, std::greater<T>> q;`
 - Ne peut pas utiliser «`list`» (manque opérateur «`[]`»)

■ Méthodes spécifiques

- ❑ Constructeur (qui attend une politique «comparateur»)
 - `P::P(const COMP & c = COMP())`
- ❑ Ajout / retrait
 - `void P::push(const T & elt)`
 - `void P::pop()`
- ❑ Accès au plus grand
 - `T & P::top() | const T & P::top() const`

```
std::priority_queue<int, std::vector<int>, std::greater<int>> file;
```

```
// std::greater ⇒ ordre décroissant
```

```
// ⇒ le plus petit est le plus prioritaire
```

```
file.push(23);
```

```
file.push(12);
```

```
file.push(99);
```

```
file.push(3);
```

```
while (!file.empty()) {
```

```
    int v = file.top();
```

```
    std::cout << v << " ";
```

```
    file.pop(); // Dépilement plus petit → plus grand
```

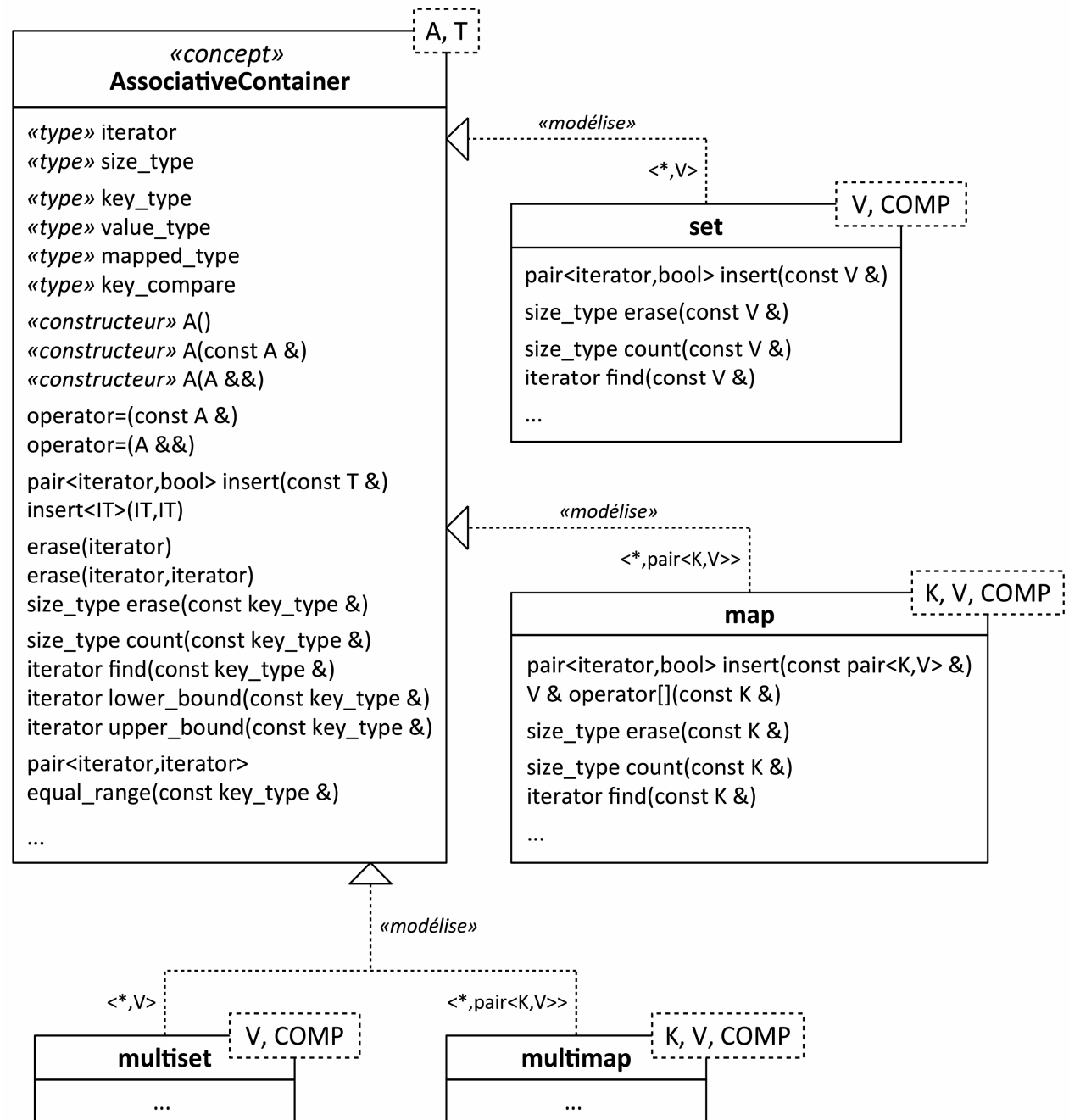
```
} // ⇒ 3 12 23 99
```

Conteneurs associatifs (1/5)

- Ensemble avec unicité (`set<V,COMP>`)
- Ensemble sans unicité (`multiset<V,COMP>`)
- Association avec unicité (`map<K,V,COMP>`)
- Association sans unicité (`multimap<K,V,COMP>`)

Depuis C++11

- Versions non triées
 - `unordered_set`
 - `unordered_multiset`
 - `unordered_map`
 - `unordered_multimap`



- Principe de l'association
 - ❑ Associer une clé à chaque élément
 - ❑ On accède à l'élément par sa clé
- Structure utilisée pour l'association: `std::pair`

```
template <typename T1, typename T2>
struct pair {
    T1 first;
    T2 second;

    pair() {}

    pair(const T1 & t1, const T2 & t2)
        : first(t1), second(t2) {}
};
```

- Création d'une paire

- ❑ `p = std::pair<int,double>(13,27.14);`
- ❑ Obligé d'écrire les types paramètres de la paire

- Pour éviter d'écrire les types: `std::make_pair()`

```
template <typename T1, typename T2>  
std::pair<T1,T2> make_pair(const T1 & a, const T2 & b)  
{ return std::pair<T1,T2>(a,b); }
```

- Utilise la déduction de types du compilateur

- ❑ `p = std::make_pair(13,27.14);`
- ⇔ `std::make_pair<int,double>(13,27.14);`

- Conteneurs associatifs triés sur la clé
 - ❑ Nécessitent une relation d'ordre sur les clés $\Rightarrow$ politique «comparateur»
 - ❑ Représentation interne typique: RB-tree
- Ensembles: «**set**» ou «**multiset**»
 - ❑ L'élément contient sa clé ou «est» la clé
- Associations: «**map**» ou «**multimap**»
 - ❑ Les éléments stockés sont des paires clé-valeur
 - ❑ **first** = clé
 - ❑ **second** = valeur associée
- Clé unique ou multiple ?
 - ❑ Unicité $\Rightarrow$ «**set**» ou «**map**»
 - ❑ Multiplicité $\Rightarrow$ «**multiset**» ou «**multimap**»

- «set» et «multiset» $\Rightarrow$ un seul paramètre
 - `set<V> | multiset<V>`
 - Éléments stockés: `T = V`
- «map» et «multimap» $\Rightarrow$ deux paramètres
 - `map<K,V> | multimap<K,V>`
 - Éléments stockés: `T = std::pair<const K,V>`
- Quelques méthodes communes (1/3)
- Constructeur (qui attend une politique «comparateur»)
 - `A::A(const COMP & c = COMP())`
- Insertion
 - `pair<A::iterator,bool> A::insert(const T & elt)`
 - `template <typename InputIterator>`
`void A::insert(InputIterator deb, InputIterator fin)`

■ Quelques méthodes communes (2/3)

■ Suppressions

- ❑ `void A::erase(A::iterator pos)`
- ❑ `void A::erase(A::iterator deb, A::iterator fin)`
- ❑ `size_type A::erase(const A::key_type & cle)`

■ Recherche élément

- ❑ `size_type A::count(const A::key_type & cle) const`
- ❑ Nombre d'éléments ayant la clé fournie
- ❑ `A::iterator A::find(const A::key_type & cle) const`
- ❑ Itérateur sur le premier élément ayant la clé fournie ou «`A::end()`» sinon

■ Quelques méthodes communes (3/3)

■ Intervalle

- ❑ `A::iterator A::lower_bound(const A::key_type & cle) const`
- ❑ Itérateur sur le 1^{er} élément dont la clé est égale ou supérieure à celle fournie
- ❑ `A::iterator A::upper_bound(const A::key_type & cle) const`
- ❑ Itérateur sur le 1^{er} élément dont la clé est supérieure à celle fournie
- ❑ `pair<A::iterator,A::iterator>`
`A::equal_range(const A::key_type & cle) const`
- ❑ Fournit un encadrement des éléments ayant la clé fournie
(intervalle défini par «`lower_bound`» et «`upper_bound`»)

- Conteneur trié d'éléments contenant leur propre clé
- Utilisation
 - ❑ Entête: `<set>`
 - ❑ Déclaration
 - `std::set<V> s; // COMP = less<V>`
 - `std::set<V, std::greater<V>> s;`
- Méthodes spécifiques
 - ❑ Insertion dans «`set`»
 - ❑ `pair<S::iterator, bool> S::insert(const V & elt)`
 - ❑ Insertion dans «`multiset`»
 - ❑ `M::iterator M::insert(const V & elt)`

```
struct Personne { std::string nom; std::string prenom; };
```

```
struct CompPersonne {  
    bool operator()(const Personne & p1, const Personne & p2) const {  
        return p1.nom < p2.nom || (p1.nom == p2.nom && p1.prenom < p2.prenom);  
    }  
};
```

```
std::multiset<Personne,CompPersonne> ensemble;
```

```
ensemble.insert(Personne{"Doe","John"});  
ensemble.insert(Personne{"Smith","John"});  
ensemble.insert(Personne{"Doe","Jane"});
```

```
for (const Personne & p : ensemble) {  
    std::cout << p.nom << ";" << p.prenom << " ";  
} // ⇒ Doe;Jane Doe;John Smith;John
```


- Conteneur trié d'éléments associés à une clé
- Utilisation
 - ❑ Entête: `<map>`
 - ❑ Déclaration
 - `std::map<K,V> s; // COMP = less<K>`
 - `std::map<K,V,std::greater<K>> s;`
- Méthodes spécifiques
 - ❑ Insertion
 - ❑ `pair<M::iterator,bool> M::insert(const pair<const K,V> & elt)`
 - ❑ Accès élément
 - ❑ `V & M::operator[](const K & cle)`

- Remarques sur l'opérateur « `[]` »
- Permet un accès indexé similaire au vecteur
 - Indice = clé
 - Complexité d'accès en $O(\log n)$
- Attention: si la clé n'existe pas dans le conteneur, elle est ajoutée et associée à l'élément par défaut (`v()`)
- Il est conseillé d'utiliser l'opérateur « `[]` » pour...
 - l'écriture (insertion)
 - la lecture dont on est sûr de l'existence de la clé
- Si on n'est pas sûr de l'existence d'une clé
 - Appel préalable à « `find` » ou « `count` »
 - Utilisation des itérateurs pour parcourir

■ Exemple (1/2)

```
std::multimap<std::string, Personne> asso;
```

```
asso.insert(std::make_pair("homme", Personne{"Smith", "John"}));
```

```
asso.insert(std::make_pair("homme", Personne{"Doe", "John"}));
```

```
asso.insert(std::make_pair("femme", Personne{"Doe", "Jane"}));
```

```
for (const std::pair<std::string, Personne> & a : asso) {
```

```
    std::cout << a.first << "=" << a.second.nom
```

```
                << ";" << a.second.prenom << " ";
```

```
} // ⇒ femme=Doe;Jane homme=Smith;John homme=Doe;John
```

■ Exemple (2/2)

```
using iter_t = std::multimap<std::string,Personne>::iterator;

std::pair<iter_t,iter_t> hommes = asso.equal_range("homme");

iter_t it = hommes.first;

while (it != hommes.second) {
    std::cout << it->second.nom << ";"
               << it->second.prenom << " ";
    ++it;
} // ⇒ Smith;John Doe;John
```

- Les conteneurs définissent des types internes
 - Embarqués dans les classes
- Pour tous les conteneurs
 - `C::value_type` = type des éléments stockés
 - Pour les associations: `pair<const K,V>`
 - `C::iterator` (et variations) = type des itérateurs du conteneur
- Pour les conteneurs associatifs
 - `C::key_type` = type des clés
 - Pour les associations: `K`
 - Pour les ensembles: `V`
 - `C::mapped_type` = type des valeurs
 - `C::key_compare` = comparateur des clés

- Fonctionnalités classiques/récurrentes
 - ❑ Chercher, compter
 - ❑ Copier, insérer, supprimer
 - ❑ Remplir, transformer, trier
 - ❑ ...

- Indépendants des conteneurs $\Rightarrow$ manipulation d'itérateurs
 - ❑ Paramètres génériques: tout type d'itérateur/pointeur
 - ❑ Itérateurs de début et de fin = séquence où lire les éléments
 - ❑ Parfois itérateur de sortie pour écrire le résultat

- Algorithmes à trous $\Rightarrow$ souvent paramétrés par une politique
 - ❑ Paramètre générique: pointeur de fonction, foncteur ou lambda
 - ❑ Politique = comparateur, prédicat, générateur...

- Exemple: `for_each` $\Rightarrow$ traiter chaque élément d'une séquence

```
std::vector<int> v = {3,5,7,13};
```

```
// Parcours avec modification des éléments: +1 pour chacun  
std::for_each(v.begin(),v.end(),[] (int & x) { x += 1; });
```

```
for (int x : v) std::cout << x << " "; //  $\Rightarrow$  4 6 8 14
```

```
int s = 0;
```

```
// Parcours sans modification des éléments: calcul somme  
std::for_each(v.begin(),v.end(),[&s] (int x) { s += x; });
```

```
std::cout << "s = " << s << std::endl; //  $\Rightarrow$  s = 32
```

■ Recherche d'un élément dans une séquence

- ❑ `it = std::find(it_debut, it_fin, valeur)`
⇒ recherche $x = \text{valeur}$
- ❑ `it = std::find_if(it_debut, it_fin, predicat)`
⇒ recherche x tel que $\text{predicat}(x) = \text{vrai}$
- ❑ Si non trouvé ⇒ `it = it_fin`

■ Exemple (suite précédent)

- ❑ `auto it = std::find_if(v.begin(), v.end(),
[] (int x) { return x%3 == 0; });`
- ❑ `if (it != v.end()) std::cout << *it << std::endl; // ⇒ 6`

■ Comptage des éléments d'une séquence

- ❑ `n = std::count(it_début, it_fin, valeur)`
- ❑ `n = std::count_if(it_début, it_fin, predicat)`

- Copie des éléments d'une séquence dans une autre
 - *[itérateur_début, itérateur_fin] → [itérateur_destination, ...]*
 - Tous: `it = std::copy(it_debut, it_fin, it_dest)`
 - Filtre: `it = std::copy_if(it_debut, it_fin, it_dest, predicat)`
 - «`it_dest`» pointe sur le 1^{er} élément où faire la copie
 - Utilisation des opérateurs «`*`» et «`++`» pour faire la copie et avancer
 - «`it`» pointe sur la valeur de «`it_dest`» en fin de copie

■ Exemple

```
int buffer[100];
std::list<int> liste;

// Copie vecteur → tableau
// Itérateur tableau = pointeur
int * fin = std::copy(v.begin(), v.end(), buffer);

// Copie tableau → liste
// «back_insérer» ⇒ itérateur qui appelle «push_back»
std::copy(buffer, fin, std::back_inserter(liste));
```

■ Suppression d'éléments

- ❑ `it = std::remove(it_debut, it_fin, valeur)`
- ❑ `it = std::remove_if(it_debut, it_fin, predicat)`
- ❑ Déplace seulement les éléments à la fin de la séquence
- ❑ Retourne un itérateur sur la «nouvelle fin»
- ❑ Enchaîner avec méthode «`erase`» pour supprimer réellement
- ❑ Exemple: `v.erase(std::remove(v.begin(), v.end(), 14), v.end())`

■ Remplacement des éléments

- ❑ `std::replace(it_debut, it_fin, valeur, nouvelleValeur)`
- ❑ `std::replace_if(it_debut, it_fin, predicat, nouvelleValeur)`

- Appliquer une opération à chaque élément d'une séquence
 - Résultat dans une autre séquence
 - `it = std::transform(it_debut, it_fin, it_dest, opUnaire)`
 - Ou une opération binaire sur les éléments de 2 séquences (deux-à-deux)
 - `it = std::transform(it_debut1, it_fin1, it_debut2, it_dest, opBinaire)`

■ Exemple

```
std::vector<int> v1 = {1,2,3,4,5}, v2, v3;
```

```
std::transform(v1.begin(), v1.end(), std::back_inserter(v2),  
               [] (int x) { return 2*x; });
```

```
for (int x : v2) std::cout << x << " "; // ⇒ 2 4 6 8 10
```

```
std::transform(v1.begin(), v1.end(), v2.begin(), std::back_inserter(v3),  
               [] (int x, int y) { return x + y; });
```

```
for (int x : v3) std::cout << x << " "; // ⇒ 3 6 9 12 15
```

- Remplir une séquence avec la même valeur
 - `std::fill(it_debut, it_fin, valeur)`
 - `std::fill_n(it_debut, n, valeur)`
- Ou avec une valeur produite à chaque appel d'une fonction
 - `std::generate(it_debut, it_fin, generateur)`
 - `std::generate_n(it_debut, n, generateur)`

- Exemple

```
std::vector<int> v1(4), v2(3);  
int n = 0;
```

```
std::generate(v1.begin(), v1.end(), [&n] () { return n+=2; });  
for (int x : v1) std::cout << x << " "; // ⇒ 2 4 6 8
```

```
std::generate_n(v2.begin(), v2.size(), [&n] () { return n+=2; });  
for (int x : v2) std::cout << x << " "; // ⇒ 10 12 14
```

PARTIE X

Patrons de conception *(Design Patterns)*

Bruno Bachelet

Loïc Yon

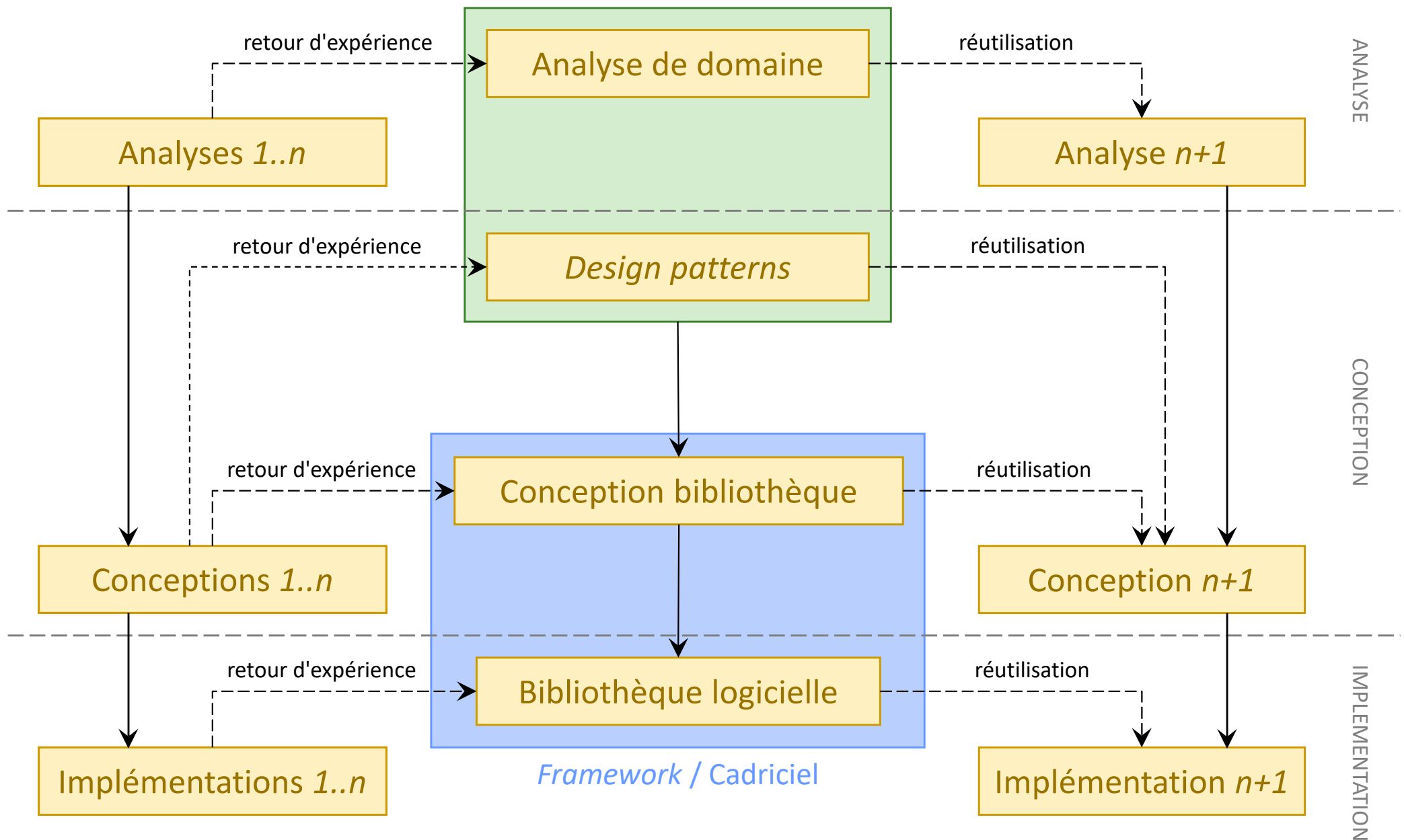
- Concevoir un système objet est difficile
- Beaucoup d'aspects à considérer
 - ❑ Décomposition du système
 - ❑ Factorisation du code
 - ❑ Relations entre les composants
 - Héritage, association, agrégation / composition, délégation
- Prévoir et intégrer dès la conception
 - ❑ Réutilisation du code
 - ❑ Evolutions / extensions possibles

⇒ introduire de la réutilisabilité

- Bénéficier des bonnes pratiques de l'industrie
 - ❑ Minimiser les risques dans la phase de développement
 - ❑ Se référer à l'existant
 - ❑ Reprendre des solutions éprouvées

- Permettre une réutilisation
 - ❑ Au niveau implémentation
 - Mêmes structures de données / algorithmes
 - ⇒ bibliothèques logicielles
 - ❑ Au niveau conception
 - Mêmes organisations des composants
 - ⇒ patrons de conception (ou «*design patterns*»)

Réutilisation à tous les niveaux



Patrons de conception (ou *design patterns*)

■ Définition

- ❑ Un *design pattern* traite un problème de conception récurrent
- ❑ Il apporte une solution générale, indépendante du contexte

■ En clair

- ❑ Description de l'organisation de classes et d'instances en interaction pour résoudre un problème de conception

■ Solution générique de conception

- ❑ Doit être compréhensible et réutilisable
- ❑ Doit être testée et validée dans l'industrie logicielle
- ❑ Doit viser un gain en terme de génie logiciel
- ❑ Doit être indépendante du contexte

■ Volonté de référencement des solutions

Patrons de conception du *GoF* (1/3)

- Les patrons présentés ici sont issus du «*GoF*»
 - ❑ «*Gang of Four*»: Gamma, Helm, Johnson, Vlissides
 - ❑ Livre fondateur: 1^{ère} proposition de référencement (1994)
 - ❑ *Design Patterns: Elements of Reusable Object-Oriented Software*
 - ❑ 23 patrons de conception

- Mais ce ne sont pas les seuls
 - ❑ Patron MVC (Modèle-Vue-Contrôleur)
 - ❑ Patrons GRASP
 - ❑ Patrons de concurrence
 - ❑ ...

- Communauté active
 - ❑ De nouveaux *patterns* proposés régulièrement
 - ❑ Démocratique: adoptés si utilisés et généraux

- Classification selon deux critères
- Cible: qui est concerné ?
 - Les classes
 - Relations d'héritage
 - Aspect statique
 - Les instances
 - Relations de composition
 - Aspect dynamique
- Objectif: que veut-on faire ?
 - Création de composants
 - Assemblage de composants
 - Comportement des composants

Patrons de conception du GoF (3/3)

Critères		Objectif		
		Création	Structure	Comportement
Cible	Classe	<i>Factory Method</i>	Adapter	<i>Interpreter</i> Template Method
	Instance	Abstract Factory <i>Builder</i> Prototype Singleton	Adapter <i>Bridge</i> Composite Decorator <i>Facade</i> <i>Flyweight</i> Proxy	<i>Chain of Responsibility</i> Command <i>Iterator</i> <i>Mediator</i> <i>Memento</i> Observer <i>State</i> Strategy Visitor

- Favoriser une bonne conception
 - ❑ Facile à appréhender
 - ❑ Facile à faire évoluer
 - ❑ Résistante aux modifications

- Quelques principes permettent de tendre à ces buts
 - ❑ Principes «SOLID»
 - Responsabilité unique (*Single responsibility*)
 - Ouvert/fermé
 - Substitution de Liskov
 - Ségrégation des Interfaces
 - Inversion des Dépendances
 - ❑ Connaissance minimale (Loi de Déméter)
 - ❑ Encapsuler ce qui varie
 - ❑ Programmer envers une interface
 - ❑ Préférer la composition à l'héritage
 - ❑ ...

- Chaque classe doit s'occuper d'une seule chose
 - ⇒ cohésion forte intra-module
 - ⇒ cohésion faible inter-module

- Une classe devrait avoir une seule raison de changer
 - ❑ Facilite la compréhension et la maintenabilité
 - ❑ Limite le risque d'introduction de bugs
 - Particulièrement lors d'évolutions
 - ❑ Facilite les tests

- Exemple: séparer le calcul de données de leur importation/exportation dans un fichier
 - ❑ Besoin de changer la manière de calculer
 - ❑ Ou besoin de changer le format d'import/export

⇒ une seule classe doit être modifiée

- Un composant doit être ouvert aux extensions...
 - Permettre l'ajout de fonctionnalités
 - Permettre la modification du comportement
- ...mais fermé aux modifications
 - Le code d'un composant ne devrait pas être modifié si les besoins évoluent
- Les besoins changent régulièrement
⇒ nécessité de pouvoir évoluer
- Tout en évitant de casser du code existant
- Exemple: les «politiques»
 - Prédicat d'un algorithme de copie (e.g. `std::copy_if`)

- Séparer les aspects susceptibles de changer de ce qui ne changera pas
- Protège contre le changement
 - ❑ Stabilité du code face aux modifications
- Flexibilité pour les comportements sujets à variation
- Exemple
 - ❑ Isoler une partie d'un comportement dans une autre classe
 - ❑ Par agrégation, possibilité de changer de classe et donc de comportement
 - ❑ Encapsulation $\Rightarrow$ modification sans impact sur le reste du code

Programmer envers une interface

- Programmer envers une interface et non envers une implémentation
- Favoriser un code sans dépendance avec les détails d'implémentation
 - ❑ Modification de l'implémentation sans impact sur le code
 - ❑ Changement d'implémentation facilitée
- Exemple
 - ❑ En C++: manipuler un vecteur avec des itérateurs plutôt qu'avec des indices
 - ❑ En Java: faire référence à un conteneur via une classe abstraite
 - `Collection<String> c = new ArrayList<String>();`
 - ❑ Dans les 2 cas, changement de conteneur ⇒ aucun changement sur le code
- Rejoint le principe «Inversion des Dépendances» de SOLID
 - ❑ Privilégier la dépendance à une interface plutôt qu'à une classe concrète

Préférer la composition à l'héritage

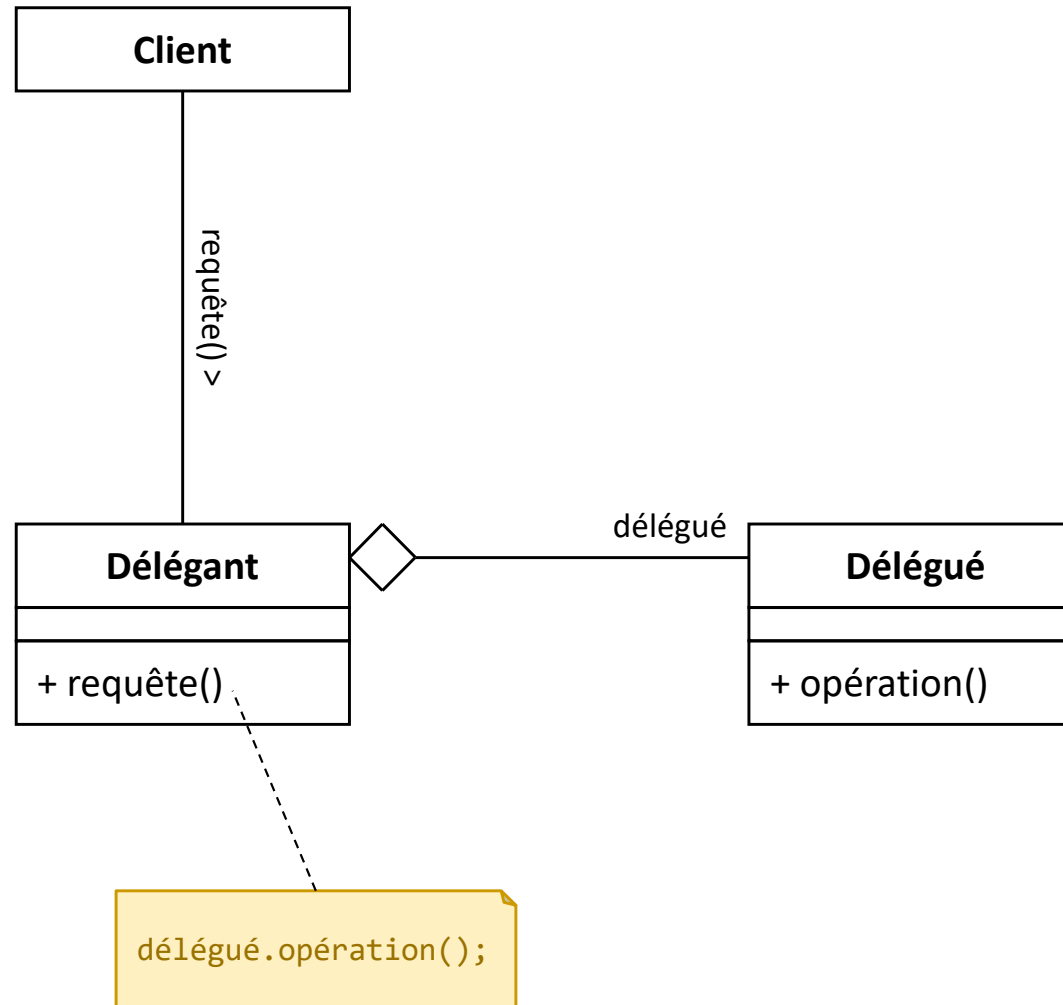
- Héritage → statique
- Composition → dynamique
- Utilisation de la délégation
 - ❑ Couplage plus faible
 - ❑ Changement de l'objet fournissant les services à l'exécution
- Permet d'éviter un héritage conceptuellement bancal
 - ❑ Une erreur classique (cf. principe de substitution de Liskov)
- Evite un accès aux données membres
 - ❑ Respect de la notion d'interface
 - ❑ Respect de l'encapsulation

- Principe
 - ❑ Rediriger un message vers un autre objet
 - ❑ Utilise la composition: délégant vers délégué

- Intervient dans de nombreux patrons du *GoF*
 - ❑ Peut être une alternative à l'héritage

- Plusieurs manières de rediriger
 - ❑ Contrôle du message (e.g. *Proxy*)
 - ❑ Changement de message (e.g. *Adapter*)
 - ❑ Changement de délégué (e.g. *Chain of Responsibility*)
 - ❑ Redéfinition du message (e.g. *Decorator*)

Mécanisme de délégation (2/2)



- Abstraction du processus de création
 - ❑ Indépendance du type réel
 - ❑ Indépendance de l'initialisation
 - ❑ Indépendance de la composition

- Niveau classe
 - ❑ Utilisation de l'héritage

- Niveau objet
 - ❑ Délégation de l'instanciation

- Utiles pour la création d'objets par composition
 - ❑ Introduit de la flexibilité dans l'assemblage

- Objectif
 - ❑ Garantir une seule instance pour une classe
 - ❑ Fournir un point d'accès global à cette instance
- Principe
 - ❑ Empêcher toute construction ou copie de cette classe
 - Impossibilité de créer un objet en dehors de la classe elle-même
 - ❑ Fournir une méthode de classe qui retourne l'objet unique
- Motivation
 - ❑ Représentation de ressources physiques uniques
 - ❑ Exemple: flux d'entrée et sortie standards

Singleton / Singleton (2/2)

■ Exemple C++

```
class Singleton {  
    private:  
        static Singleton unique;  
        // Attributs du singleton  
  
        Singleton(...) { ... }  
        Singleton(const Singleton &) = delete;  
        Singleton & operator=(const Singleton &) = delete;  
  
    public:  
        static Singleton & getInstance() { return unique; }  
        // Méthodes du singleton  
};  
  
Singleton Singleton::unique(...);
```

Singleton
- <u>unique</u> : Singleton - état : Etat
- « <i>constructeur</i> » Singleton(...) + <u>getInstance()</u> : Singleton + opérations() + getEtat() : Etat

- Constructions et copies d'un objet interdites ⇒ opérateurs privés ou supprimés
- Seule possibilité: utiliser l'instance unique via «**getInstance**»

Fabrique abstraite / *Abstract Factory* (1/3)

■ Objectif

- ❑ Créer une famille d'objets cohérents
- ❑ Des objets de classes différentes sont à créer
- ❑ Mais les classes doivent être cohérentes entre elles

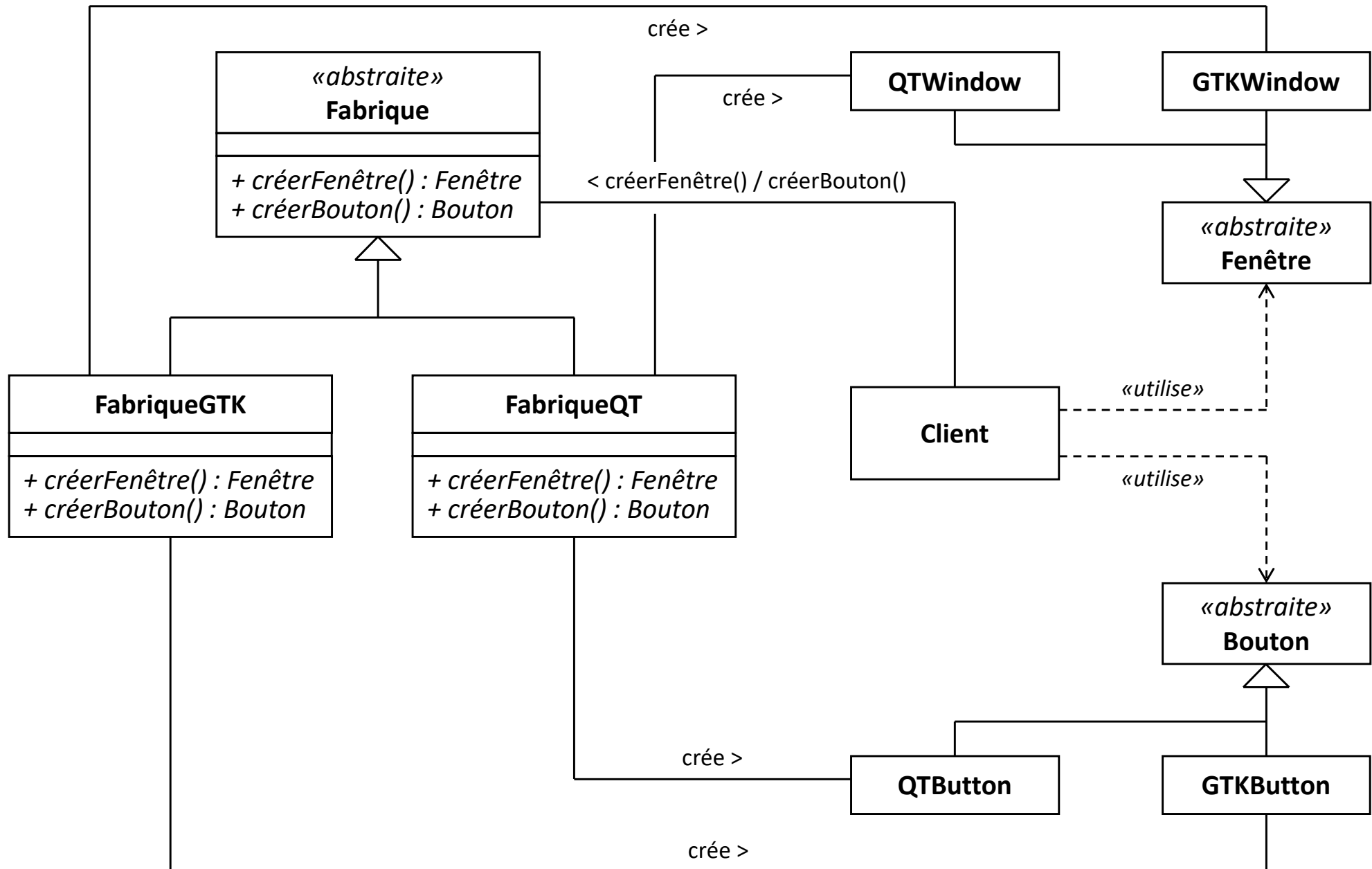
■ Principe

- ❑ Fournir une interface pour créer une famille d'objets ⇒ la «fabrique»
- ❑ Le client demande à la fabrique de lui fournir des instances
 - Le client ne connaît que les interfaces des objets
 - Seule la fabrique connaît les classes réelles des objets

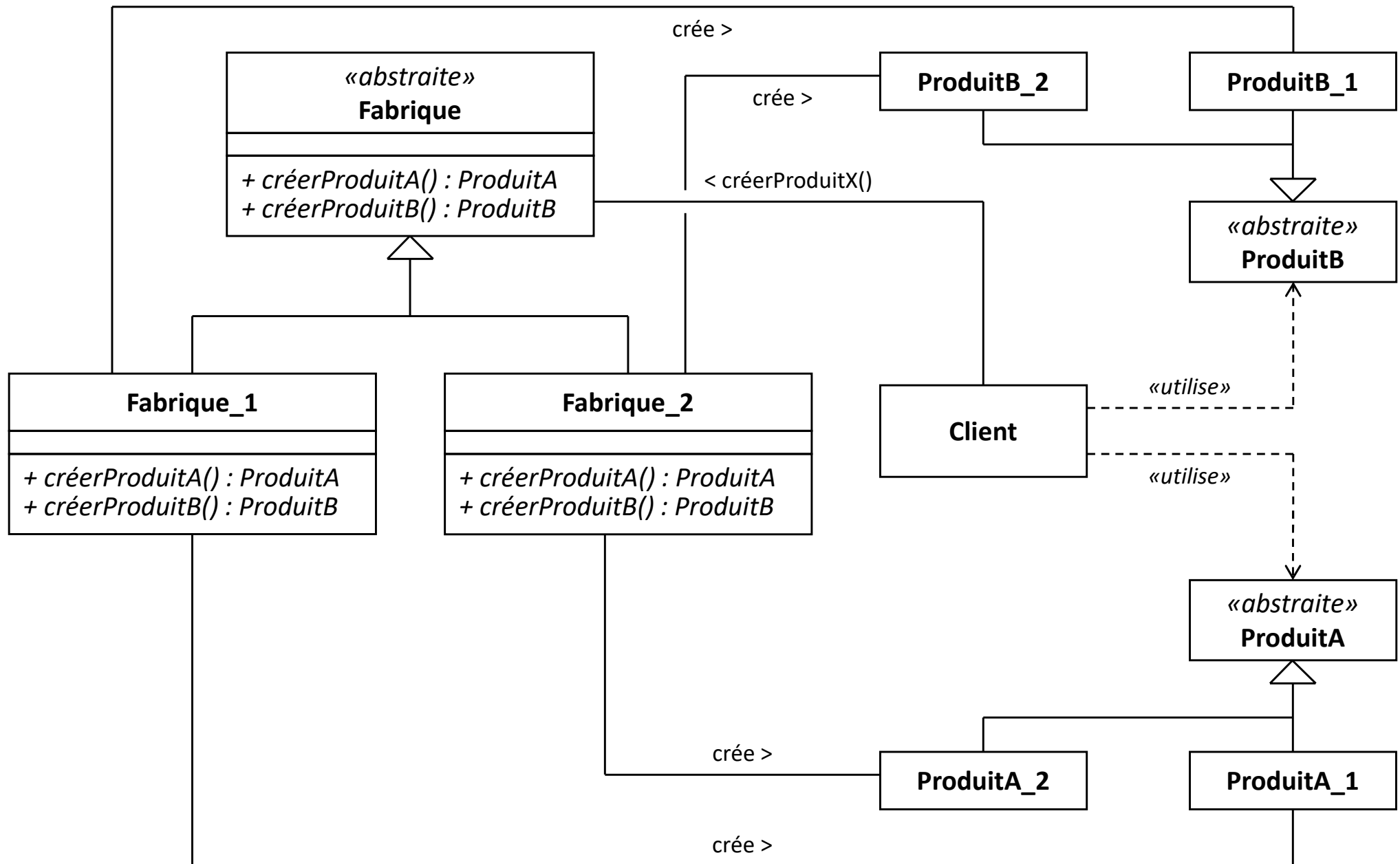
■ Motivation

- ❑ Système indépendant de l'interface graphique
- ❑ Créer des composants graphiques cohérents selon la plateforme

Fabrique abstraite / Abstract Factory (2/3)



Fabrique abstraite / Abstract Factory (3/3)



■ Objectif

- ❑ Créer un objet par clonage d'une instance modèle
- ❑ Le type de l'objet est déterminé par celui de l'instance modèle

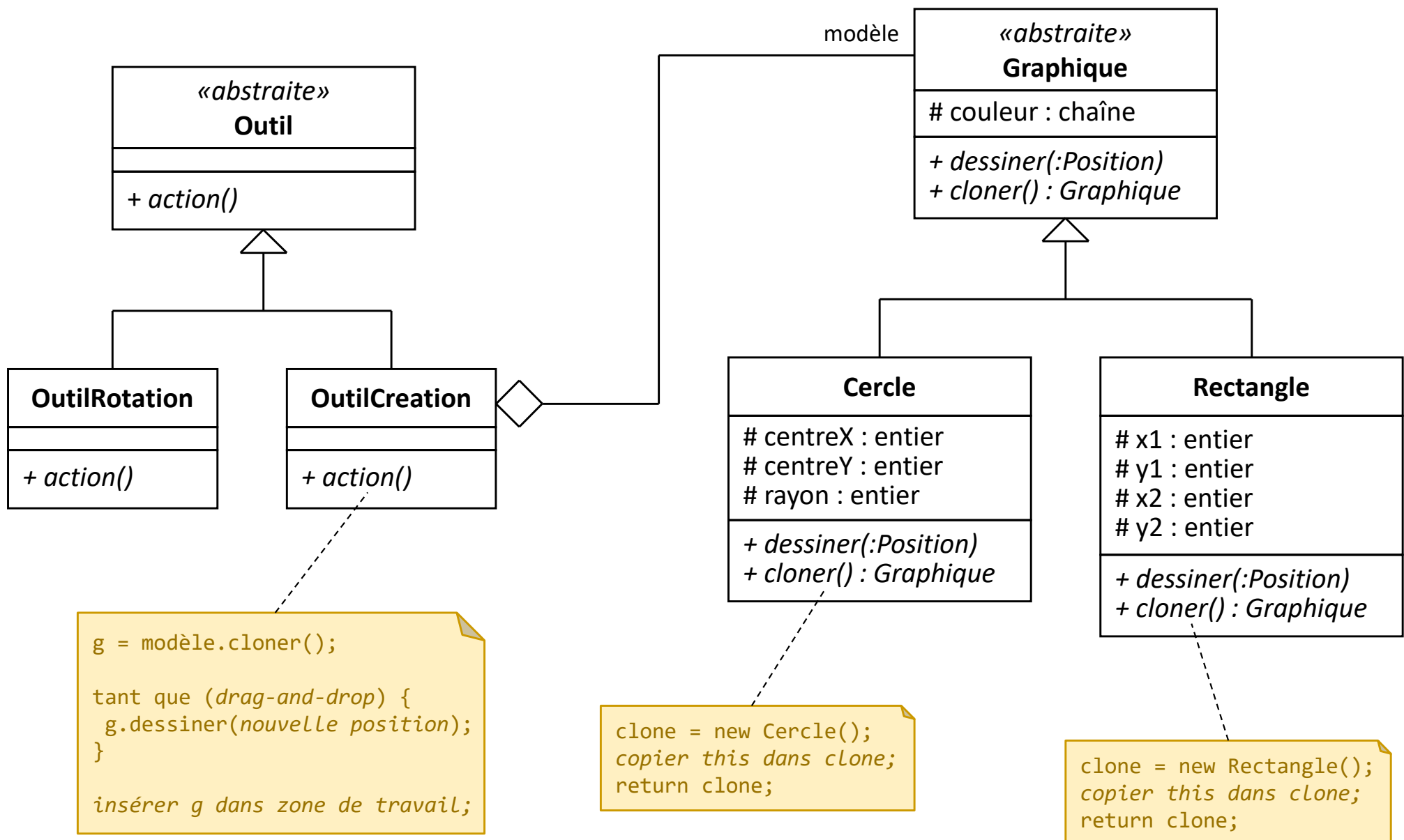
■ Principe

- ❑ Un objet «prototype» est fourni
- ❑ Il possède une méthode de clonage
- ❑ Le client utilise cette méthode pour obtenir une copie de l'objet

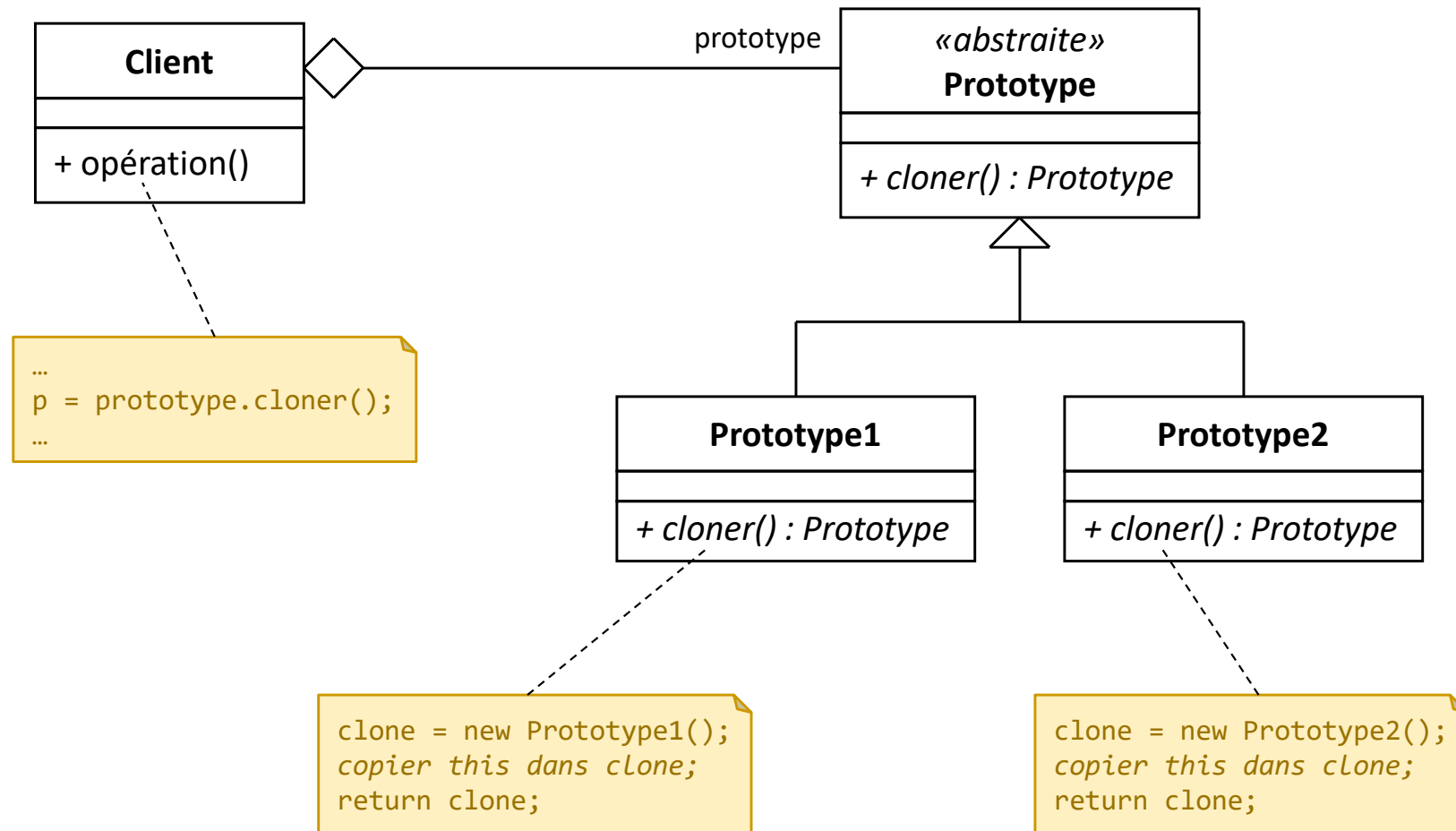
■ Motivation

- ❑ Boîte à outils: déposer des objets par *drag-and-drop*
- ❑ Une copie du modèle est déposée sur la zone de travail

Prototype / Prototype (2/3)



Prototype / Prototype (3/3)



- Concevoir de nouveaux composants par assemblage
 - Pour former des structures plus vastes
 - Avec un comportement plus complexe
- Objectif: exploiter les capacités d'un composant et les adapter à de nouveaux besoins
- Niveau classe
 - Utilisation de l'héritage
 - ⇒ composition d'interfaces ou d'implémentations
- Niveau objet
 - Utilisation de la composition

■ Objectif

- ❑ Adapter l'interface d'une classe à ses besoins
- ❑ Permettre le dialogue entre classes incompatibles

■ Principe

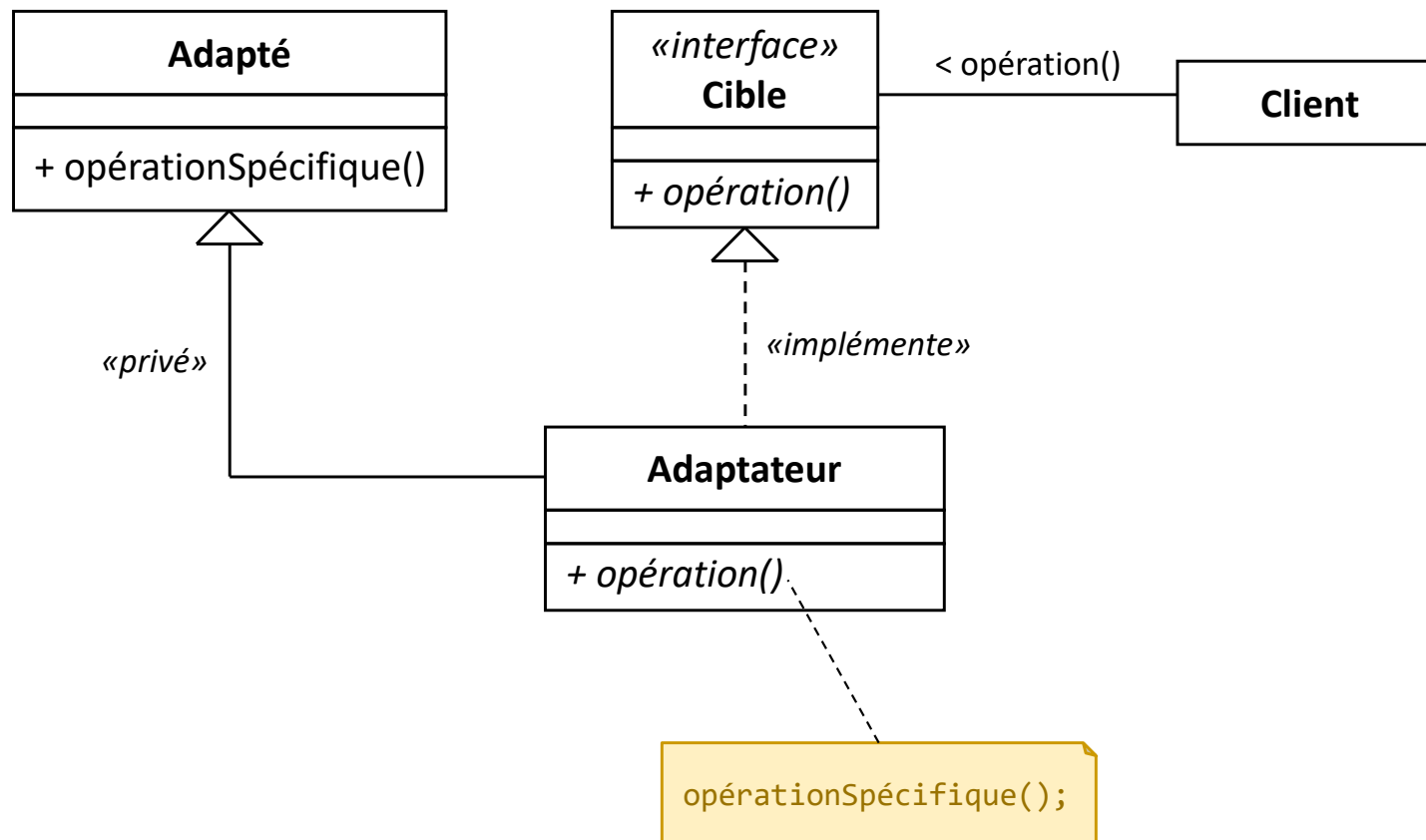
- ❑ Deux approches
- ❑ Classe «adaptateur» ⇒ héritage «multiple»
 - Héritage de la nouvelle interface
 - Héritage de l'implémentation de l'ancienne classe
- ❑ Objet «adaptateur» ⇒ délégation
 - Héritage de la nouvelle interface
 - Agrégation d'un objet de l'ancienne classe

■ Motivation

- ❑ Utiliser une fonctionnalité d'une bibliothèque tierce
- ❑ Mais l'interface n'est pas adaptée

Adaptateur / Adapter (2/3)

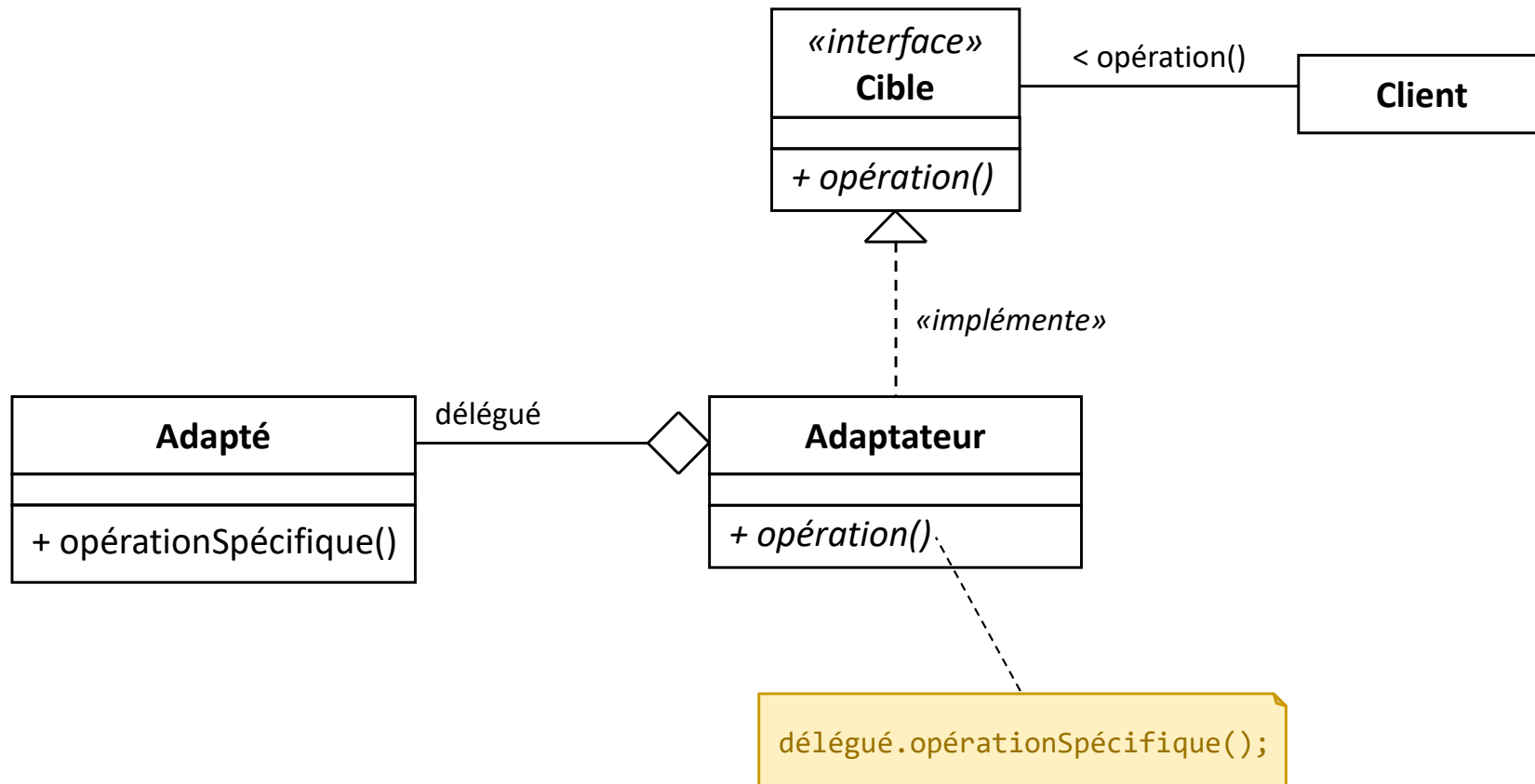
- Classe adaptateur
 - ❑ Héritage de la nouvelle interface
 - ❑ Héritage de l'implémentation de l'ancienne classe
 - ❑ Un seul objet (adapté + adaptateur) est créé



Adaptateur / Adapter (3/3)

■ Objet adaptateur

- ❑ Héritage de la nouvelle interface
- ❑ Agrégation d'un objet de l'ancienne classe + délégation
- ❑ Permet l'adaptation d'une classe et de ses sous-classes



■ Objectif

- ❑ Composer des objets sous forme arborescente
- ❑ Objet individuel ou composition traités de la même manière

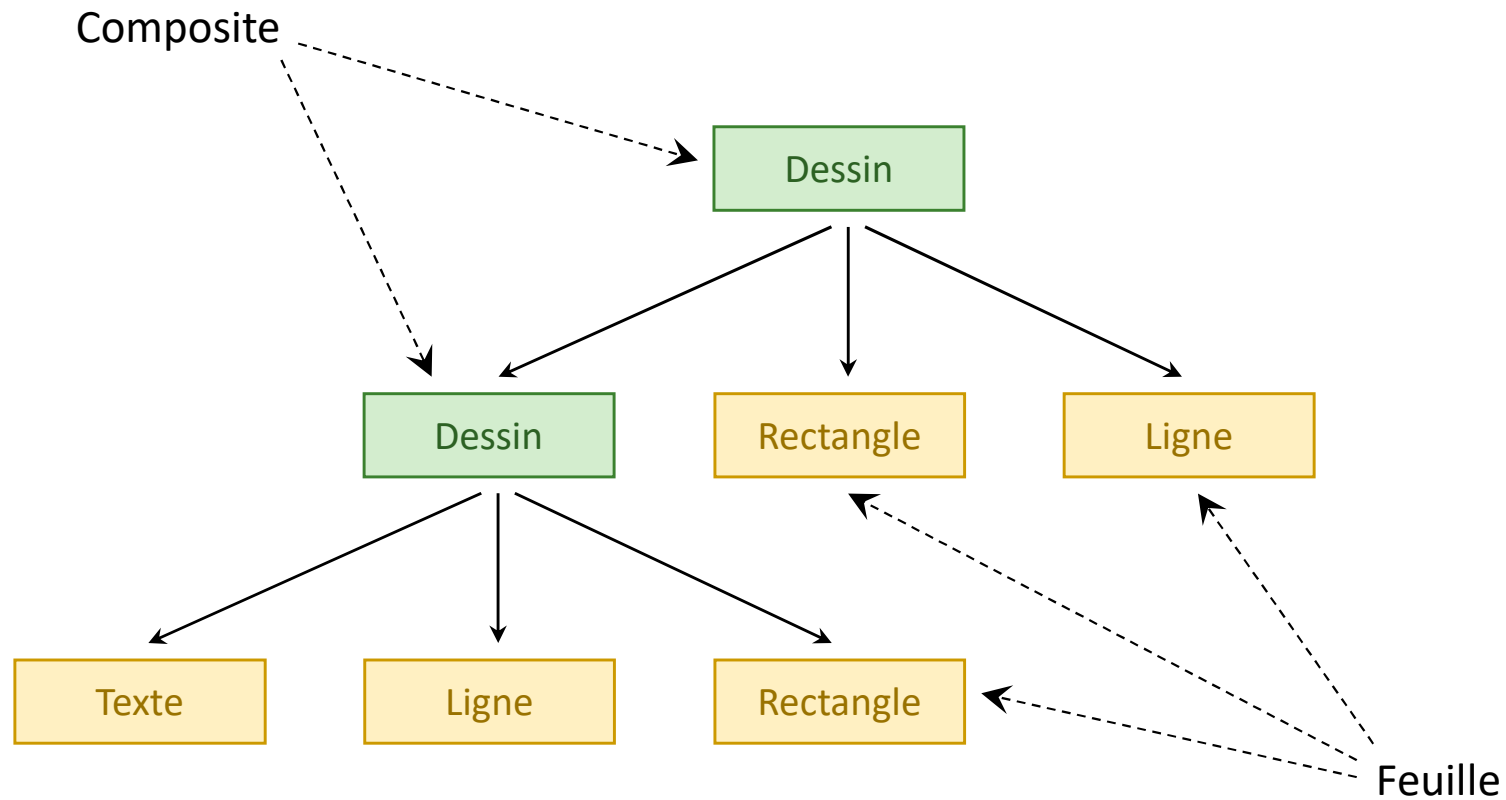
■ Principe

- ❑ Un objet est composé d'autres objets
 - ❑ Ces objets peuvent également être des agrégats d'objets
- ⇒ récursivité dans la composition

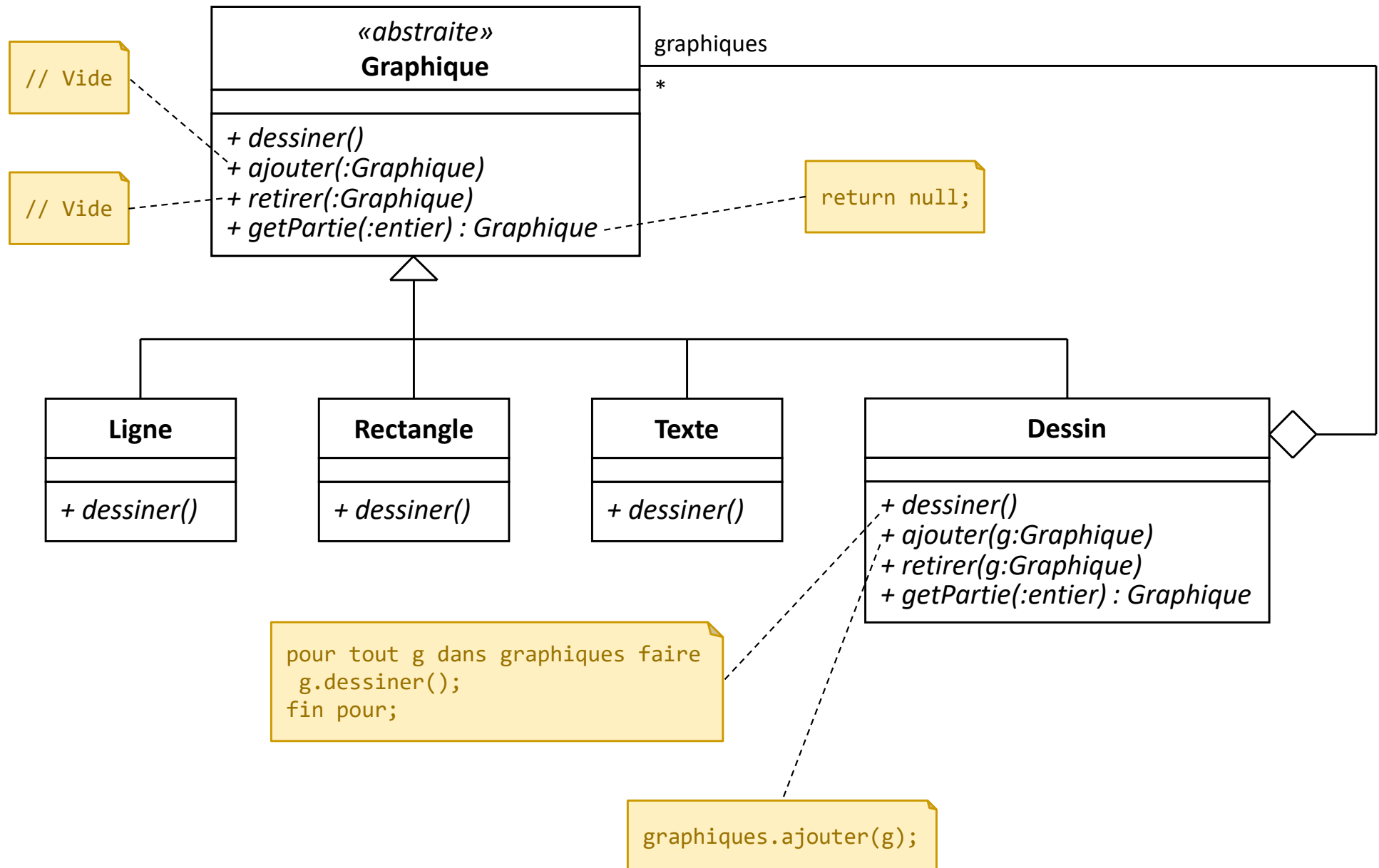
■ Motivation

- ❑ Schéma/dessin composé d'objets graphiques
- ❑ Hiérarchie d'héritage des objets graphiques
- ❑ Un objet graphique peut être un groupement d'objets graphiques

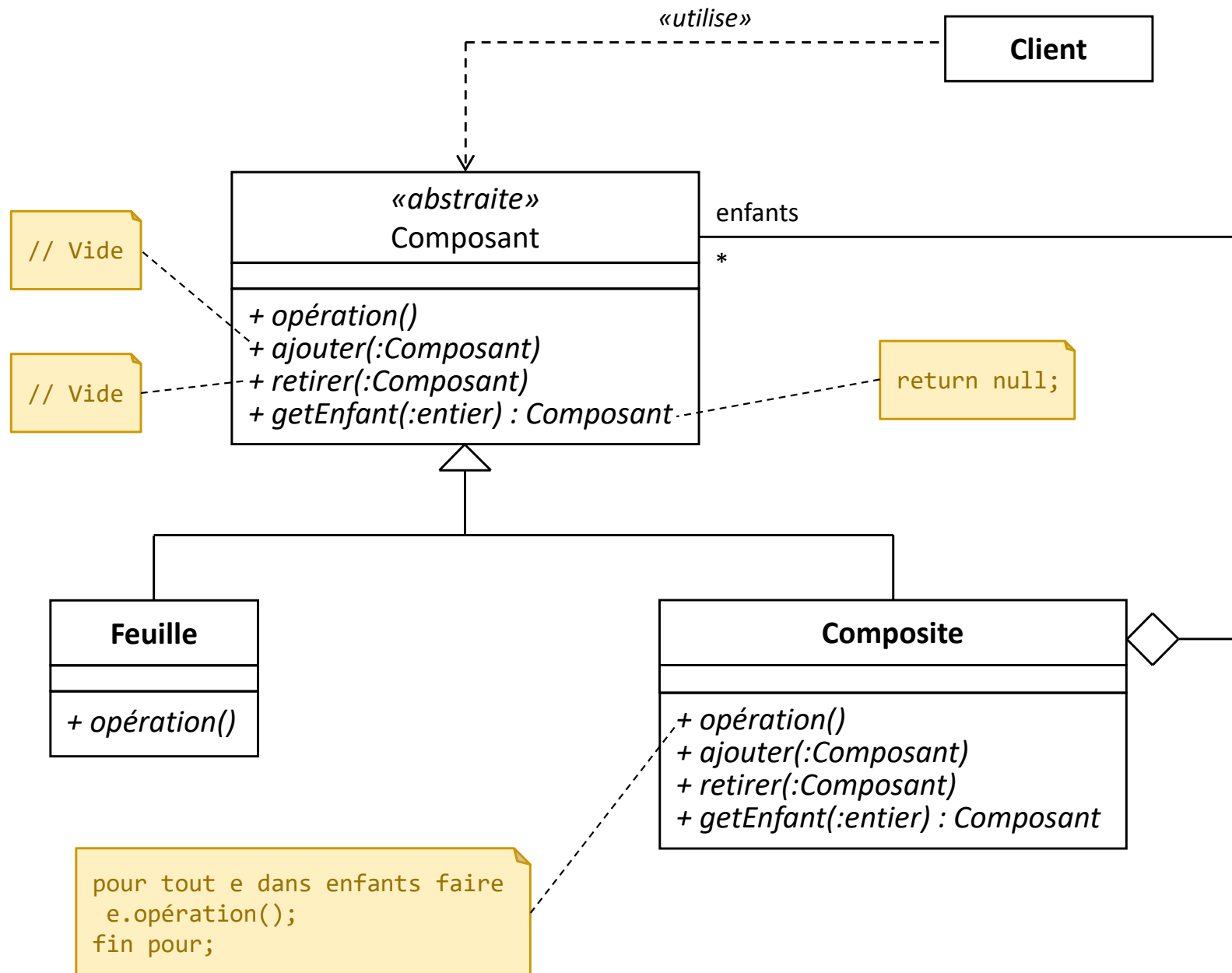
Composite / Composite (2/5)



Composite / Composite (3/5)



Composite / Composite (4/5)



- Le client fait abstraction de la classe réelle des composants
- S'il peut manipuler un objet simple,
il peut manipuler un agrégat
- L'ajout d'un nouveau type de composant est très simple
 - Sans modification, le client pourra le manipuler
 - Sans modification, il pourra être ajouté dans un composite

■ Objectif

- ❑ Ajouter dynamiquement des fonctionnalités à un objet
- ❑ Alternative à l'héritage pour étendre les fonctionnalités

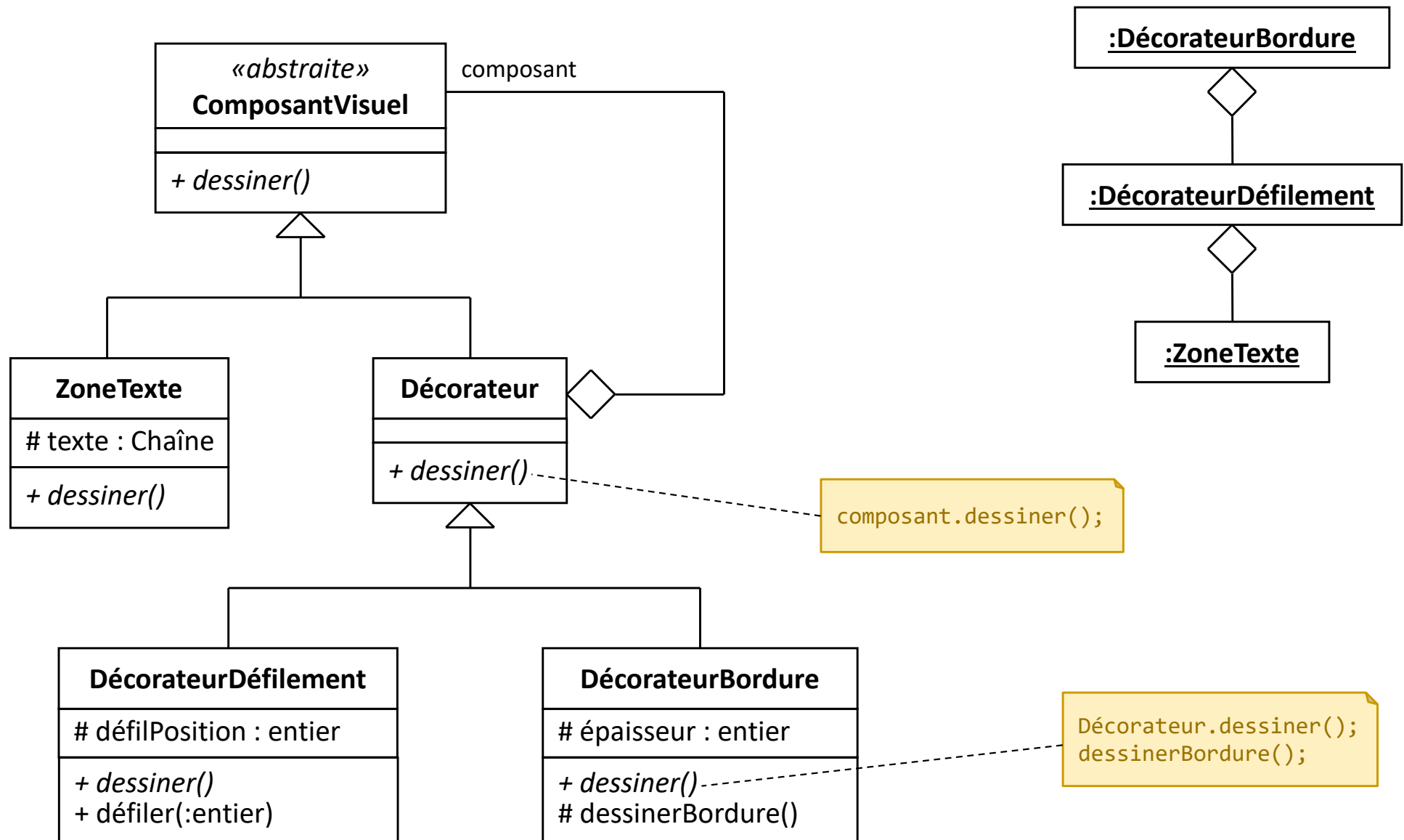
■ Principe

- ❑ Le «décorateur» agrège le composant qu'il adapte
- ❑ Fournit la même interface de base que le composant
- ❑ Il est donc manipulé comme le composant

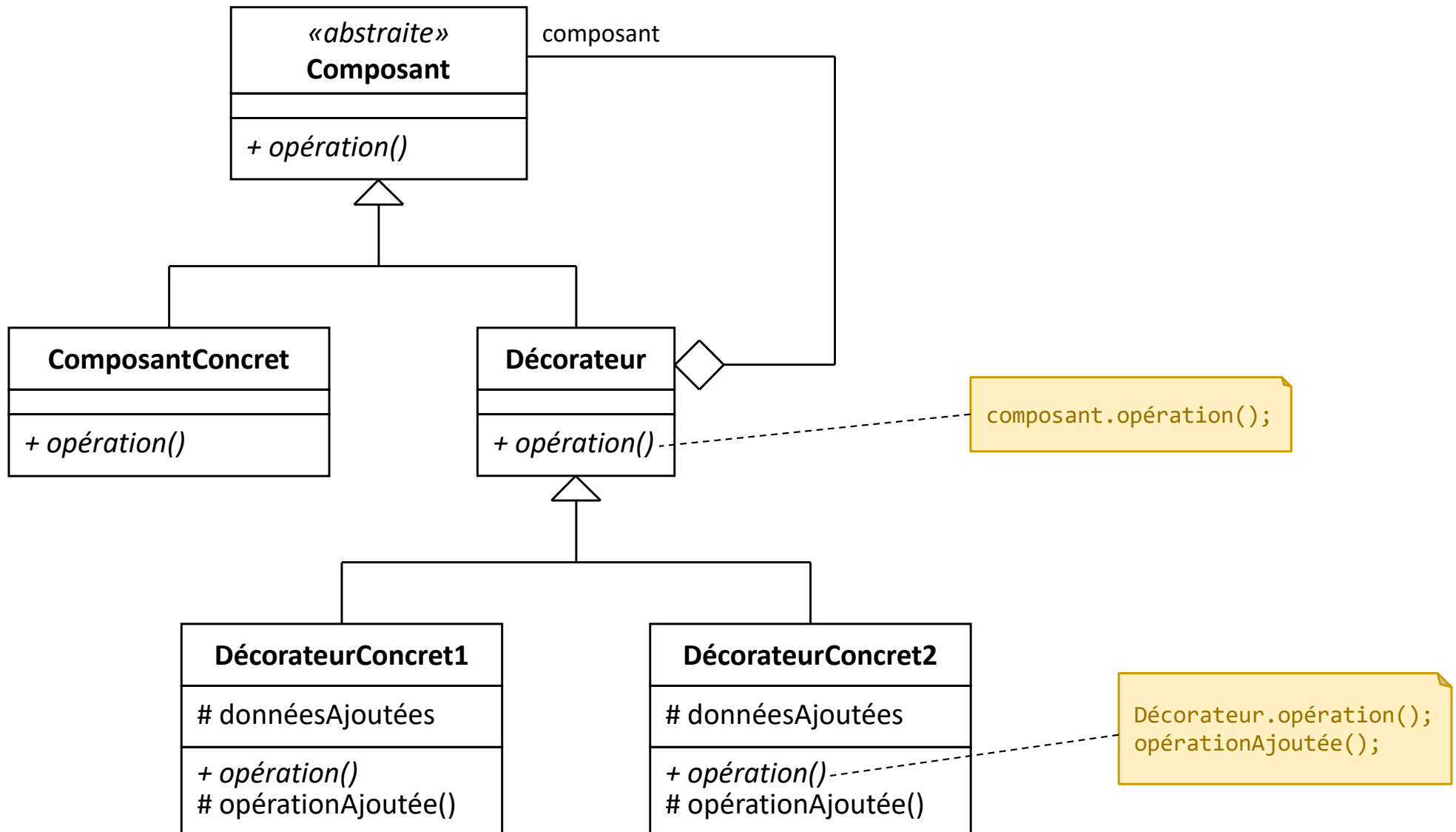
■ Motivation

- ❑ Ajout de fonctionnalités à un composant graphique
- ❑ Eviter l'héritage (car hiérarchie trop complexe)
- ❑ Exemple: zone de texte avec ajout de bordure et barre de défilement

Décorateur / Decorator (2/4)



Décorateur / Decorator (3/4)



- Evite l'extension par héritage
 - ❑ Ajout dynamique de fonctionnalités
 - ❑ Ajout individualisé (un seul objet est touché)

- L'héritage pourrait conduire à une hiérarchie lourde
 - ❑ Exemple de la zone de texte
 - ❑ 3 héritages sont nécessaires (bordure, défilement, les deux)
 - ❑ Extension de la zone de texte $\Rightarrow$ extension des 3 classes

- Mais le décorateur ajoute un objet à chaque décoration

■ Objectif

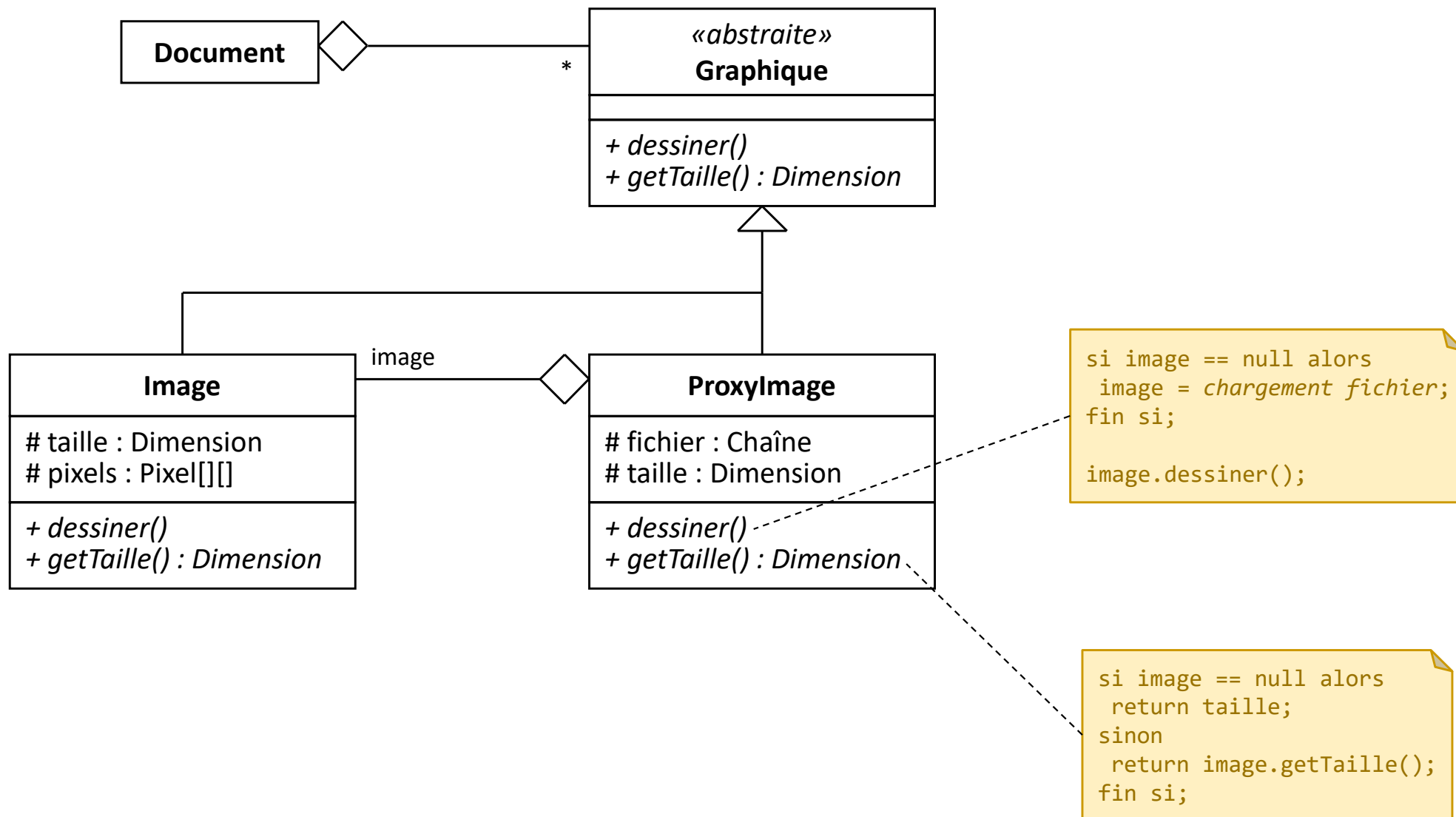
- Fournir un substitut, un intermédiaire, pour accéder à un objet
- Permettre ainsi de contrôler l'accès

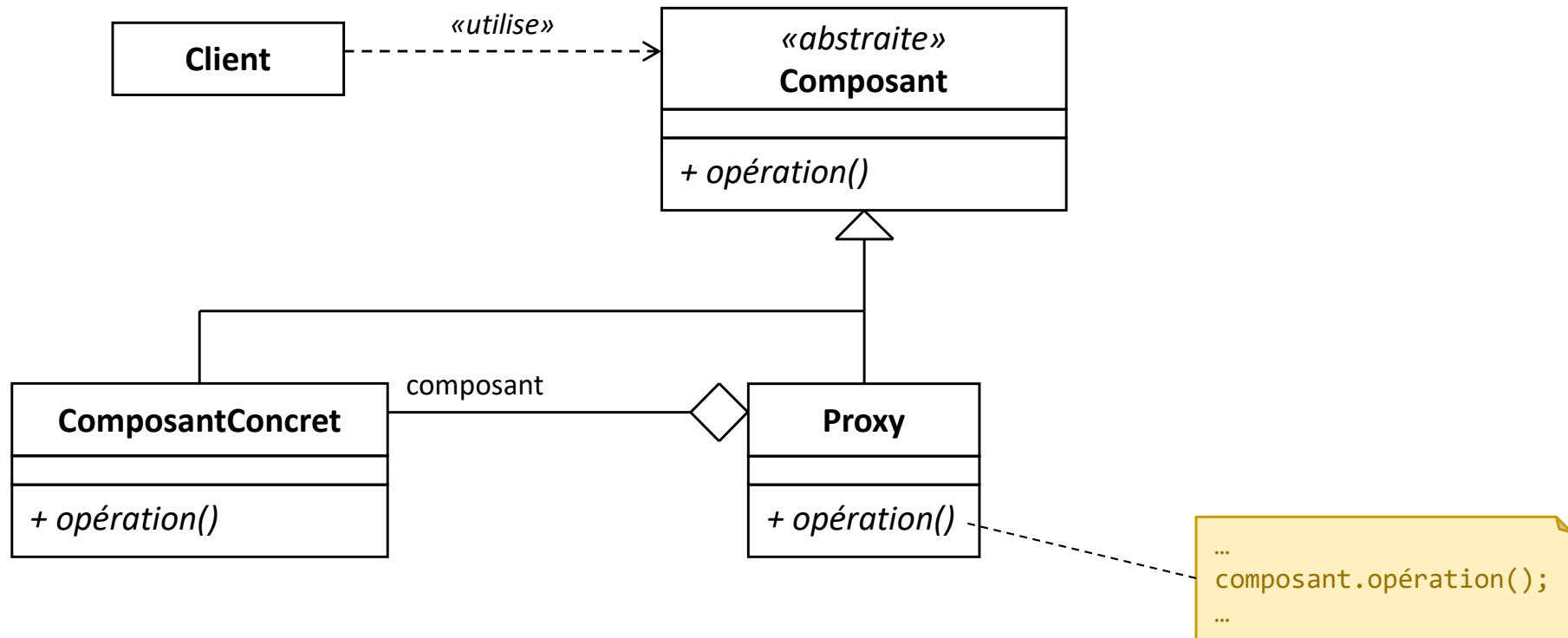
■ Principe

- Le substitut, le «proxy», possède la même interface que l'objet
- Lorsqu'il reçoit un message, il le transmet à l'objet
- Il peut effectuer un contrôle sur le message
 - Refuser de le transmettre
 - Différer la transmission
 - Altérer le message

■ Motivation

- Différer la création d'un objet car elle est coûteuse
- Exemple: chargement d'un document avec des images
 - Différer la lecture des images au moment où celles-ci sont visibles





- ❑ Abstraction de l'accès à un objet
 - Niveau d'indirection supplémentaire

- ❑ Permet une représentation locale d'un objet distant
 - Autre zone mémoire, sur disque ou réseau

- ❑ Permet des optimisations d'exécution des méthodes
 - Technique de cache
 - Création différée («*lazy*»)

- Abstraction du comportement
 - Structure algorithmique
 - Affectation de responsabilités aux objets
 - Communication entre objets

- Niveau classe
 - Utilisation de l'héritage
 - Répartition du comportement

- Niveau objet
 - Utilisation de la composition
 - Coopération d'objets pour effectuer une tâche

- Permet l'assemblage de composants
 - Pour obtenir une fonctionnalité plus élaborée
 - Algorithmes vus comme des objets

- Comment les composants communiquent ?
 - Niveau de connaissance des pairs
 - Références explicites les uns envers les autres
 - Ou perte des références, utilisation d'un intermédiaire
 - Propagation d'un message
 - Délégation
 - Transmission
 - Messages vus comme des objets

■ Objectif

- ❑ Encapsuler une action dans un objet
- ❑ Permet l'abstraction de l'action (découplage déclencheur/receveur)
- ❑ Possibilité de file d'attente, annulation...

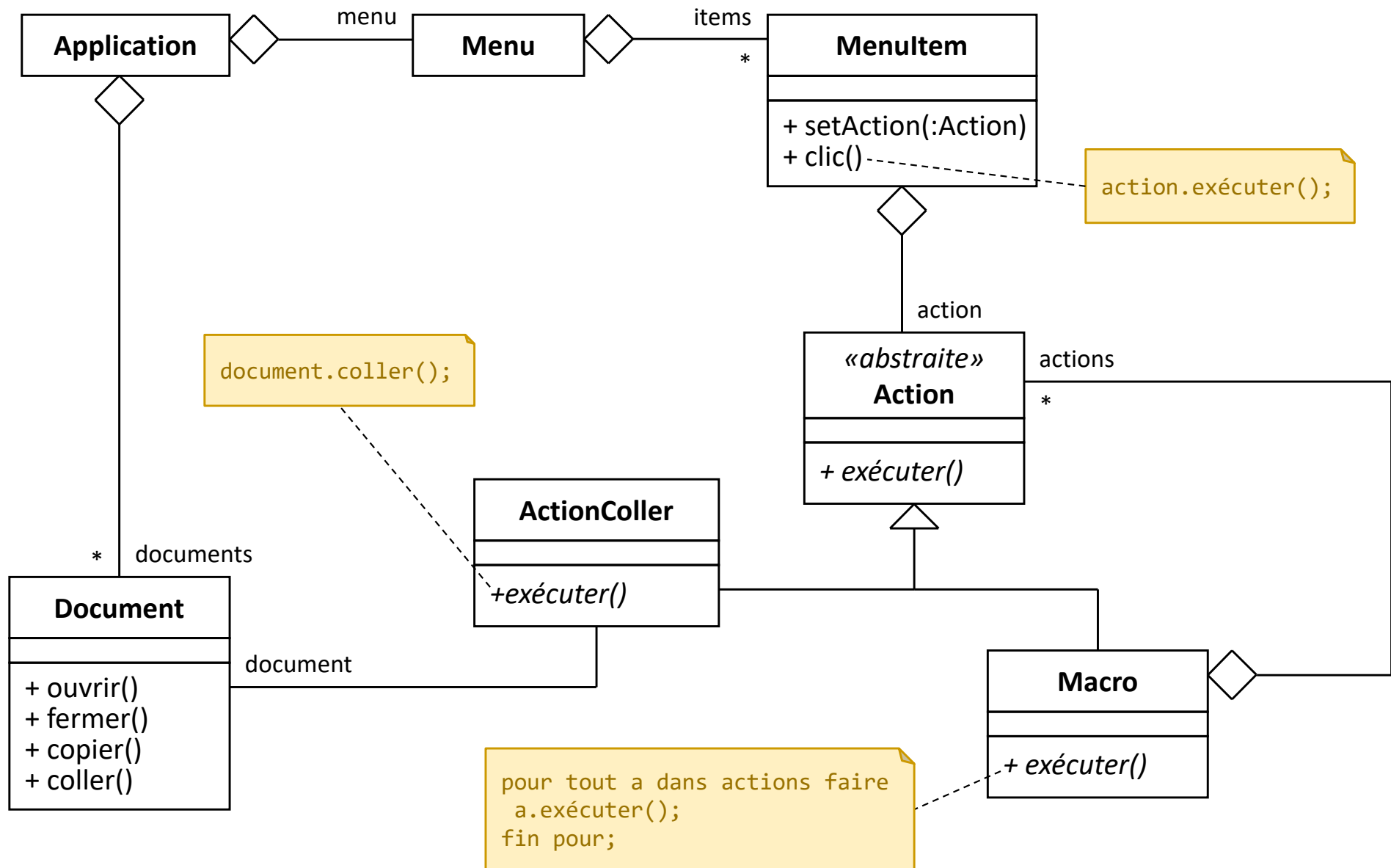
■ Principe

- ❑ Une interface modélise les actions
- ❑ Les objets déclencheurs agrègent une action
 - Déclencheur activé $\Rightarrow$ exécution de l'action
- ❑ L'action connaît toutes les informations pour l'exécution
 - Procédure à exécuter
 - Quels sont les objets concernés

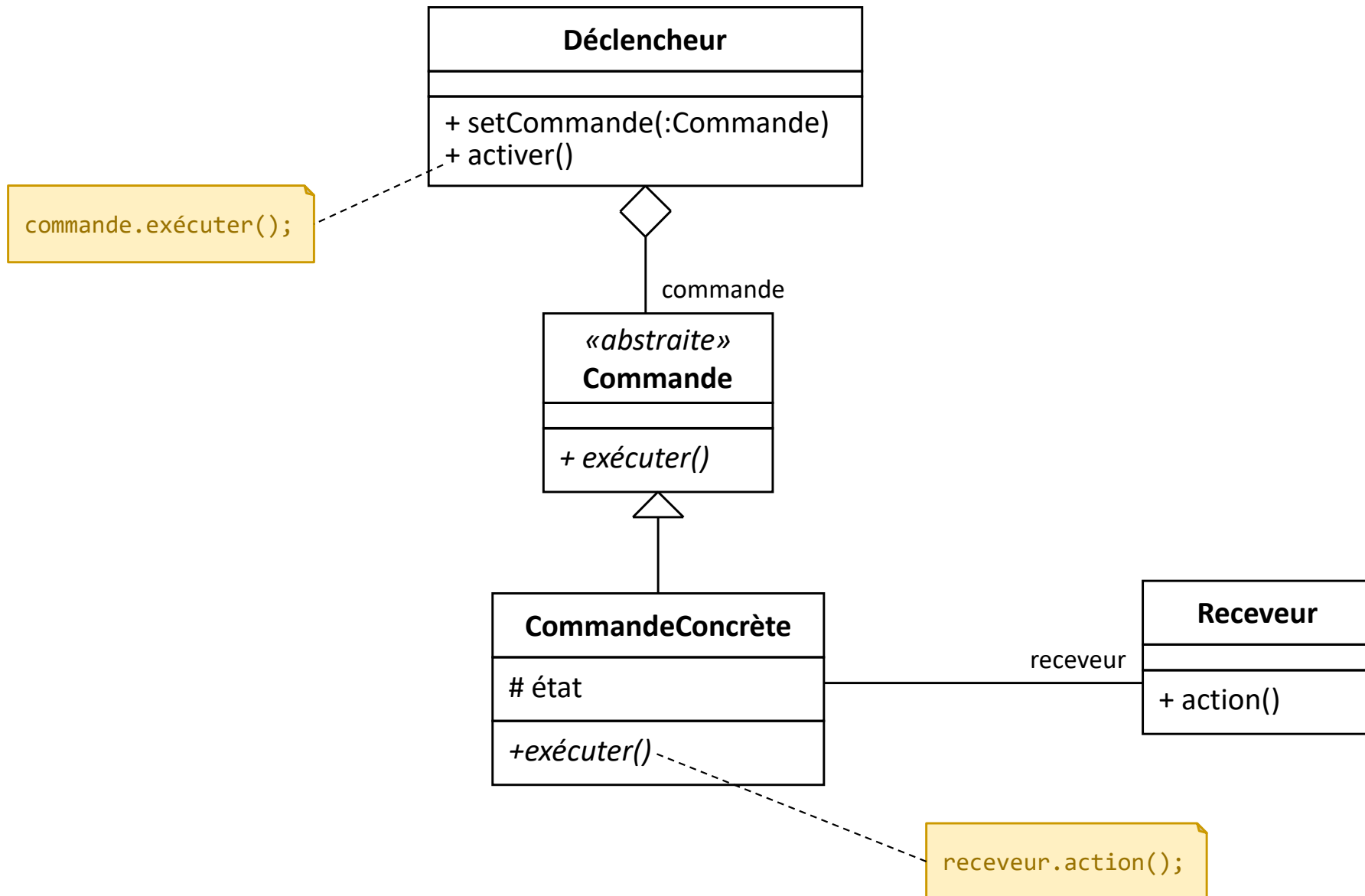
■ Motivation

- ❑ Associer des actions aux boutons d'une interface graphique
- ❑ Les boutons n'ont pas de lien direct avec le code métier

Commande / Command (2/3)



Commande / Command (3/3)



■ Objectif

- ❑ Synchroniser plusieurs objets sur l'état d'un autre objet
- ❑ Quand l'état de l'objet change
 - Les objets dépendants sont informés
 - Ils se mettent à jour

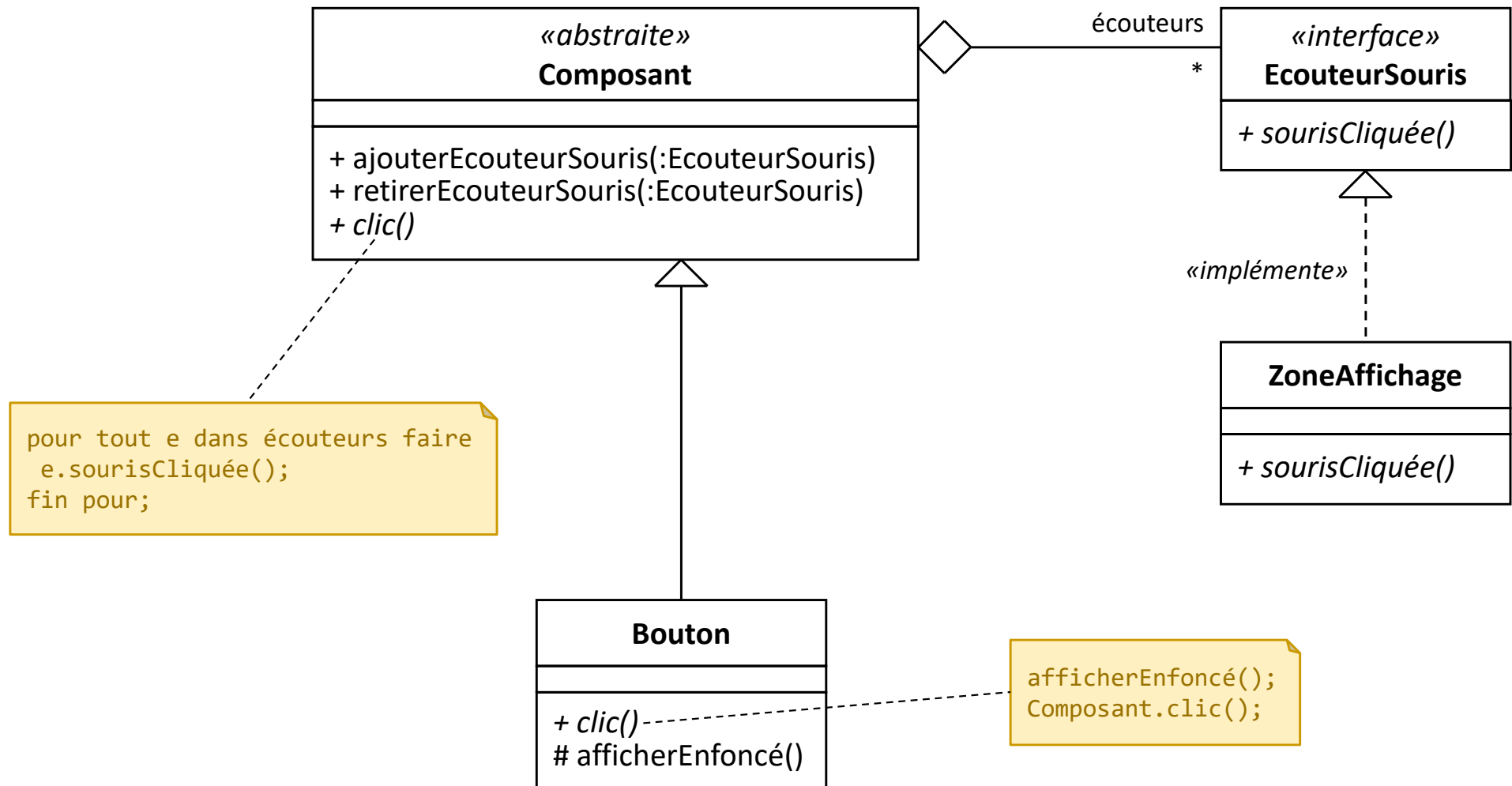
■ Principe

- ❑ Des objets «observateurs» s'enregistrent auprès d'un «sujet»
- ❑ Le sujet maintient donc une liste de ses observateurs
- ❑ Changement de l'état du sujet ⇒ notification aux observateurs
 - Une méthode spécifique des observateurs est invoquée
 - Tous les observateurs doivent donc implémenter la même interface

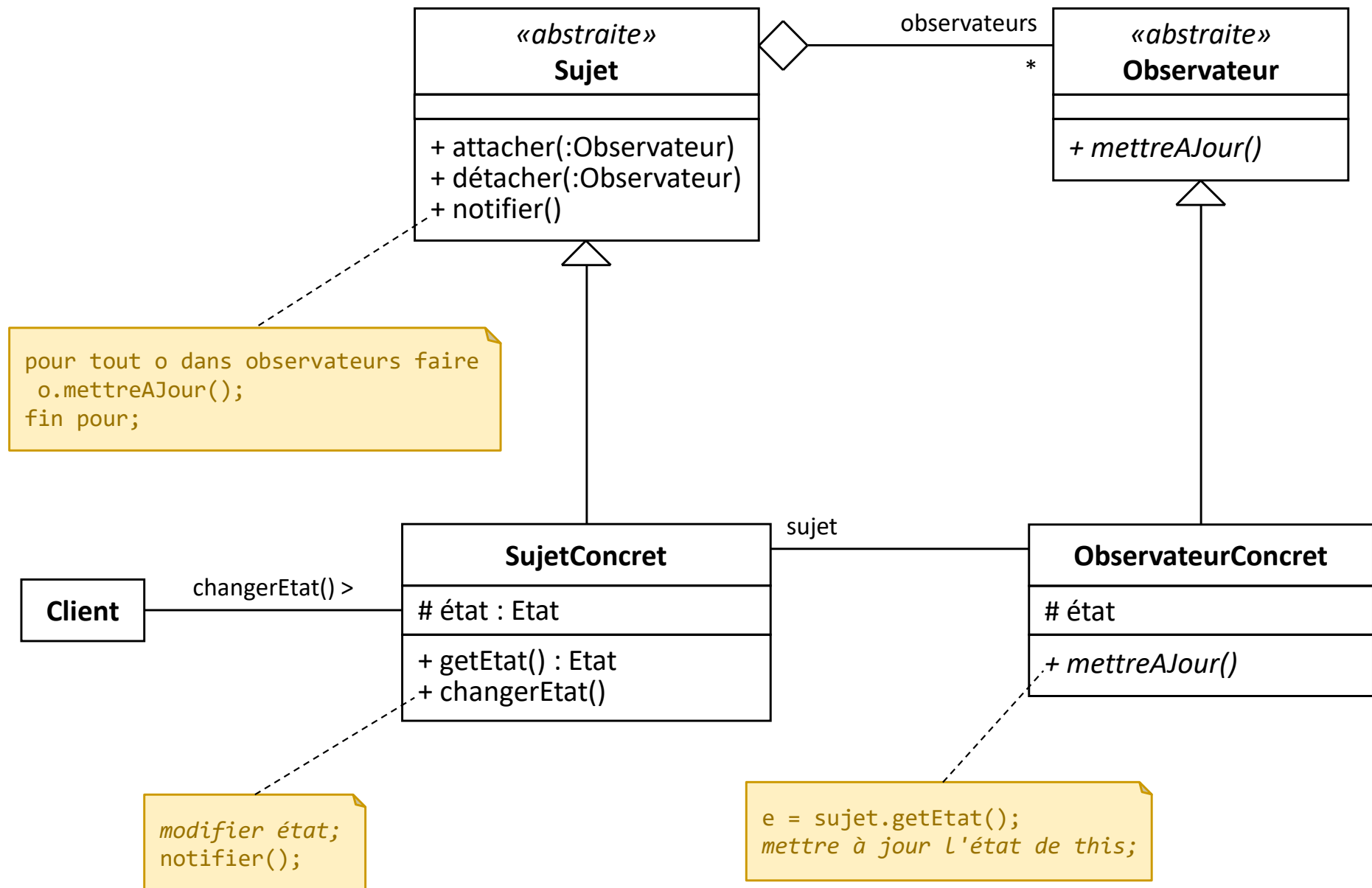
■ Motivation

- ❑ Capturer des événements dans une interface utilisateur

Observateur / Observer (2/4)



Observateur / Observer (3/4)



- Evite un couplage fort entre le sujet et les observateurs
 - Le type concret des observateurs n'est pas connu du sujet
- Les observateurs sont passifs
 - Pas besoin d'interroger le sujet en permanence
 - Informés quand le sujet change d'état
- Mais attention au coût de modification de l'état du sujet
 - En cas de chaînage des observations

■ Objectif

- Rendre les algorithmes d'une même famille interchangeables

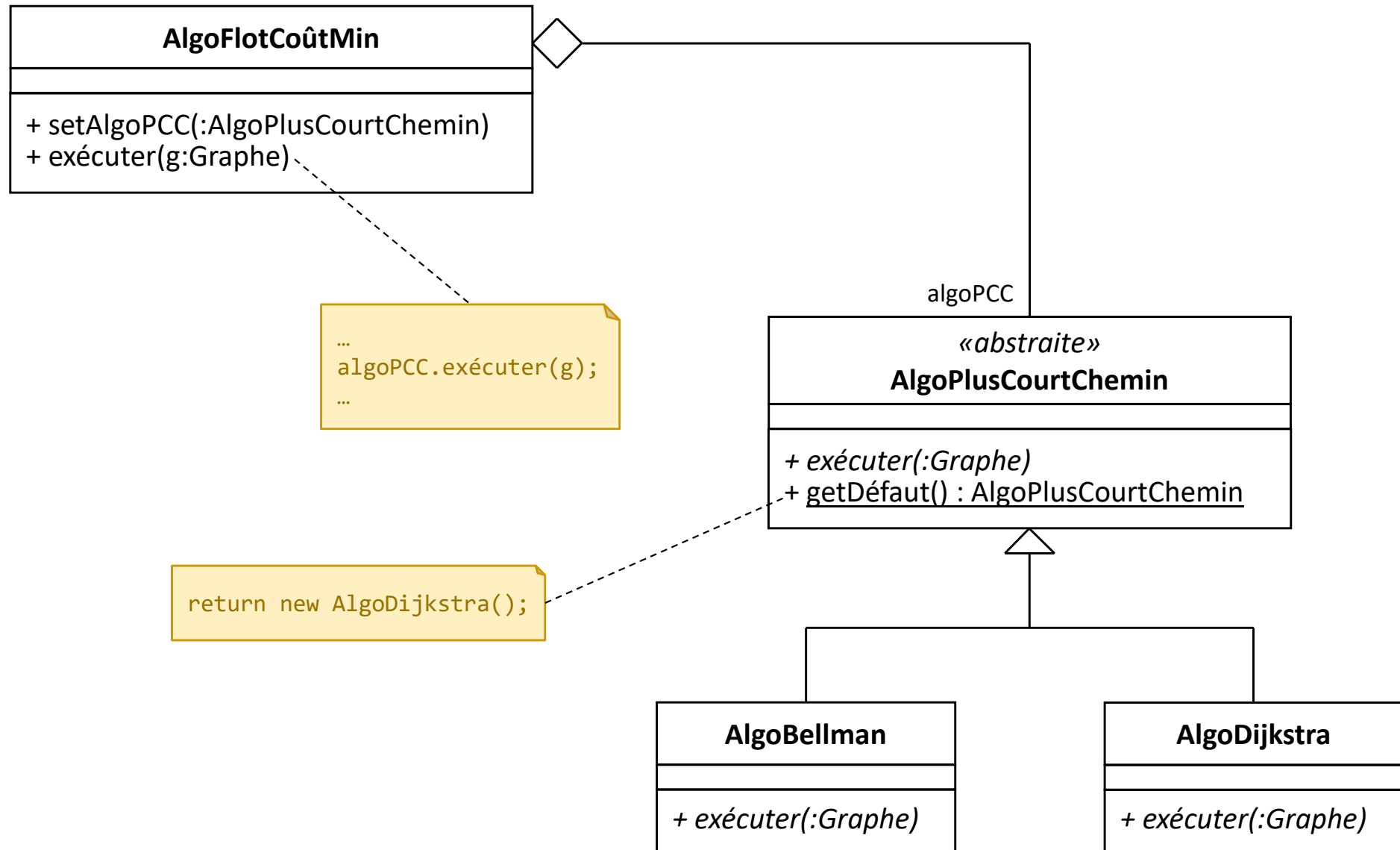
■ Principe

- Les algorithmes («stratégies») sont modélisés par des classes
 - Une méthode représente le point d'entrée
- Une classe abstraite définit une famille d'algorithmes
 - Nouvel algorithme = héritage et redéfinition du point d'entrée
- Un objet «contexte» agrège un algorithme
 - Sans connaître sa classe concrète
 - Le polymorphisme rend les algorithmes interchangeables

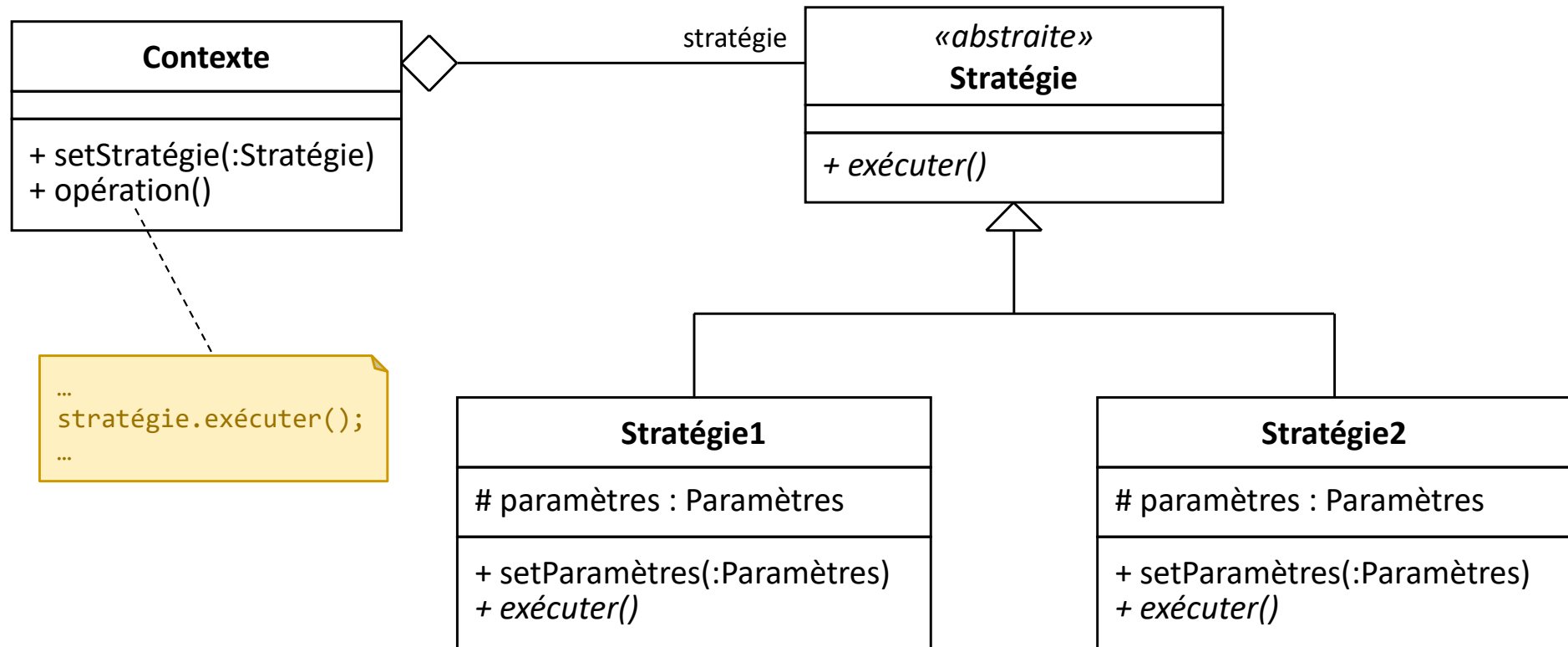
■ Motivation

- Proposer une variété d'algorithmes pour un même objectif
- Possibilité de changer dynamiquement l'algorithme

Stratégie / Strategy (2/4)



Stratégie / Strategy (3/4)



- Abstraction de la stratégie
 - Interchangeable dynamiquement
 - Nouvelle stratégie $\Rightarrow$ aucun impact sur le contexte
- La stratégie est un paramètre du contexte
- Contenu classique d'une classe stratégie/algorithmme
 - Un point d'entrée
 - Méthode publique appelée pour exécuter l'algorithme
 - Des sous-algorithmes
 - Méthodes protégées ou privées utilisées par le point d'entrée
 - Des paramètres
 - Mémorisés dans des attributs
 - Constructeurs et accesseurs nécessaires pour l'initialisation

Méthode patron / *Template Method* (1/5)

■ Objectif

- ❑ Spécialiser un algorithme sans changer sa structure générale

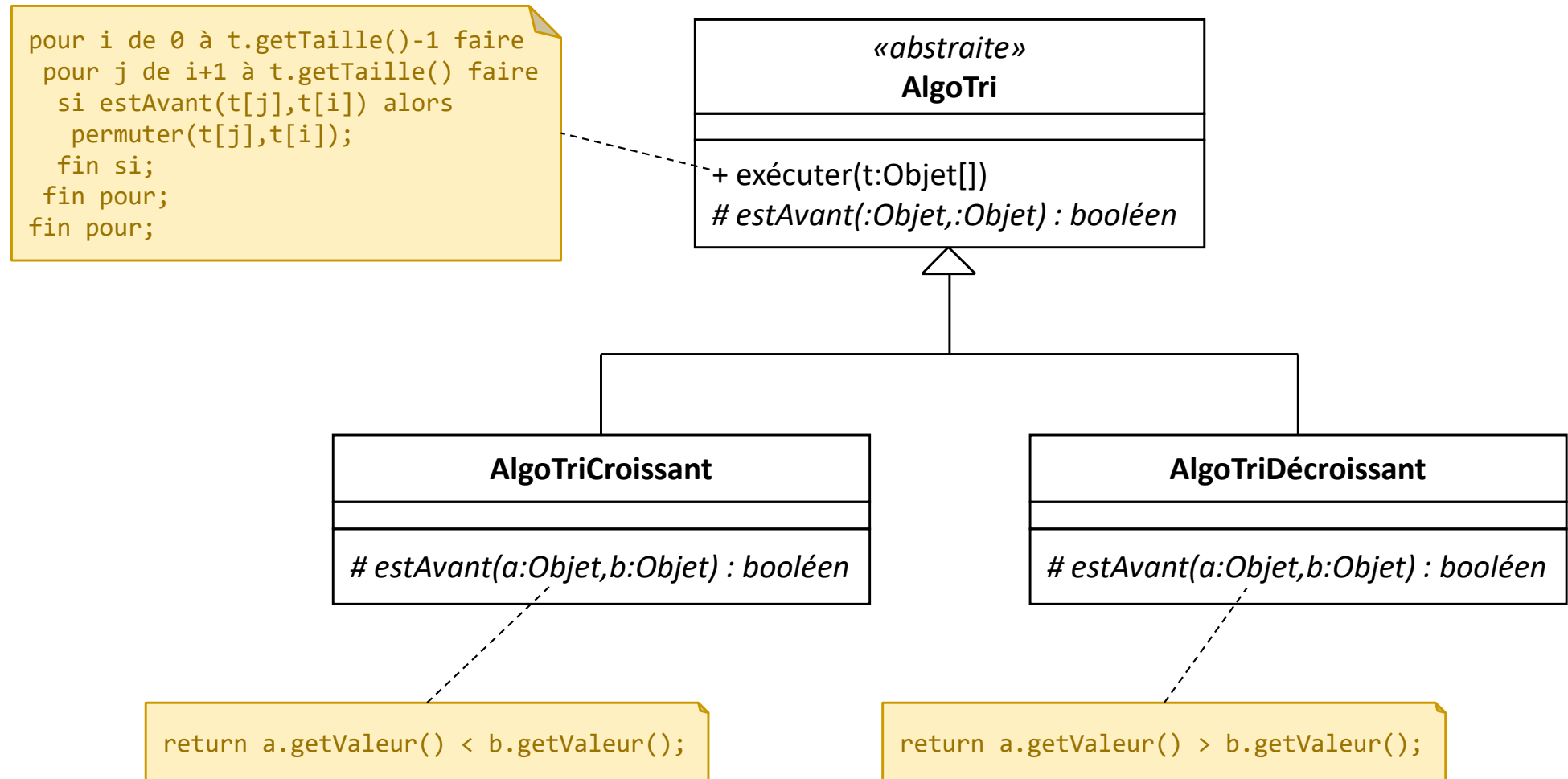
■ Principe

- ❑ Définir la structure générale d'un algorithme dans une classe
 - De la même manière que la stratégie
- ❑ Délocaliser des parties dans des méthodes virtuelles
 - Par héritage, ces parties pourront être redéfinies

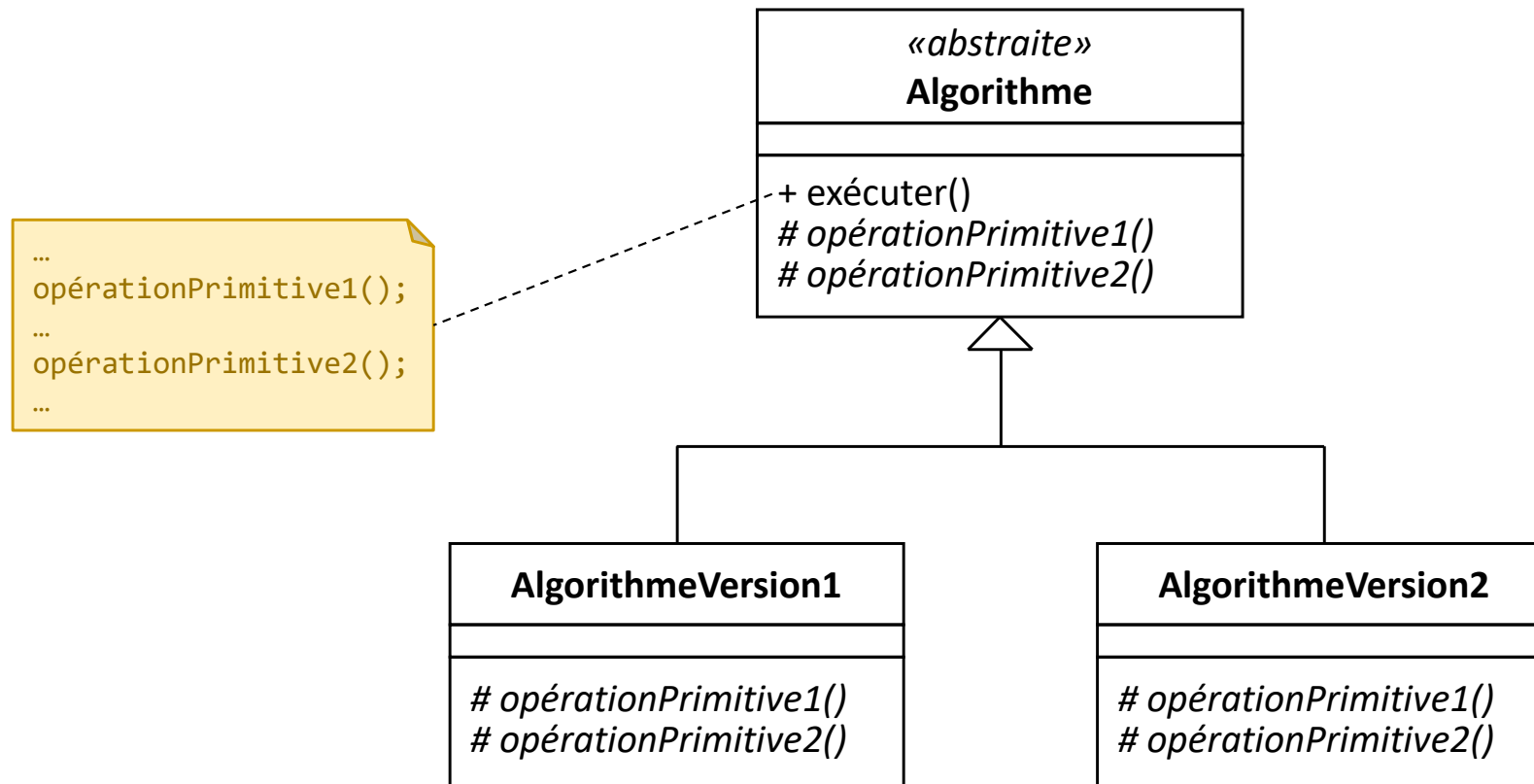
■ Motivation

- ❑ Proposer plusieurs variantes d'un algorithme
- ❑ Où la structure générale de l'algorithme est inchangée

Méthode patron / Template Method (2/5)



Méthode patron / *Template Method* (3/5)



Méthode patron / *Template Method* (4/5)

- Abstraction de parties d'un algorithme
 - ❑ Conserve la structure générale de l'algorithme
- Rôle très important dans la réutilisabilité
 - ❑ Evite un détournement du rôle d'une classe
 - ❑ Guide / facilite la spécialisation de la classe
- Mais éviter trop d'opérations primitives
 - ❑ Appelées trop souvent ⇒ surcoût lié à la virtualité
 - ❑ Trop de méthodes ⇒ redéfinition fastidieuse pour l'utilisateur
- Utilisé pour la redéfinition «par complément»
 - ❑ Objectif: redéfinir pour compléter une méthode
 - ❑ Problème: il ne faut pas oublier d'appeler la version mère
 - ❑ Solution: utiliser une méthode patron

Méthode patron / *Template Method* (5/5)

■ Redéfinition par complément

□ Approche classique

```
class Mere {  
    public: virtual void m() { /* Quelque chose */ }  
};  
  
class Fille : public Mere {  
    public: void m() override { Mere::m(); /* Autre chose */ }  
};
```

□ Approche avec méthode patron

```
class Mere {  
    protected: virtual void autreChose() {}  
    public: void m() { /* Quelque chose */ autreChose(); }  
};  
  
class Fille : public Mere {  
    protected: void autreChose() override { /* Autre chose */ }  
};
```


■ Objectif

- ❑ Représenter une opération à appliquer sur un ensemble d'éléments
- ❑ Définir une nouvelle opération sans modifier la classe des éléments

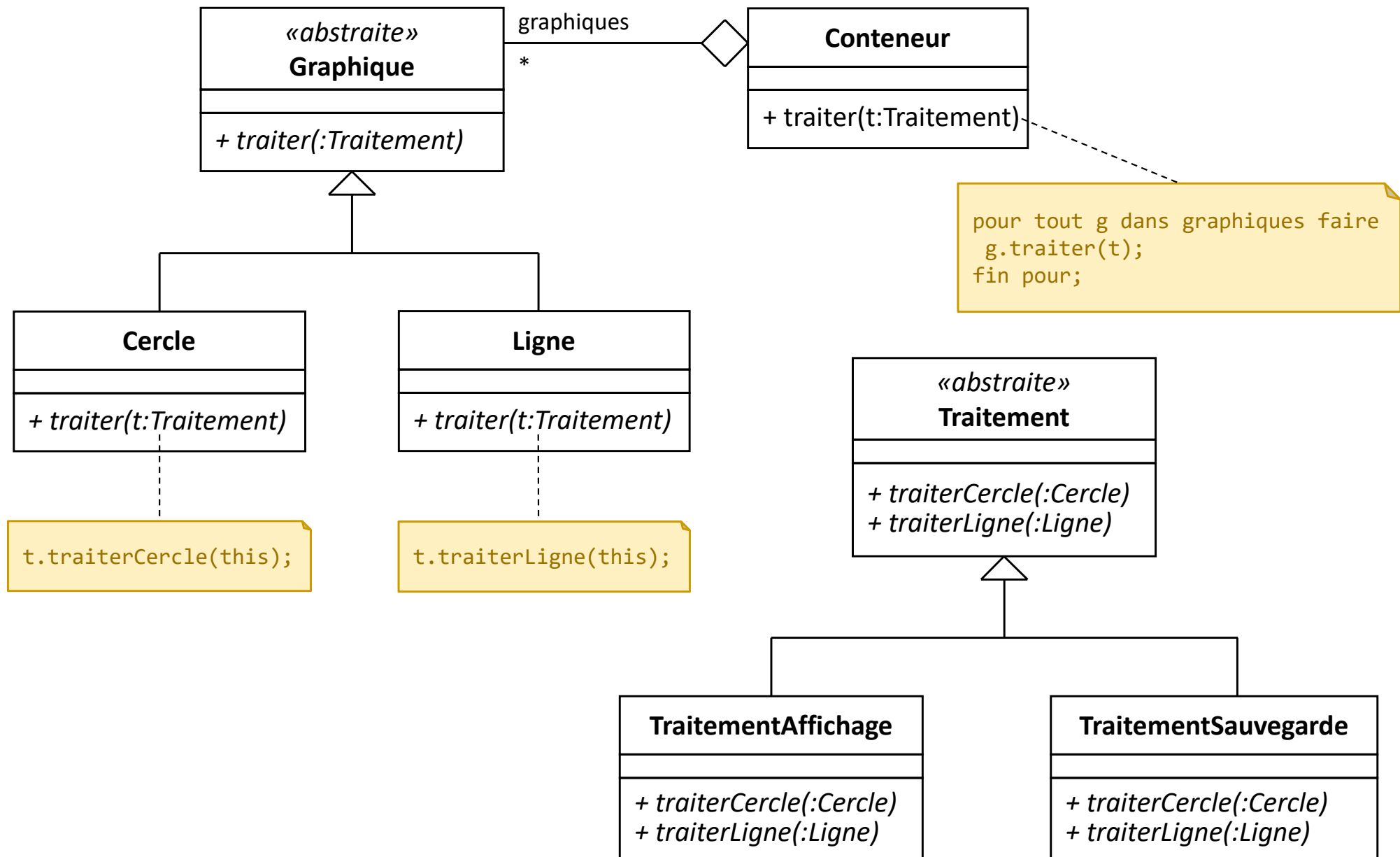
■ Principe

- ❑ L'opération est modélisée par un objet, le «visiteur»
 - Une classe, extensible, représente l'opération
- ❑ Les éléments doivent «accepter» un visiteur
 - Une méthode doit recevoir le visiteur
 - Et appliquer l'opération associée sur l'élément
- ❑ Une procédure de parcours applique l'opération aux éléments
 - Il reçoit le visiteur
 - Et le transmet à chacun des éléments

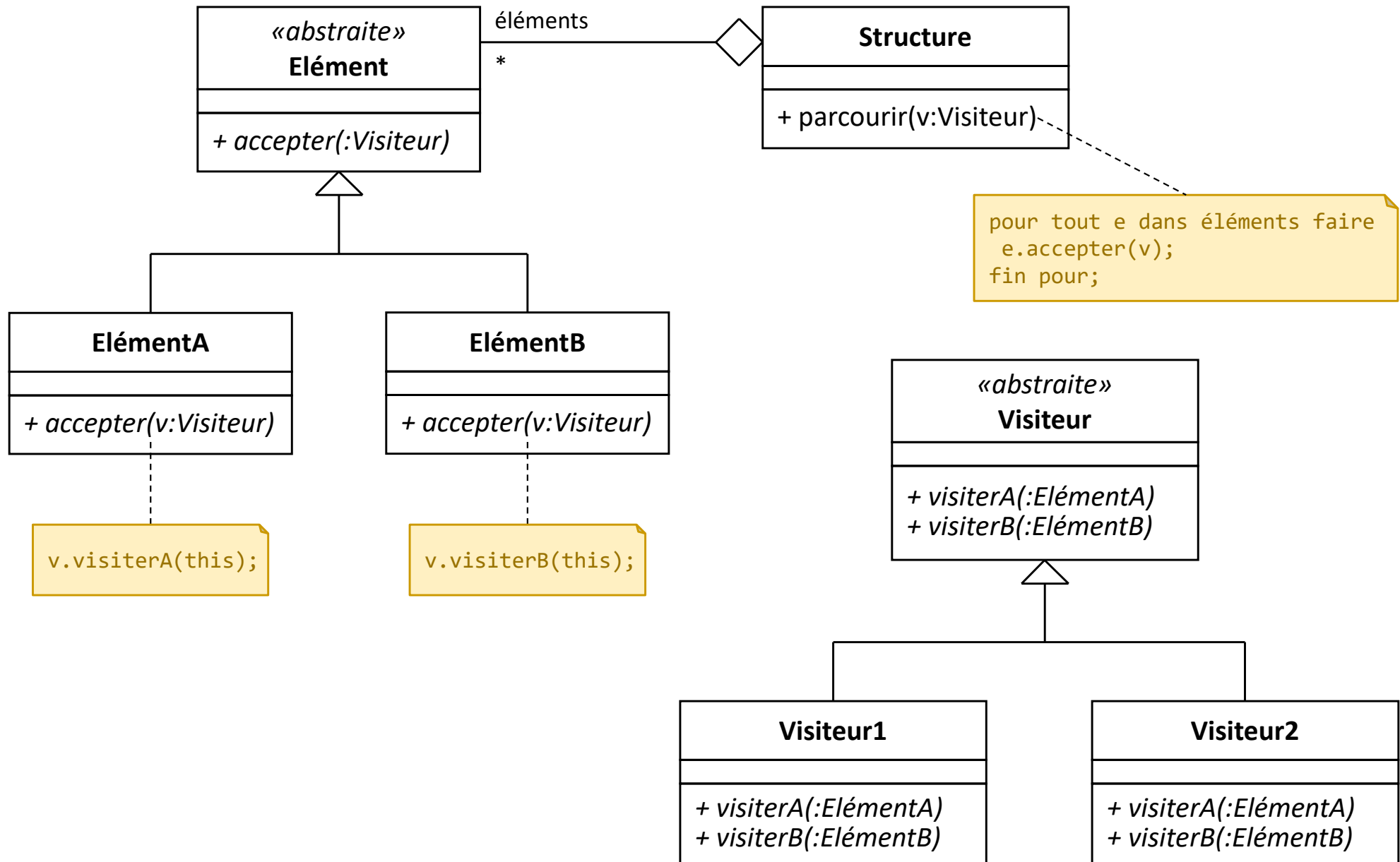
■ Motivation

- ❑ Appliquer des opérations différentes sur un ensemble d'objets
- ❑ Mais le processus de parcours est toujours le même

Visiteur / Visitor (2/4)



Visiteur / Visitor (3/4)



- ❑ Propose plusieurs traitements sur les éléments
 - Sans alourdir l'interface de la structure
 - Sans alourdir l'interface des éléments

- ❑ Facilite l'ajout d'un nouveau traitement
 - Il suffit de créer un nouveau visiteur
 - Et définir le traitement pour chaque type d'éléments

- ❑ Mais plus laborieux d'ajouter un nouveau type d'éléments
 - Il faut ajouter une méthode dans chaque visiteur
 - Pour définir chaque traitement pour le nouveau type d'éléments

PARTIE XI

Programmation parallèle

Bruno Bachelet

Loïc Yon

- Développement des processeurs multicoeurs
 - ❑ Limitation technique à l'augmentation des fréquences d'horloge
 - ❑ Nouvelle voie d'amélioration des performances d'une application

- Applications avec interface graphique nécessairement parallèles
 - ❑ 1 *thread* → interface graphique (gestion des événements)
 - ❑ 1 ou plusieurs *threads* → application métier
 - ❑ Evite le gel de l'interface graphique

- Beaucoup d'applications peuvent bénéficier du parallélisme
 - ❑ Possibilité d'exécution de tâches en parallèle
 - ❑ Réduction des temps de latence d'un programme
 - Accès à une ressource système (mémoire, fichier...) ⇒ latence
 - Mécanisme de mise en attente d'un *thread* ⇒ gain même sur un seul coeur
 - Règle empirique: efficacité «optimale» pour nombre de *threads* = 2 × nombre de coeurs

- La conception d'une application parallèle est difficile
 - Choisir la granularité des tâches à paralléliser
 - Synchroniser les tâches
 - Contrôler l'accès aux ressources partagées

- Avant C++11: bibliothèques/extensions non standards
 - POSIX Threads, OpenMP...

- Depuis C++11: API objet standard
 - Couche bas niveau (équivalent POSIX): `thread`, `mutex`...
 - Couche intermédiaire (abstraction): `async`, `future`...

- Avec C++17
 - Couche haut niveau: politiques d'exécution pour les algorithmes

- Représente un nouveau fil d'exécution dans le programme
- Nouveau contexte d'exécution
 - ❑ Possède sa propre pile
 - ❑ Possibilité de variables «globales» non partagées (cf. `thread_local`)
- Mais partage des données avec d'autres
 - ❑ Accès (lecture/écriture) au tas du programme
 - ❑ Attention aux accès concurrents
- A ne pas confondre avec un processus
 - ❑ Granularité plus fine
 - Intra-programme vs. inter-programme
 - ❑ Plus léger
 - Moins de consommation de ressources
 - Processeurs adaptés aux *threads*

- Entête: `#include <thread>`
- *Threads* représentés par la classe «`thread`»
 - Deux états sont possibles
 - Actif: représente une exécution parallèle effectivement en cours
 - Inactif: symbolise un *thread*, mais aucune exécution parallèle effective
 - Pour savoir si le *thread* est actif: méthode «`joinable`»
 - Possède un identifiant unique: méthode «`get_id`»
 - Peut être utilisé comme clé dans un conteneur associatif par exemple
- Un *thread* démarre lors de sa construction
 - Si on lui fournit un «*callable*» (i.e. fonction, foncteur ou lambda)
 - Ce *callable* est automatiquement exécuté au lancement du *thread*
 - Des arguments peuvent être fournis
 - Exemple: `t = std::thread(ma_fonction,param1,param2);`
⇒ exécution en parallèle de `ma_fonction(param1,param2)`
 - Sinon le *thread* est inactif

- Possibilité de transférer le contrôle d'un *thread* actif
 - Exemple: `t1 = t2;`
 - Opération de mouvement (dépouillement de «`t2`»)
 - Si «`t2`» est actif, alors «`t1`» devient actif et «`t2`» inactif
 - Intérêt: permet de séparer les phases de déclaration et de lancement

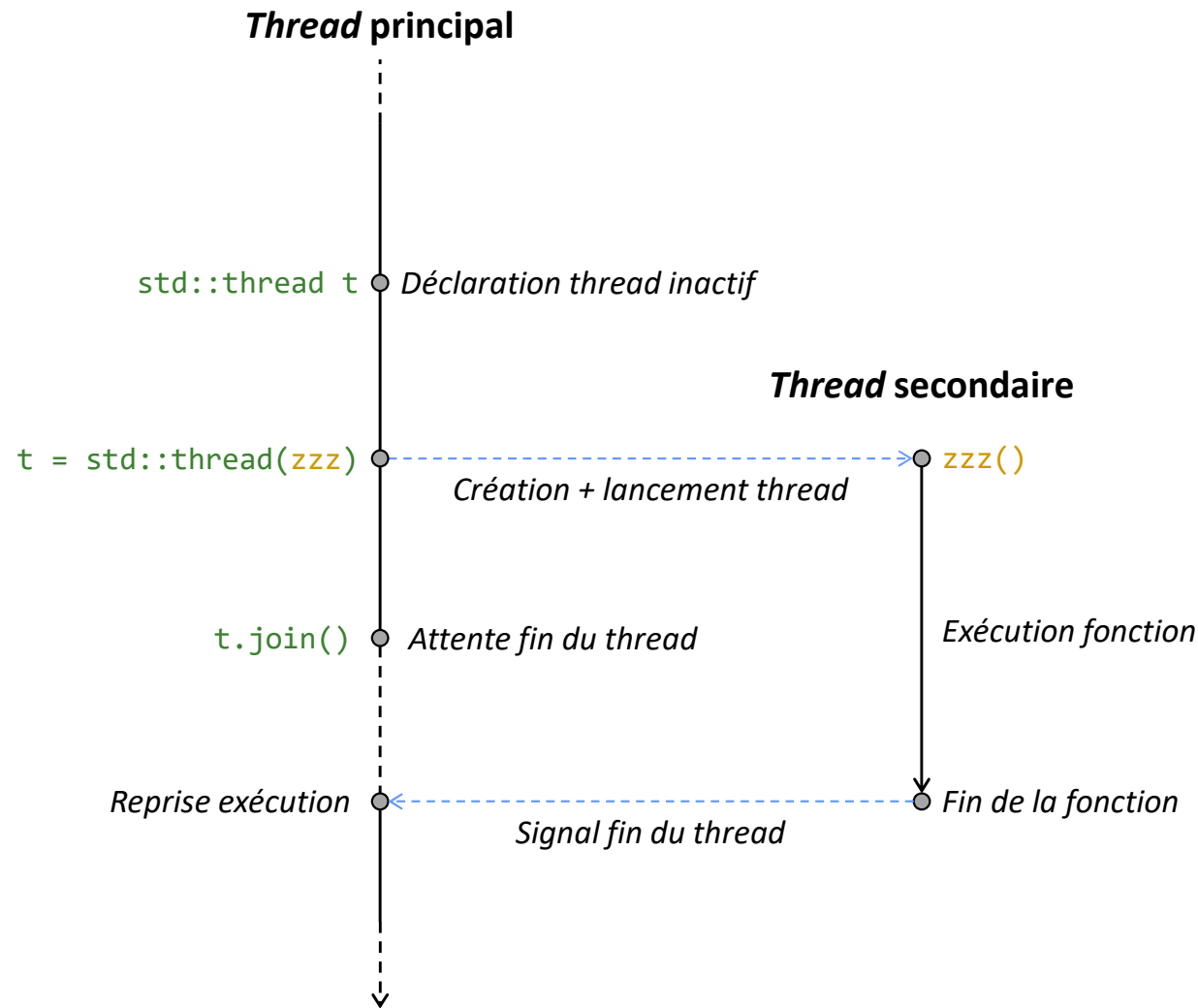
```
std::thread t;  
...  
t = std::thread(ma_fonction,param1,param2);
```
- L'exécution d'un *thread* se termine à la sortie de la fonction associée
- Le *thread* principal n'attend pas automatiquement la fin des *threads* qu'il a lancés
 - Attendre chaque *thread* à l'aide de sa méthode «`join`»
 - Sinon, erreur de conception ⇒ erreurs très probables à l'exécution

Lancement d'un *thread* (1/2)

■ Exemple

```
void zzz() {  
    std::cout << "zzz..." << std::endl;  
    std::this_thread::sleep_for(std::chrono::seconds(1));  
}  
  
int main() {  
    std::thread t; // Déclaration d'un thread inactif  
  
    t = std::thread(zzz); // Exécution parallèle de la fonction  
                        // Objet temporaire = thread actif  
                        // Mouvement dans «t»  
  
    t.join(); // Attente de la fin du thread  
}
```

Lancement d'un *thread* (2/2)



Lancement de plusieurs *threads*

■ Exemple

```
void zzz(unsigned n) {  
    std::cout << "[" << n << "]" zzz..." << std::endl;  
    std::this_thread::sleep_for(std::chrono::seconds(1));  
}  
  
int main() {  
    std::thread t[4];  
  
    for (unsigned i = 0; i<4; ++i) t[i] = std::thread(zzz,i);  
    for (unsigned i = 0; i<4; ++i) t[i].join();  
}
```

Exécution parallèle d'une lambda

■ Lambda sans capture

```
t[i] = std::thread(  
    [] (unsigned n) {  
        std::cout << "[" << n << "]" zzz..." << std::endl;  
        std::this_thread::sleep_for(std::chrono::seconds(1));  
    }  
    ,i);
```

■ Lambda avec capture

```
t[i] = std::thread(  
    [=] () {  
        std::cout << "[" << i << "]" zzz..." << std::endl;  
        std::this_thread::sleep_for(std::chrono::seconds(1));  
    }  
    );
```

Accès à une ressource partagée

- Exemple précédent, sortie prévue (ordre non garanti)

[0] zzz...

[1] zzz...

[2] zzz...

[3] zzz...

- Mais sortie possible

[[01] zzz...] zzz...

[2] zzz...

[3] zzz...

- Car problème de partage de ressource
 - ❑ La sortie standard est partagée par tous les *threads*
 - ❑ Synchronisation nécessaire: chacun son tour $\Rightarrow$ *mutex*

- Mutual exclusion
- Objectif: empêcher l'exécution simultanée d'une portion de code par plusieurs *threads*
- Mécanisme de jeton
 - ❑ *Mutex* partagé par plusieurs *threads*
- Acquisition d'un *mutex* (méthode «**lock**»)
 - ❑ Si le *mutex* n'est pas verrouillé, le *thread* obtient l'accès
 - La méthode «**lock**» se termine
 - ❑ Si le *mutex* est verrouillé, le *thread* se met en pause
 - La méthode «**lock**» continue
 - ❑ Possibilité d'utiliser la méthode «**try_lock**» qui n'attend pas

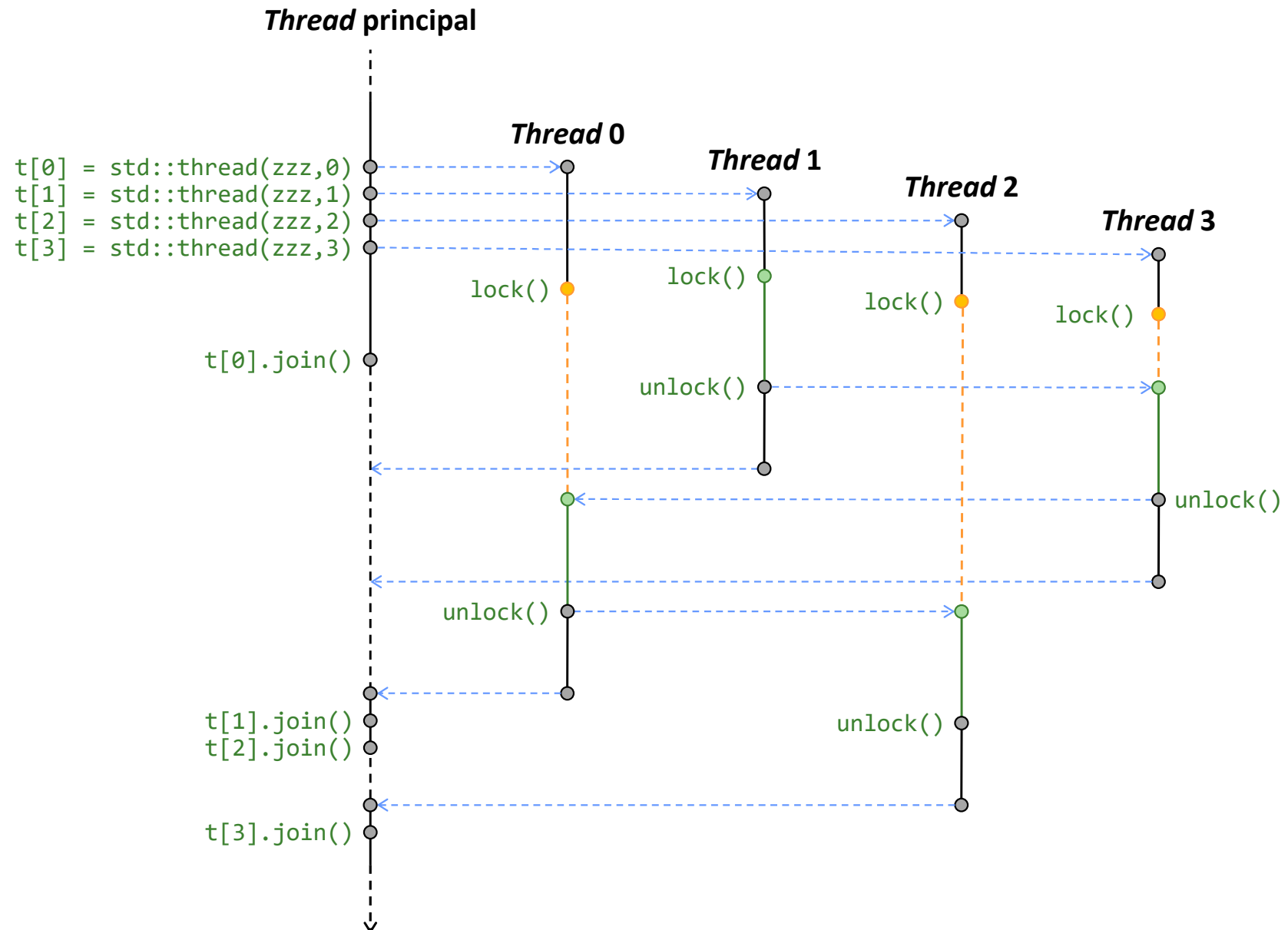
- Libération d'un *mutex* (méthode «**unlock**»)
 - Doit être déclenchée par le *thread* qui a verrouillé
 - Signal aux *threads* en attente $\Rightarrow$ l'un d'eux acquiert alors le *mutex*

- Exemple

```
std::mutex mtx;
```

```
void zzz(unsigned n) {  
    mtx.lock();  
    std::cout << "[" << n << "]" zzz..." << std::endl;  
    mtx.unlock();  
    std::this_thread::sleep_for(std::chrono::seconds(1));  
}
```

Principe du *mutex* (3/3)



Eviter l'interblocage de *threads* (1/2)

- Attention: oubli de libération $\Rightarrow$ blocage potentiel
 - *Thread* 1: verrouillage du *mutex* *m*
 - *Thread* 2: demande d'acquisition de *m* $\Rightarrow$ attente libération *m*
 - *Thread* 1: fin (sans libérer *m*)
 - *Thread* principal: attente fin de *thread* 2...

- Pour éviter les blocages $\Rightarrow$ toujours utiliser un «verrou»
 - Il s'agit d'un proxy implémentant l'idiome RAI
 - Proxy = objet (verrou) qui encapsule un objet (*mutex*)
 - Il se fait passer pour l'objet $\Rightarrow$ interface similaire
 - Il contrôle l'appel à ses méthodes
 - RAI (*Resource Acquisition Is Initialization*)
 - Le constructeur acquiert le *mutex*
 - Le destructeur libère le *mutex*
 - Réduction du risque de blocage des *threads*

Eviter l'interblocage de *threads* (2/2)

■ Plusieurs types de verrous

□ `scoped_lock`

- Interface restreinte: appel explicite à «`lock`» et «`unlock`» impossible
- Copie impossible

□ `unique_lock`

- Même interface que le *mutex*
- Copie impossible, mais mouvement autorisé ($\Rightarrow$ transfert de propriété)

■ Exemple

```
void zzz(unsigned n) {  
    { std::scoped_lock<std::mutex> verrou(mutex);  
      std::cout << "[" << n << "]" zzz..." << std::endl; }  
  
    std::this_thread::sleep_for(std::chrono::seconds(1));  
}
```

- Mécanisme de synchronisation des *threads*
- Attendre qu'une condition sur des données partagées se réalise
 - ❑ Attente passive du *thread* (comme l'acquisition d'un *mutex*)
 - ❑ Plusieurs *threads* peuvent attendre la même condition
 - ❑ Méthode «*wait*»
- Un *thread* qui change l'état des données surveillées émet un signal vers le ou les *threads* en attente
 - ❑ Méthodes «*notify_one*» et «*notify_all*»

- Un *mutex* doit être associé à la condition
 - ❑ Ce qui permet de synchroniser l'accès aux données partagées
 - ❑ Modification état: `lock()` → écriture → `unlock()` → signal
 - ❑ Vérification condition: signal → `lock()` → lecture → `unlock()`
- Signal émis ⇒ un ou plusieurs *threads* tentent d'acquérir le *mutex*
 - ❑ Le premier qui verrouille le *mutex* peut vérifier la condition ⇒ reprise «`wait`»
 - ❑ Condition vraie ⇒ «`wait`» se termine avec le *mutex* verrouillé
 - ❑ Condition fausse ⇒ «`wait`» libère le *mutex* et attend le prochain signal
 - ❑ Prochain signal ⇒ «`wait`» verrouille le *mutex* avant de vérifier la condition
- Exemple (1/4): variables globales

```
std::condition_variable condition;
std::mutex                  mutex;
unsigned                    compteur = 0;
```

- Exemple (2/4): 1 maître et 3 esclaves

```
int main() {  
    std::thread t[4];  
  
    t[0] = std::thread(maitre);  
    for (unsigned i = 1; i<4; ++i) t[i] = std::thread(esclave,i);  
    for (unsigned i = 0; i<4; ++i) t[i].join();  
}
```

- Exemple (3/4): les esclaves attendent une condition

```
void esclave(unsigned n) {  
    std::unique_lock<std::mutex> verrou(mutex);  
    std::cout << "[" << n << "]" attend..." << std::endl;  
    condition.wait(verrou,[] () { return compteur==5; });  
    std::cout << "[" << n << "]" termine" << std::endl;  
}
```

- Exemple (4/4): le maître modifie les données liées à la condition

```
void maitre() {  
    for (unsigned i = 0; i<5; ++i) {  
        { std::scoped_lock<std::mutex> verrou(mutx);  
          std::cout << "[0] compteur = " << compteur << std::endl;  
        }  
  
        std::this_thread::sleep_for(std::chrono::seconds(1));  
  
        { std::scoped_lock<std::mutex> verrou(mutx);  
          ++compteur; }  
  
        condition.notify_all();  
    }  
}
```


- Couche pour masquer les mécanismes *multithread*
 - ❑ Simplifier le code
 - ❑ Eviter les interblocages
 - ❑ Garantir l'attente de la fin des *threads*

- Fonction «*async*»
 - ❑ Exécution asynchrone d'un *callable*
 - ❑ Création et démarrage automatique d'un *thread*
 - ❑ Garantie de l'attente de la fin du *thread*

- Objets «*future*» et «*promise*»
 - ❑ Mécanisme de synchronisation
 - ❑ Attente des résultats d'un *thread*

- Syntaxe similaire au constructeur d'un *thread*
 - Arguments: politique d'asynchronisme + *callable*
 - Politique d'asynchronisme
 - `std::launch::async`: lancement sur un nouveau *thread*
 - `std::launch::deferred`: lancement en mode «*lazy*»
- Exemple...

```
for (unsigned i = 0; i<4; ++i)
    std::async(std::launch::async, zzz, i);
```
- ...qui ne fait pas ce qu'on pense
 - «*async*» retourne un objet «*future*»
 - Son destructeur attend la fin du *thread* (équivalent de «*join*»)
 - Dans l'exemple, l'exécution est donc synchrone !!!

- Eviter de laisser un objet «*future*» dans une *rvalue*
 - ❑ Destruction immédiate $\Rightarrow$ synchronisation
- Stocker l'objet «*future*» dans une variable locale
 - ❑ Variable détruite à la fin du bloc d'instructions
 - ❑ Donc bien choisir l'endroit de la déclaration

- Exemple

```
{  
    std::future<void> f[4];  
  
    for (unsigned i = 0; i<4; ++i)  
        f[i] = std::async(std::launch::async, zzz, i);  
  
    // Attente des threads à la fin du bloc  
}
```

- Représente la valeur retournée par l'exécution d'un *thread*
 - ❑ Encapsule le mécanisme de synchronisation du *thread*
 - ❑ L'attente de la fin du *thread* est prise en charge

- Méthode «*wait*»
 - ❑ Attend que le résultat soit disponible
 - ❑ Autrement dit, que le *thread* se termine

- Méthode «*get*»
 - ❑ Retourne le résultat une fois qu'il est disponible
 - ❑ Attend aussi que le *thread* se termine

- Si aucune des deux méthodes n'est appelée, le destructeur attend le résultat
 - ❑ Afin d'être sûr que la fin du *thread* est toujours attendue

■ Exemple

```
double calcul(unsigned n) {  
    std::this_thread::sleep_for(std::chrono::seconds(1));  
    return (n+1)*(n+1);  
}
```

```
int main() {  
    std::future<double> f[4];  
    double somme = 0;  
  
    for (unsigned i = 0; i<4; ++i)  
        f[i] = std::async(std::launch::async, calcul, i);  
  
    for (unsigned i = 0; i<4; ++i) somme += f[i].get();  
  
    std::cout << "somme = " << somme << std::endl;  
}
```

- Permet à un *thread* de fournir un résultat avant la fin de son exécution
- Représente une valeur associée à un objet «*future*»
 - Lorsque le *thread* attribue une valeur à un objet «*promise*»
 - Avec la méthode «*set_value*»
 - L'objet «*future*» associé est informé
 - Sa méthode «*get*» ou «*wait*» en attente est débloquée

- Exemple (1/2)

```
void calcul(std::promise<double> p1, std::promise<double> p2) {  
    std::this_thread::sleep_for(std::chrono::seconds(1));  
    p1.set_value(3);  
    std::this_thread::sleep_for(std::chrono::seconds(1));  
    p2.set_value(7);  
}
```

- Pas de constructeur de copie
 - Utiliser «`std::move`» pour invoquer le constructeur de mouvement

- Exemple (2/2)

```
int main() {  
    std::promise<double> p1, p2;  
  
    std::future<double> f1 = p1.get_future();  
    std::future<double> f2 = p2.get_future();  
  
    std::future<void> f = std::async(std::launch::async, calcul,  
                                    std::move(p1), std::move(p2));  
  
    f1.wait();  
    std::cout << "valeur1 = " << f1.get() << std::endl;  
    f2.wait();  
    std::cout << "valeur2 = " << f2.get() << std::endl;  
}
```